Multi-channel AI Agent

MASTER FINAL PROJECT DOCUMENTATION

n8n • Website Chat • Email • WhatsApp • Google Sheets CRM • Central AI Brain • Knowledge Base • Human Escalation

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Platform	n8n + Docker + Google Sheets + AI / APIs
Supported Channels	Website Chat, Email, WhatsApp
Documentation Scope	Phases 1-10 + implementation screenshots + troubleshooting + portfolio summary
Security Note	Credentials, access tokens and secrets are intentionally not reproduced in this master documen

This master PDF combines the detailed phase archives already created during the internship with the latest integrated pipeline, Phase 9 architecture, Phase 10 testing plan, and portfolio-ready documentation.

1. Executive Project Summary

The project is a centralized multi-channel AI customer-service automation built in n8n. Website Chat, Email and WhatsApp all feed into standardized customer data, a simulated CRM in Google Sheets, centralized conversation history and one shared Central AI Brain. The AI classifies intent, retrieves approved knowledge, produces a channel-appropriate response and can stop automation for human handoff when escalation is required.

Website Chat / Email / WhatsApp → Standardize Incoming Data → Identify or Create Customer → Retrieve Previous Context → Store Conversation → Central AI Brain → Intent / Action → Knowledge Base → Human Escalation Gate → Channel Response Switch → Website / Email / WhatsApp → Update CRM + Conversation History

Core components used

Component	Role
n8n	Main workflow orchestration platform.
Docker	Local n8n runtime and repeatable development environment.
Website Chat + Webhook	Website-side incoming customer messages and HTTP response delivery.
Gmail / Email workflow	Incoming email processing, replies, drafts and escalation notification.
Meta WhatsApp Cloud API	Live WhatsApp inbound/outbound customer channel.
Google Sheets CRM	Customers, conversations, knowledge data, channel rules and test data.
Central AI Brain	Shared AI classification, decision structure, intent routing and final response logic.
Knowledge Base	Grounded product, pricing, technical, order, complaint and general business information.
Cross-channel context	Customer ID based retrieval of previous conversations across channels.
Human Escalation	AI decision → IF branch → summary → internal email → CRM status → handoff response.
ngrok / public tunnel	Public webhook connectivity for local n8n and Meta callbacks during development.

2. Phase-by-Phase Master Index

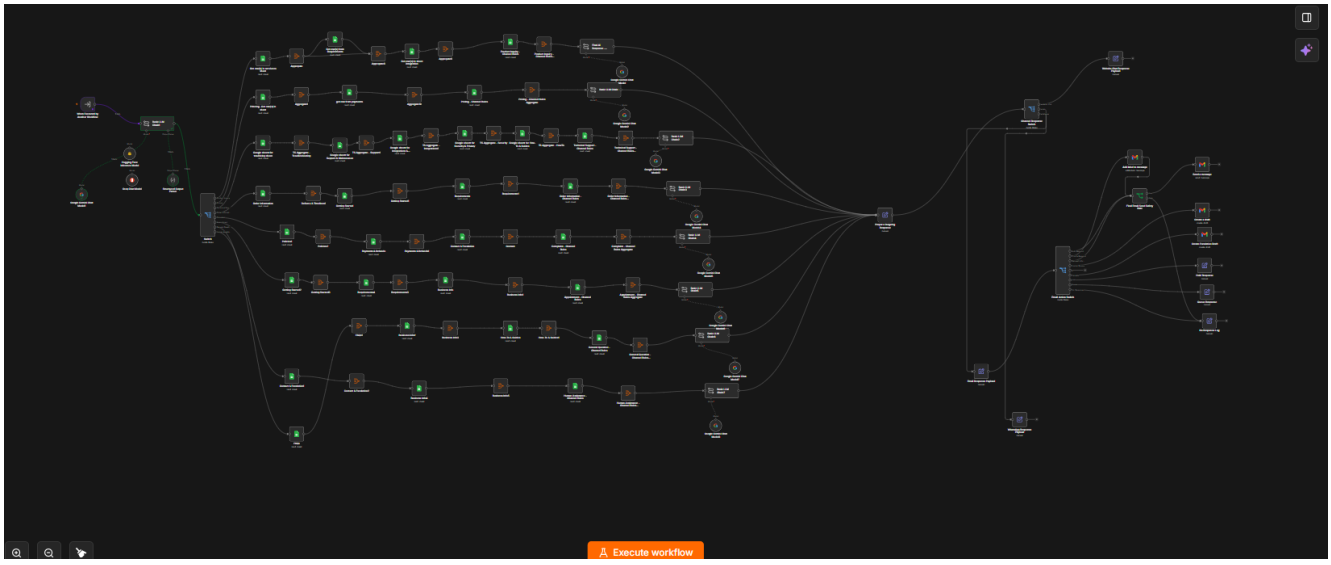
Phase	Main objective	Master status
1	Channel connectivity and initial Website / Email / WhatsApp webhook foundation.	Archived in detail
2	Collect and standardize every channel into one message schema.	Archived in detail
3	Unified customer profile, simulated CRM, duplicate prevention and conversation storage.	Archived in detail
4	Central AI Brain shared by all channels with structured intent decisions.	Archived in detail
5	Centralized knowledge base and grounded responses.	Archived in detail
6	Live WhatsApp Cloud API integration and automated response.	Archived in detail
7	Cross-channel conversation context and shared customer history.	Archived in detail
8	AI routing, human handoff and escalation.	Archived in detail
9	Complete centralized multi-channel pipeline combining all prior components.	Integrated architecture documented below
10	Testing, optimization, documentation and final demonstration.	30-scenario QA matrix prepared; final exe

The following merged sections preserve the full detailed phase reports, including implementation notes, commands, node mappings, errors, fixes, and screenshots created during the project.

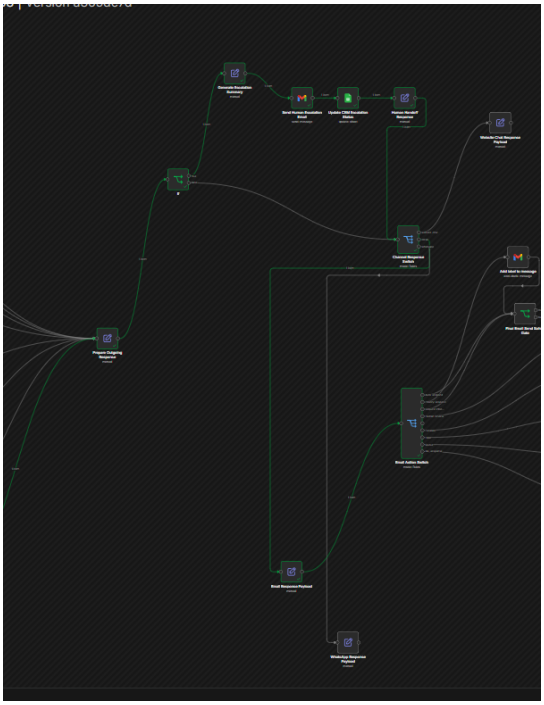
PHASE 9 - Complete Multi-channel AI Pipeline

Phase 9 does not restart the project. It combines the existing channel, CRM, context, AI, knowledge-base and escalation components into one complete centralized pipeline.

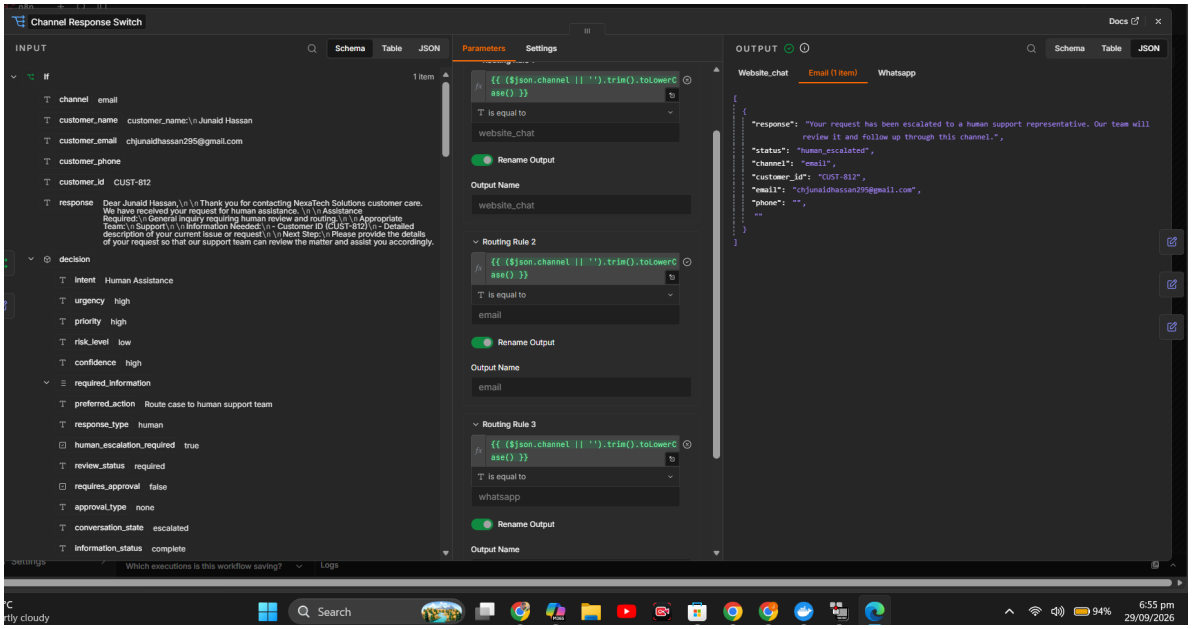
Step	Implementation in this project	Status
1. Incoming message	Website Chat, Email or WhatsApp workflow receives the message.	Implemented
2. Identify channel	channel is standardized as website_chat, email or whatsapp.	Implemented
3. Standardize information	Edit Fields nodes normalize identity, message, channel, timestamp and message ID.	Implemented
4. Customer profile	Google Sheets lookup + IF + update/append identifies or creates the customer.	Implemented
5. Conversation history	Get Previous Conversations + Build Previous Context retrieves centralized history.	Implemented
6. Complete AI context	Call Central AI Brain sends customer ID, identity, channel, message and prior context.	Implemented
7. Intent/action	Structured AI decision identifies intent, urgency, action, reply mode and escalation requirement.	Implemented
8. Knowledge retrieval	Intent branches retrieve approved knowledge/rules for grounded responses.	Implemented
9. Correct-channel response	Channel Response Switch routes Website, Email or WhatsApp output.	Implemented
10. Database update	Customer and conversation records are stored/updated centrally.	Implemented



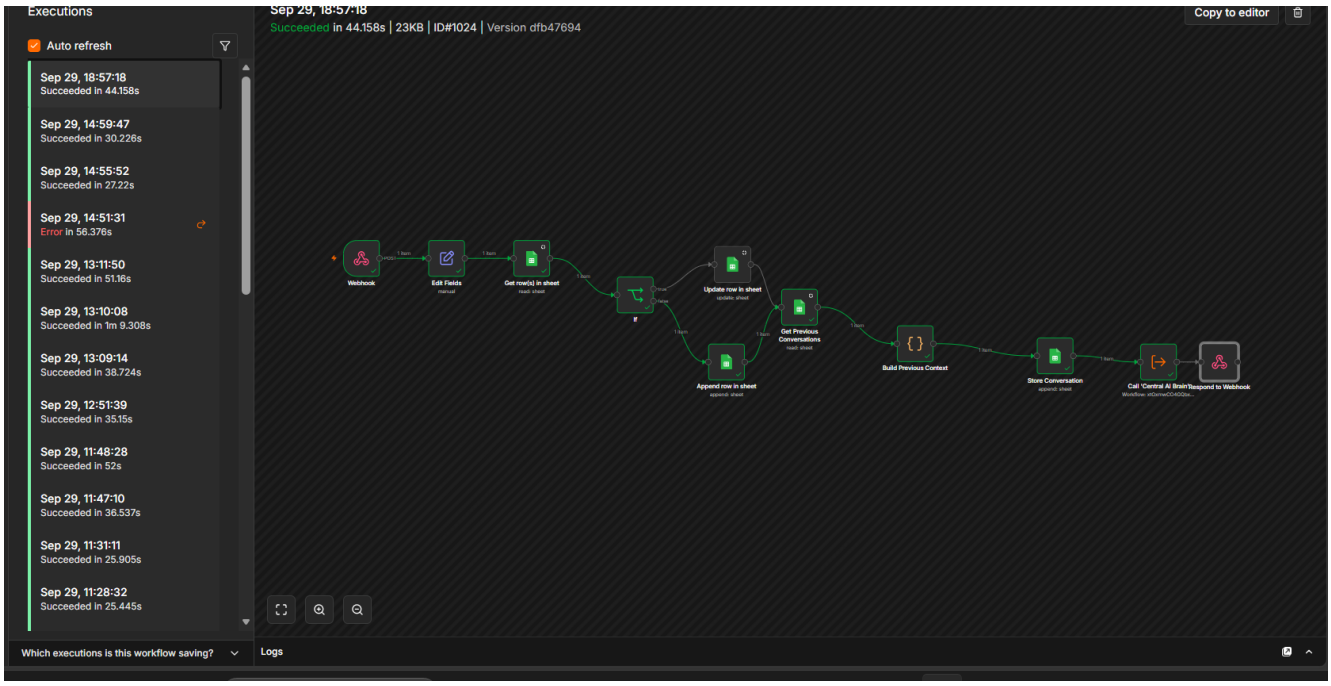
Integrated Central AI Brain - shared AI workflow used by all supported channels.



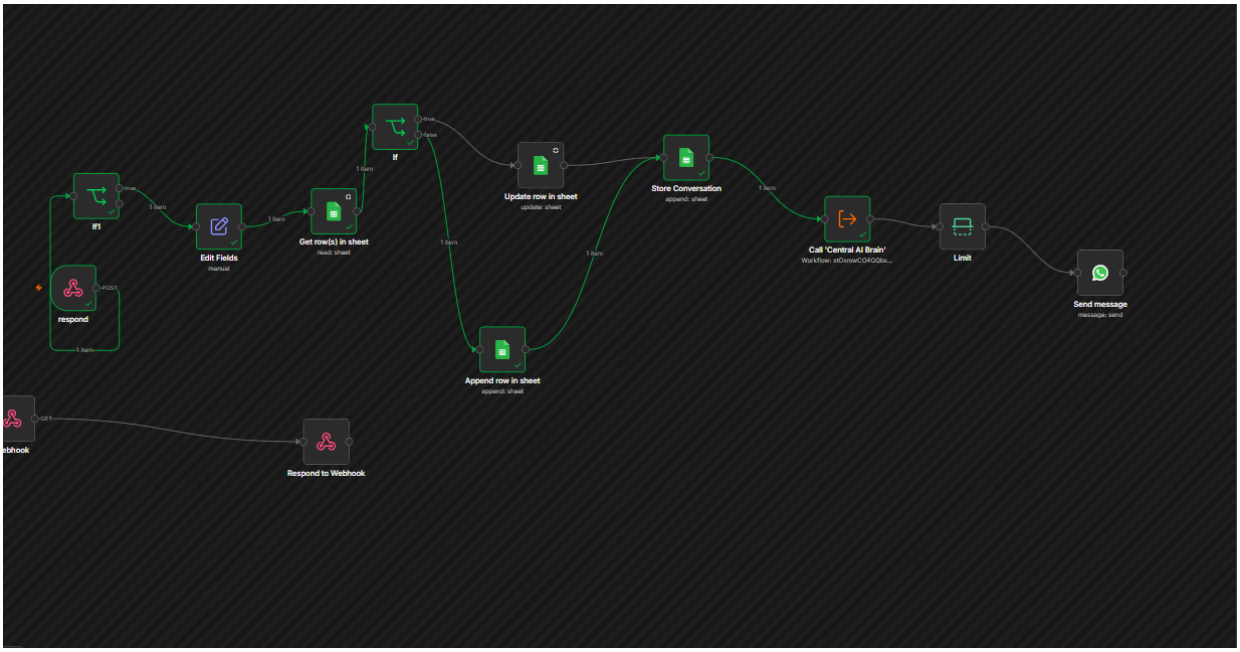
Human escalation true path and normal false path integrated into the centralized response flow.



Channel Response Switch routes the prepared payload to Website Chat, Email or WhatsApp.



Successful end-to-end Website workflow execution reaching Call Central AI Brain and Respond to Webhook.



WhatsApp workflow connected to the centralized AI Brain, Limit node and final Send message node.

PHASE 10 - Testing, Optimization, Documentation and Final Demonstration

Phase 10 is the final QA and portfolio phase. The automation architecture already exists; this phase proves stability using a structured test matrix, edge cases, error handling and a final multi-channel demonstration.

Requirement	Current project preparation
30 fictional scenarios	Prepared: 10 Website Chat + 10 Email + 10 WhatsApp scenarios.
New + returning customers	Included in the scenario matrix.
Cross-channel switching	Included using repeated fictional identities across Website, Email and WhatsApp.
Intent coverage	Product, pricing, technical support, complaint and general inquiries included.
Missing knowledge	Dedicated unknown-service/product cases included to verify no hallucination.
Human escalation	Explicit human request and complaint escalation cases included for each channel.
Duplicate/incomplete identity	Duplicate email/phone and missing-field scenarios included.
Error handling / cleanup	Existing troubleshooting, branch gating, duplicate prevention and safe handoff logic documented.
Final demonstration	Plan: one fictional customer moves through all three channels while preserving one centralized profile/

Important accuracy note: the 30-case matrix is prepared for final QA. The actual-result and PASS/FAIL columns should be filled from real n8n executions before the internship presentation is treated as fully complete.

Phase 10 Test Matrix - Scenarios 1-10

ID	Customer	Channel	Type	Test message	Expected inten	Expected resu
P10-01	Ali Khan	website_chat	New	What services does NexaTech Solutions provide?	Product Inquiry	KB answer
P10-02	Sara Malik	website_chat	New	What is the price of your basic package?	Pricing	Pricing answer
P10-03	Ahmed Raza	website_chat	New	My website is not loading properly. Can you help?	Technical Support	Troubleshooting
P10-04	Hina Noor	website_chat	New	I am unhappy because my issue is still not solved.	Complaint	Human escalation
P10-05	Usman Tariq	website_chat	New	What are your business working hours?	General Question	General answer
P10-06	Bilal Ahmed	website_chat	New	Do you provide blockchain auditing services?	Unknown KB	Safe fallback
P10-07	Ayesha Khan	website_chat	New	I want to speak with a human representative.	Human Assistance	Human escalation
P10-08	Ali Khan	website_chat	Returning	Can you continue from my previous conversation?	Context	Use history
P10-09	Unknown Customer	website_chat	Incomplete	Please tell me your premium package price.	Pricing	Handle missing email
P10-10	Sameer Ali	website_chat	Duplicate	I think I already registered with this email before.	Duplicate	Reuse profile

Phase 10 Test Matrix - Scenarios 11-20

ID	Customer	Channel	Type	Test message	Expected inten	Expected resu
P10-11	Ali Khan	email	Cross-channel	I asked about your services on website chat. What was the package?	Product Inquiry	Use website history
P10-12	Maryam Iqbal	email	New	Please send details of your premium plan.	Product Inquiry	KB answer
P10-13	Fahad Sheikh	email	New	How much does your enterprise package cost?	Pricing	Pricing answer
P10-14	Zoya Ahmed	email	New	I cannot access my account dashboard.	Technical Support	Troubleshooting
P10-15	Hamza Javed	email	New	Third time asking; my problem is still unresolved.	Complaint	Human escalation
P10-16	Noor Fatima	email	New	Where is your company located?	General Question	General answer
P10-17	Daniyal Hussain	email	New	Do you offer quantum computing consulting?	Unknown KB	Safe fallback
P10-18	Maha Ali	email	New	Please connect me with your manager.	Human Assistance	Human escalation
P10-19	Sara Malik	email	Cross-channel	I asked about pricing earlier on your website. Can you continue?	Pricing	Use website history
P10-20	Incomplete Email Customer	email	Incomplete	I need technical support for my account.	Technical Support	Ask only needed info

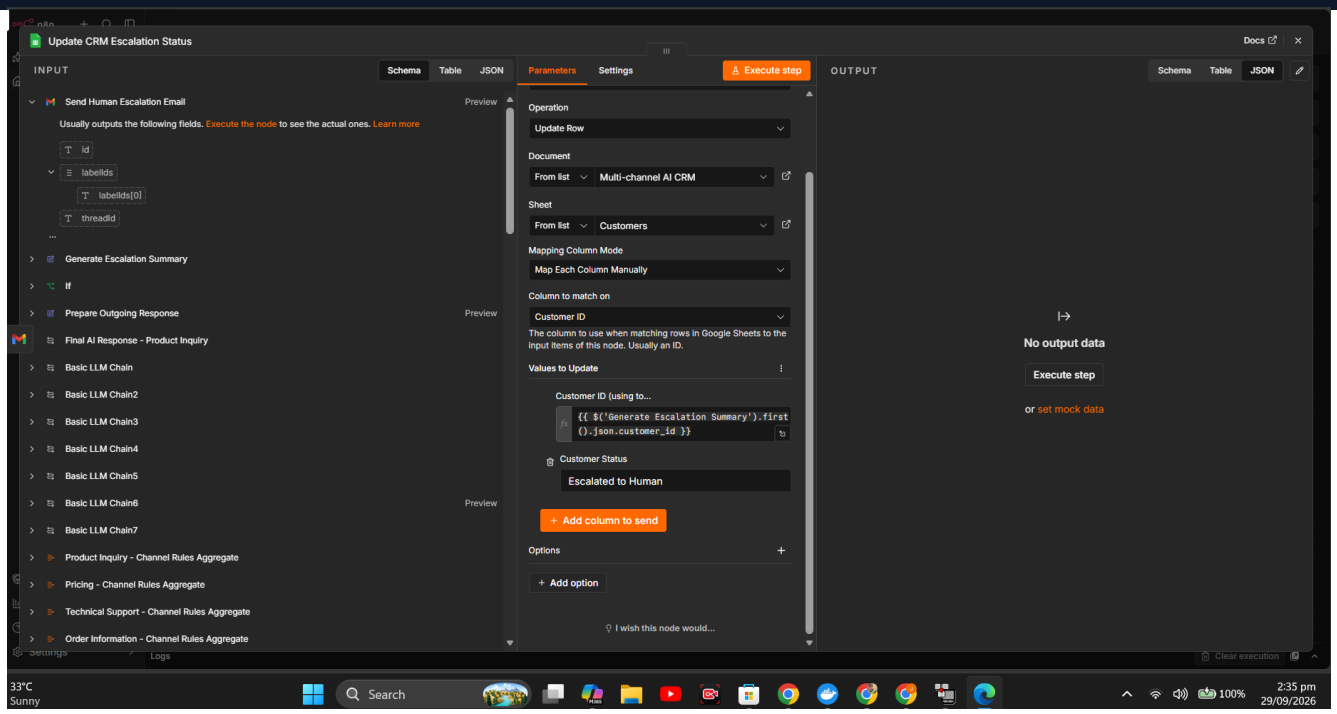
Phase 10 Test Matrix - Scenarios 21-30

ID	Customer	Channel	Type	Test message	Expected inten	Expected resu
P10-21	Ahmed Raza	whatsapp	Cross-channel	I talked to you on website about my technical issue. Another solution?	Technical Support	Use website history
P10-22	Areeba Hassan	whatsapp	New	What products do you offer?	Product Inquiry	KB answer
P10-23	Imran Shah	whatsapp	New	What is your monthly subscription price?	Pricing	Pricing answer
P10-24	Laiba Noor	whatsapp	New	My integration keeps failing. What should I do?	Technical Support	Troubleshooting
P10-25	Omer Khan	whatsapp	New	Your service has been very disappointing and I want this fixed now.	Complaint	Human escalation
P10-26	Iqra Tariq	whatsapp	New	Can you tell me your support timings?	General Question	General answer
P10-27	Abdullah Saeed	whatsapp	New	Do you support a product called NexaQuantum Pro?	Unknown KB	Safe fallback
P10-28	Hassan Ali	whatsapp	New	I need a real person. Please stop the bot.	Human Assistance	Human escalation
P10-29	Maryam Iqbal	whatsapp	Cross-channel	I emailed you about the premium plan. Can we continue here?	Product Inquiry	Use email history
P10-30	Duplicate Ali	whatsapp	Duplicate	This is Ali again. I am contacting you from WhatsApp now.	Duplicate	Reuse profile

Final Demonstration Script

Use one fictional customer with the same identity across three channels. This demonstrates the exact Phase 10 requirement without using real customer data.

1) Website Chat: customer asks about a product. Show standardized input, CRM profile creation, Central AI Brain intent, knowledge retrieval and website reply.
2) Email: same customer follows up about pricing. Show the existing Customer ID and previous website context being retrieved.
3) WhatsApp: same customer reports a problem or asks for a human. Show the same profile/history, human_escalation_required = true, escalation email, CRM status update and WhatsApp handoff response.



CRM escalation update uses Customer ID as the match key and changes only Customer Status, protecting other profile fields.

3. Key Troubleshooting and Engineering Lessons

Webhook / tunnel connectivity

Local n8n webhooks required a public tunnel for Meta and external website access. ngrok/Cloudflare connectivity was tested and callback URLs were updated after tunnel changes.

Standardization errors

Incorrect field names such as name vs sender_name and phone mapped from email produced blank/incorrect CRM data. Standardized sender_* mappings fixed the issue.

Sub-workflow no output

The Central AI Brain could complete internally but return no parent item when channel was empty or a switch route did not match. Correct parent input mapping solved this.

Boolean IF type

human_escalation_required is a boolean; configuring the IF node as a string produced a type error. Boolean/is true fixed the branch.

Duplicate responses

Duplicate connections or parallel output paths can send repeated messages. The response graph was reduced to one normal path and one controlled escalation path.

CRM data protection

Updating a row with blank mapped fields can overwrite existing customer data. The escalation update was reduced to Customer ID + Customer Status only.

Provider/model availability

During development, external AI providers returned temporary capacity/access/model compatibility errors. Workflow logic was separated from provider availability so failures could be isolated.

Security

Access tokens and credentials shown during development should be rotated when exposed and are intentionally omitted/redacted from final documentation.

4. Portfolio-Ready Project Summary

Built a complete Multi-channel AI Agent in n8n connecting Website Chat, Email and WhatsApp to one centralized AI workflow. The system standardizes inbound messages, identifies or creates customer profiles, stores data in a simulated Google Sheets CRM, retrieves cross-channel conversation history, detects intent, retrieves grounded knowledge, routes channel-specific responses and escalates complex cases to a human representative. The project includes live WhatsApp integration, webhook/API work, AI routing, duplicate handling, CRM updates, cross-channel context, human handoff, troubleshooting and a structured 30-scenario final QA plan.

Master document assembly note

After these master pages, the complete detailed Phase 1-8 archives are appended in order. These archives contain the implementation screenshots, commands, node configurations, errors and fixes recorded during development. A portfolio/LinkedIn visual summary is appended at the end.

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 1

n8n Setup and Multi-channel Architecture

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	COMPLETE

Foundation: Docker, n8n, webhooks and independent Website/Email/WhatsApp tests.

1. Phase 1 Objective and Final Architecture

Phase 1 built the complete development foundation. The goal was not to build the full AI agent yet; it was to prove that Website Chat, Email and WhatsApp could independently send customer data into one n8n development environment.

```
Website Chat \
Email ---> n8n ---> Future Central AI Brain
WhatsApp /
```

Environment created

• Windows + WSL 2 / Ubuntu support prepared for Docker Desktop.

• Official n8n Docker image used instead of relying on a fragile direct Windows install.

• Persistent n8n data volume configured so workflows survive container restarts.

• Local n8n opened at http://localhost:5678, using port 5678 and Asia/Karachi timezone configuration.

Docker / n8n commands that mattered

```
docker --version
docker run hello-world
docker pull n8nio/n8n:latest
docker ps -a
docker start multichannel-ai-agent-n8n
```

Workflows created

Workflow	Purpose
Multi-channel AI Agent	Main project container / architecture
Website Chat - Test	Receive fictional website chat data
Email - Test	Receive fictional email data
WhatsApp - Test	Receive fictional WhatsApp data

Webhook simulation

```
Website Chat: POST /website-chat
Email: POST /email-incoming
WhatsApp: POST /whatsapp-incoming
```

Major troubleshooting learned

• Virtualization / WSL / Virtual Machine Platform issues were resolved before Docker could run reliably.

• PowerShell variable syntax pasted into CMD failed; CMD-compatible curl was used instead.

• n8n test webhooks expire: click “Listen for test event” again and immediately send the request.

• ERR_EMPTY_RESPONSE immediately after container start can be a startup-delay issue; wait and retry.

Phase 1 final proof: all three channels were independently received in n8n. The original detailed learning notes and screenshots are appended after this summary.

PHASE 1

Multi-channel AI Agent

Complete Learning Notes - Easy Roman Urdu

n8n | Docker | WSL 2 | Ubuntu | Webhooks | HTTP POST | JSON | curl | Multi-channel Architecture

Ye notes sirf assignment proof nahi hain. Inka maqsad Phase 1 ke har concept ko samajhna, revise karna aur baad me khud se repeat kar pana hai.

Fictional Business: NexaTech Solutions

Channels: Website Chat + Email + WhatsApp

Contents

- 1. Project ka overview aur Phase 1 ka objective
- 2. Phase 1 ke 10 steps aur completion status
- 3. n8n kya hai aur automation ka basic idea
- 4. Docker, Image aur Container
- 5. WSL 2, Ubuntu, Virtualization aur Windows Features
- 6. localhost, port 5678, Volume aur Environment Variables
- 7. n8n ko start/run karne ka complete flow
- 8. Workflow kya hota hai aur humne kya banaya
- 9. Fictional business aur services
- 10. Multi-channel ka concept
- 11. Webhook kya hota hai
- 12. HTTP GET/POST aur JSON
- 13. curl aur test requests
- 14. Website Chat workflow test
- 15. Email workflow test
- 16. WhatsApp workflow test
- 17. Listen for test event aur webhook testing
- 18. Architecture aur Centralized AI Brain
- 19. Real integration vs simulation
- 20. Troubleshooting aur errors
- 21. Documentation, screenshots aur LinkedIn showcase
- 22. Phase 1 ke baad tumhe khud kya aana chahiye
- 23. Quick revision glossary
- 24. Self-test questions
- 25. Phase 1 final checklist aur next phase preview

1. Project ka Overview aur Phase 1 ka Objective

Project ka naam Multi-channel AI Agent hai. Is project ka final idea ye hai ke different communication channels se aane wale customer messages ek central automation system me aayen, phir unko AI samjhe, customer context maintain ho, required action decide ho aur zaroorat par human handoff bhi ho.

```
Website Chat ─┐
Email ────────┼──> n8n ──> Central AI System
WhatsApp ─────┘
```

Phase 1 me humne poora AI system nahi banaya. Phase 1 ka kaam foundation banana tha: development environment, channels, webhook simulation, separate test workflows, architecture aur independent testing.

Phase 1 ka main objective: prove karna ke Website Chat, Email aur WhatsApp se customer data n8n tak successfully aa sakta hai.

2. Phase 1 ke 10 Steps aur Completion Status

Step	Kaam	Status
1	n8n development environment create/access	Complete
2	Multi-channel AI Agent organize karna	Complete - Personal workspace workaround
3	Fictional business select karna	NexaTech Solutions
4	Products/services define karna	Complete
5	Kam az kam 3 channels select karna	Complete
6	Website Chat, Email, WhatsApp use karna	Complete
7	External channels ki jagah Webhook simulation	Complete
8	Simple architecture diagram	Complete
9	Har channel ke liye separate test workflow	Complete
10	Har workflow ko independently test karna	Complete

Note: Local Community edition me “New project” feature Enterprise-only tha, isliye main workflow aur test workflows Personal workspace me organize kiye gaye. Functionality par koi asar nahi pada.

3. n8n kya hai?

n8n ek workflow automation platform hai. Ye different apps, APIs, databases, forms, emails, webhooks aur AI services ko connect karke repetitive kaam automate kar sakta hai.

```
Customer message
    ↓
```

```
n8n receive karta hai
    ↓
Data process hota hai
    ↓
AI / CRM / Reply / Routing
```

Simple example

Agar manually kaam ho to tum email check karte ho, customer ka data copy karte ho, sheet me save karte ho aur reply likhte ho. n8n in steps ko workflow ke through automatically kar sakta hai.

n8n ko “automation manager” samjho: trigger milta hai, phir predefined steps execute hote hain.

4. Docker, Image aur Container

Docker kya hai?

Docker software ko isolated environment me run karne ka platform hai. Is isolated environment ko container kehte hain. Humne n8n ko direct Windows me install karne ke bajaye Docker container me run kiya.

```
Windows Laptop
  └── Docker Desktop
      └── n8n Container
          └── n8n Application
```

Docker Image kya hai?

Image software ka ready-made template hoti hai. Humne official n8n image download ki:

```
docker pull n8nio/n8n:latest
```

Image ko recipe samjho aur container ko us recipe se banne wali actual running dish.

Container kya hai?

Container image ka running instance hota hai. Humara container naam:

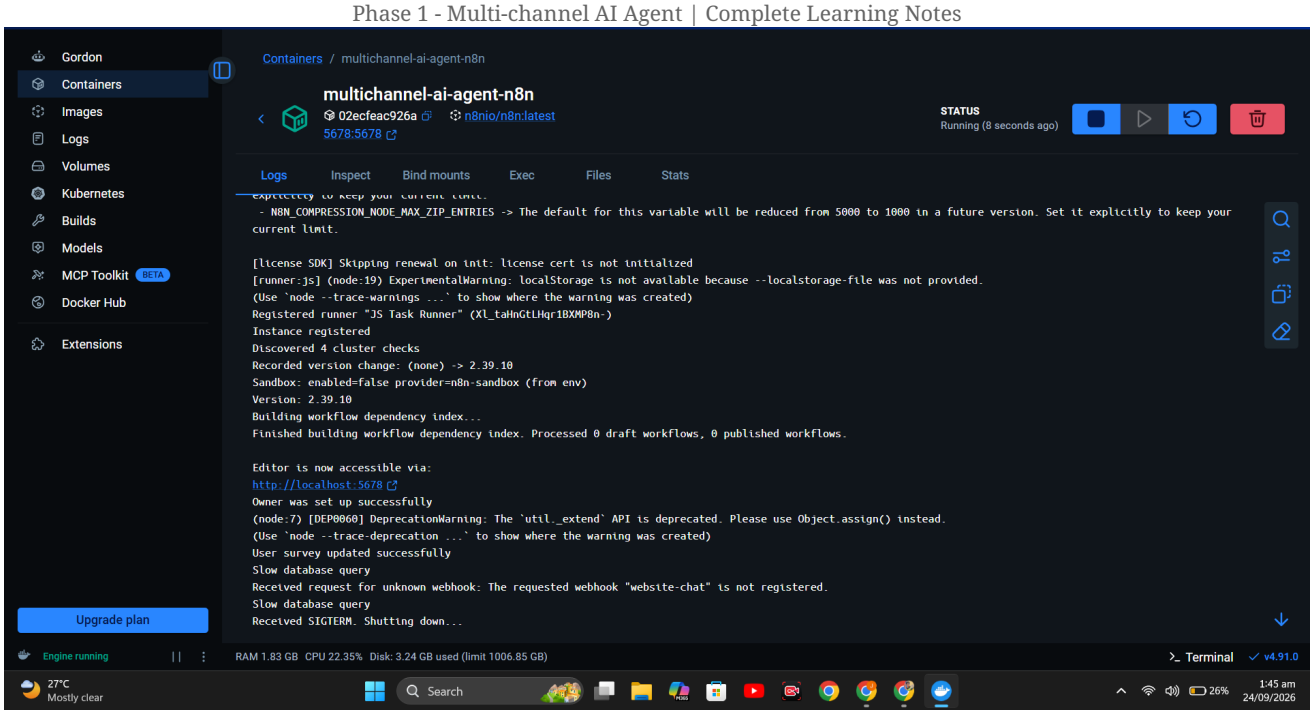
```
multichannel-ai-agent-n8n
```

Docker test: hello-world

Docker installation check karne ke liye humne:

```
docker run hello-world
```

Agar “Hello from Docker!” aaye to basic Docker engine working hota hai.



Docker Desktop me n8n container running state ka example.

5. WSL 2, Ubuntu, Virtualization aur Windows Features

WSL 2 kya hai?

WSL ka full form Windows Subsystem for Linux hai. Is se Windows ke andar Linux environment run kiya ja sakta hai. Docker Desktop Linux containers ke liye WSL 2 backend use kar sakta hai.

```
Windows
└── WSL 2
    └── Ubuntu
```

Ubuntu kya hai?

Ubuntu Linux operating system/distribution hai. Humne laptop ka main Windows replace nahi kiya; Ubuntu WSL ke through Windows ke andar run hua.

Virtualization kya hai?

Virtualization CPU ka feature hai jo virtual environments ko run karne me help karta hai. Task Manager me Virtualization disabled thi, isliye BIOS/UEFI me jaa kar Intel Virtualization Technology enable ki gayi.

Windows Features

- Virtual Machine Platform
- Windows Subsystem for Linux

Ye features WSL 2 aur Docker ke environment ko support karte hain.

6. localhost, Port 5678, Volume aur Environment Variables

localhost kya hai?

localhost ka matlab tumhara apna computer hota hai. Humara n8n internet ki public website nahi tha; woh local machine par run ho raha tha:

```
http://localhost:5678
```

Port 5678 kya hai?

Port ko room number samjho. Laptop building hai aur port 5678 woh room hai jahan n8n service sun rahi hai.

```
localhost = apna laptop
5678      = n8n ka port
```

Persistent Volume

Container temporary ho sakta hai. n8n ka important data safe rakhne ke liye persistent storage/volume map ki gayi, jisse workflows aur settings container restart/recreate ke baad bhi bach sakte hain.

```
Windows folder / n8n_data
      ↓
/home/node/.n8n
```

Environment Variables

Environment variables application ko startup par configuration deti hain. Example:

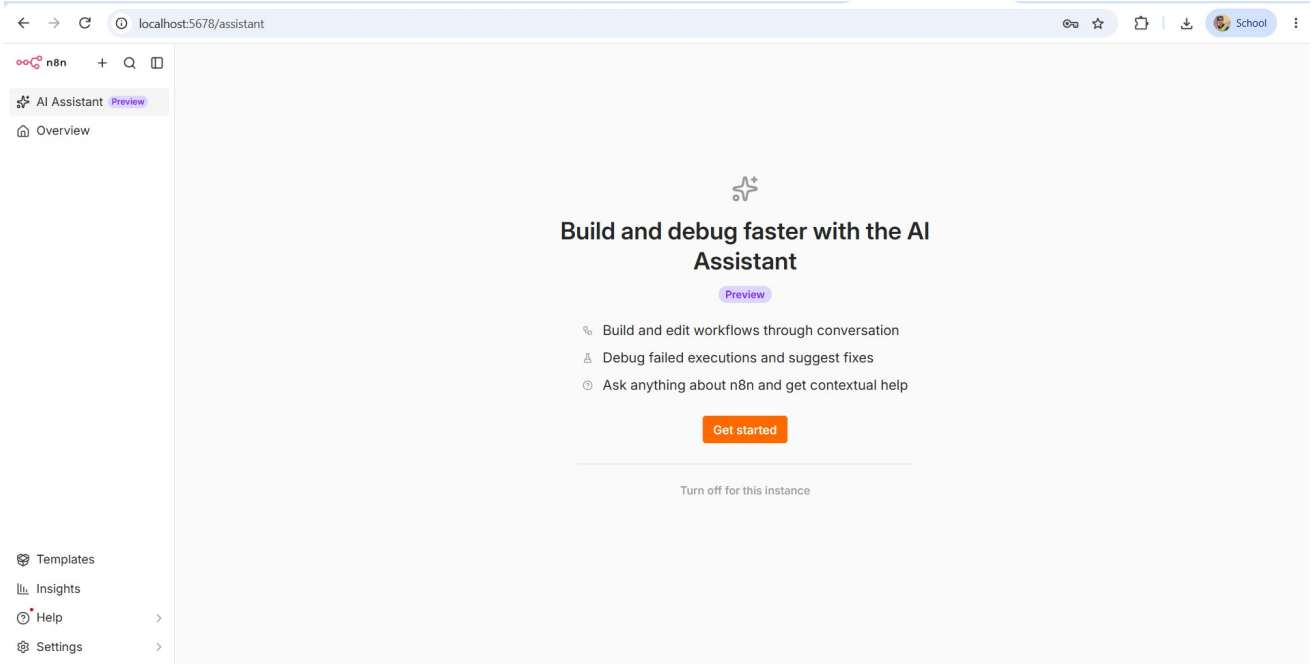
```
TZ=Asia/Karachi
GENERIC_TIMEZONE=Asia/Karachi
N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
N8N_RUNNERS_ENABLED=true
```

7. n8n ko Start/Run karne ka Complete Flow

1. Docker Desktop open karo aur Docker Engine ko running hone do.
2. Containers me multichannel-ai-agent-n8n dhoondo.
3. Agar container stopped ho to Start karo.
4. 20-30 seconds wait karo taake n8n fully initialize ho jaye.
5. Browser me http://localhost:5678 open karo.
6. n8n Overview open ho jaye to environment ready hai.

```
docker start multichannel-ai-agent-n8n
```

Note: Container “Running” show hone ke baad bhi application ko fully ready hone me kuch seconds lag sakte hain. Isi wajah se kabhi ERR_EMPTY_RESPONSE ya temporary route error aa sakta hai.



n8n local web interface browser me open.

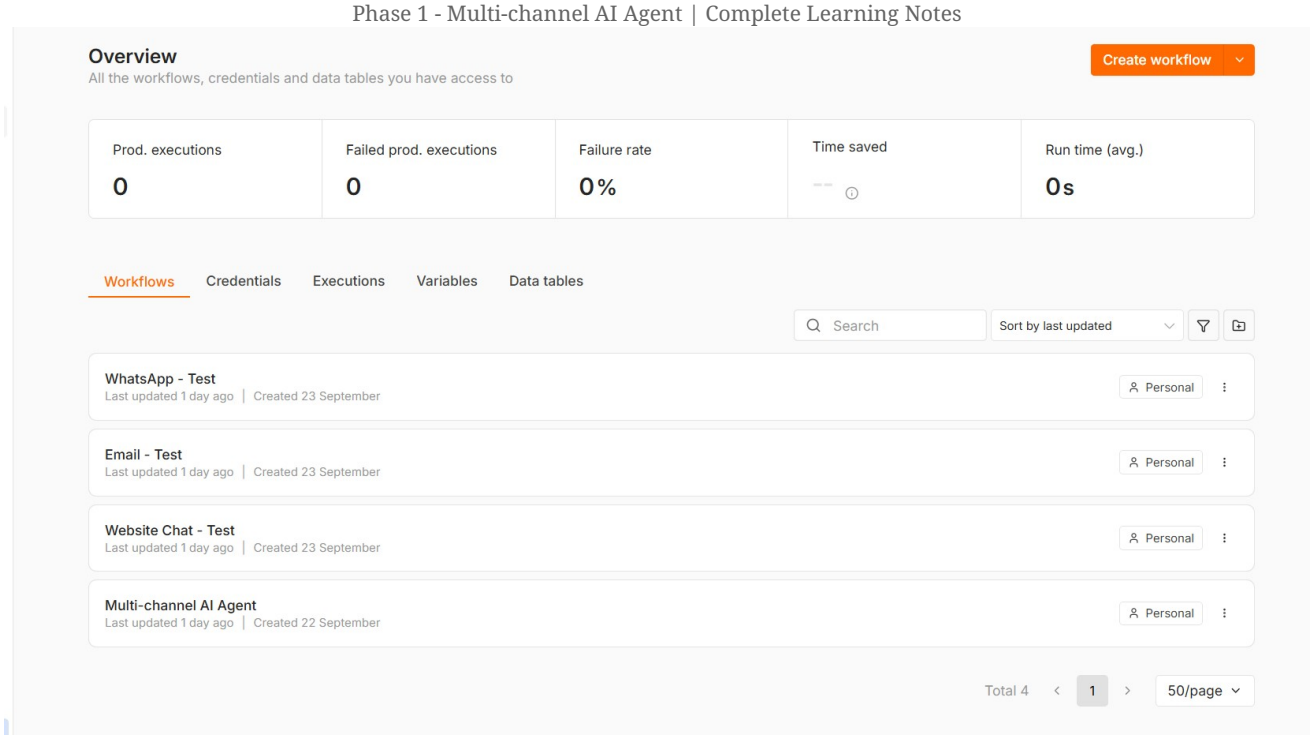
8. Workflow kya hota hai aur humne kya banaya?

n8n me automation ke complete flow ko workflow kehte hain. Workflow nodes ka sequence hota hai. Ek node trigger ho sakta hai, doosra data process karta hai, teesra API call karta hai, etc.

```
Trigger → Process → Decision → Action
```

Humne Phase 1 me ye workflows organize kiye:

```
Multi-channel AI Agent
Website Chat - Test
Email - Test
WhatsApp - Test
```



n8n Overview: Phase 1 ke workflows visible.

9. Fictional Business aur Services

Testing ke liye humne fictional company NexaTech Solutions select ki. Ye Technology & Digital Services Company hai. Fictional business use karne ka faida ye hai ke real customer data ya privacy issue ke baghair realistic automation scenarios test ho sakte hain.

- 1. Website Development
- 2. MERN Full-Stack Development
- 3. Mobile App Development
- 4. Custom Business Software Development
- 5. AI Automation Services
- 6. AI Chatbots & Virtual Assistants
- 7. n8n Workflow Automation
- 8. API Development & Integration
- 9. CRM & ERP Solutions
- 10. E-commerce Development
- 11. UI/UX Design
- 12. Cloud Deployment & Hosting
- 13. DevOps & CI/CD Setup
- 14. Database Design & Management
- 15. Cybersecurity Consulting
- 16. Web & API Security Assessment
- 17. Data Analytics & Business Dashboards
- 18. System Integration Services
- 19. Software Maintenance & Updates
- 20. Technical Support & Helpdesk Services

Business/services define karne se later AI intent, service category, pricing query, support request aur routing jaisi cheezen realistically test hongi.

10. Multi-channel ka Concept

Multi-channel ka matlab customer ek hi business se different communication channels ke through contact kar sakta hai. Humare selected channels: Website Chat, Email aur WhatsApp.

```
Website Chat ─┐
Email ────────┼──> n8n
WhatsApp ─────┘
```

Goal ye hai ke source different ho, lekin backend automation centralized ho.

11. Webhook kya hota hai?

Webhook ek URL/endpoint hota hai jahan koi external system data send karta hai. n8n me Webhook node incoming HTTP request ko receive karta hai aur workflow trigger ho jata hai.

```
External App / Test Command
        ↓
   Webhook URL
        ↓
      n8n
```

Phase 1 me real Website Chat, Gmail ya WhatsApp Business API ki jagah Webhook simulation use ki gayi.

Webhook = “data receive karne ka door”. Request aate hi workflow start ho sakta hai.

12. HTTP GET/POST aur JSON

HTTP Method kya hota hai?

HTTP methods batate hain ke request ka intent kya hai. Common methods: GET, POST, PUT/PATCH, DELETE.

Method	Simple meaning
GET	Data mangna / read karna
POST	Naya data bhejna / create/submit karna
PUT/PATCH	Existing data update karna
DELETE	Data delete karna

Humne incoming customer messages bhejne ke liye POST use kiya.

JSON kya hai?

JSON structured data format hai. Fields key-value pairs me hoti hain, isliye machine ko samajhna easy hota hai.

```
{
 "name": "Ali Khan",
 "email": "ali@example.com",
```

```
 "message": "I need a website",
 "channel": "website_chat"
}
```

13. curl kya hai aur Test Request kaise bheji?

curl command-line tool hai jo HTTP requests send kar sakta hai. Humne CMD se curl use karke test webhooks ko POST requests bheji.

```
curl -X POST URL -H "Content-Type: application/json" -d "{...json...}"
```

Is se actual website/email/WhatsApp integration ke baghair bhi n8n webhook ko realistic JSON data mil gaya.

14. Website Chat Workflow Test

Configuration

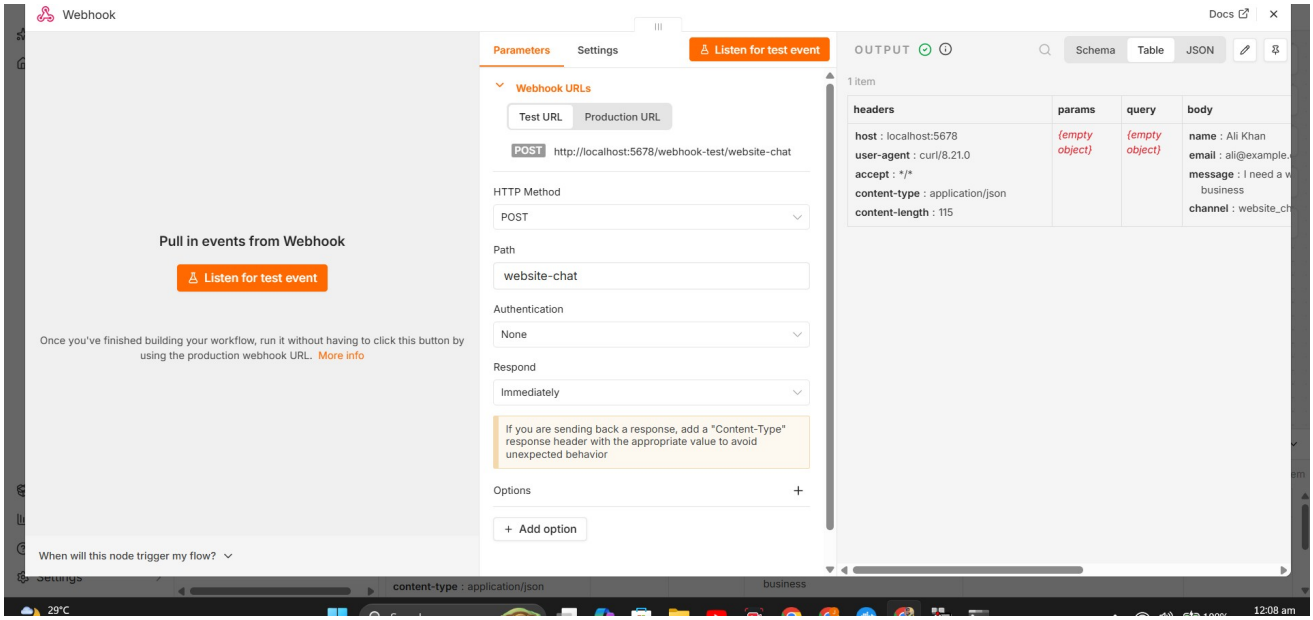
```
HTTP Method: POST
Path: website-chat
Authentication: None
Respond: Immediately
```

Test command

```
curl -X POST http://localhost:5678/webhook-test/website-chat -H "Content-Type: application/json" -d "{\"name\":\"Ali Khan\",\"email\":\"ali@example.com\",\"message\":\"I need a website for my business\",\"channel\":\"website_chat\"}"
```

Expected/received data

```
name: Ali Khan
email: ali@example.com
message: I need a website for my business
channel: website_chat
```



Website Chat test ka successful output.

Website Chat → Webhook → n8n flow successfully verify hua.

15. Email Workflow Test

Configuration

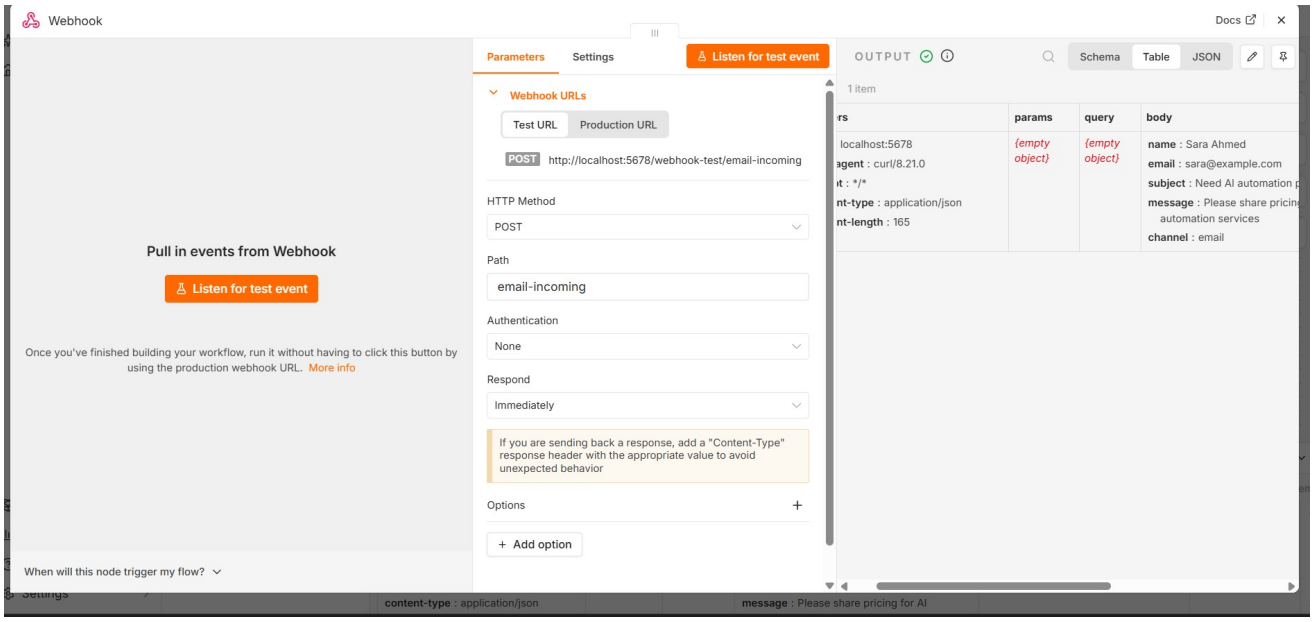
HTTP Method: POST
Path: email-incoming
Authentication: None
Respond: Immediately

Test command

```
curl -X POST http://localhost:5678/webhook-test/email-incoming -H "Content-Type: application/json" -d "{\"name\":\"Sara Ahmed\",\"email\":\"sara@example.com\",\"subject\":\"Need AI automation pricing\",\"message\":\"Please share pricing for AI automation services\",\"channel\":\"email\"}"
```

Received data

name: Sara Ahmed
email: sara@example.com
subject: Need AI automation pricing
message: Please share pricing for AI automation services
channel: email



Email simulation ka successful output.

16. WhatsApp Workflow Test

Configuration

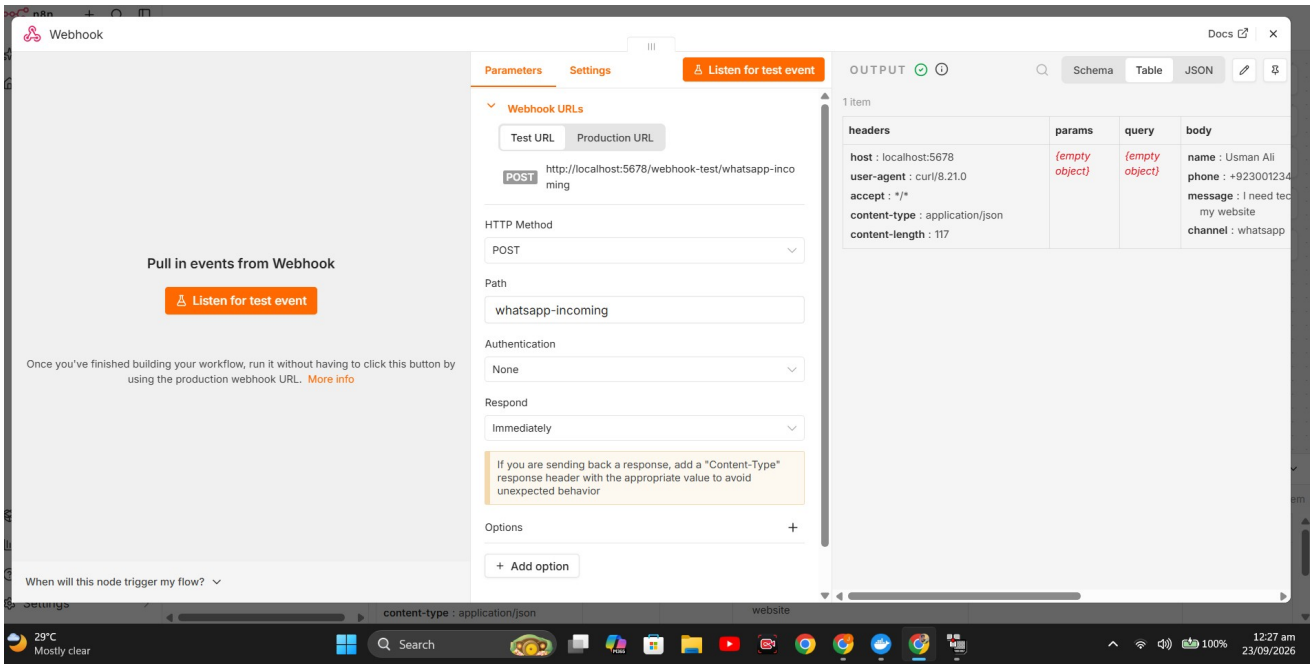
HTTP Method: POST
Path: whatsapp-incoming
Authentication: None
Respond: Immediately

Test command

```
curl -X POST http://localhost:5678/webhook-test/whatsapp-incoming -H "Content-Type: application/json" -d "{\"name\":\"Usman Ali\",\"phone\":\"+923001234567\",\"message\":\"I need technical support for my website\", \"channel\":\"whatsapp\"}"
```

Received data

```
name: Usman Ali
phone: +923001234567
message: I need technical support for my website
channel: whatsapp
```



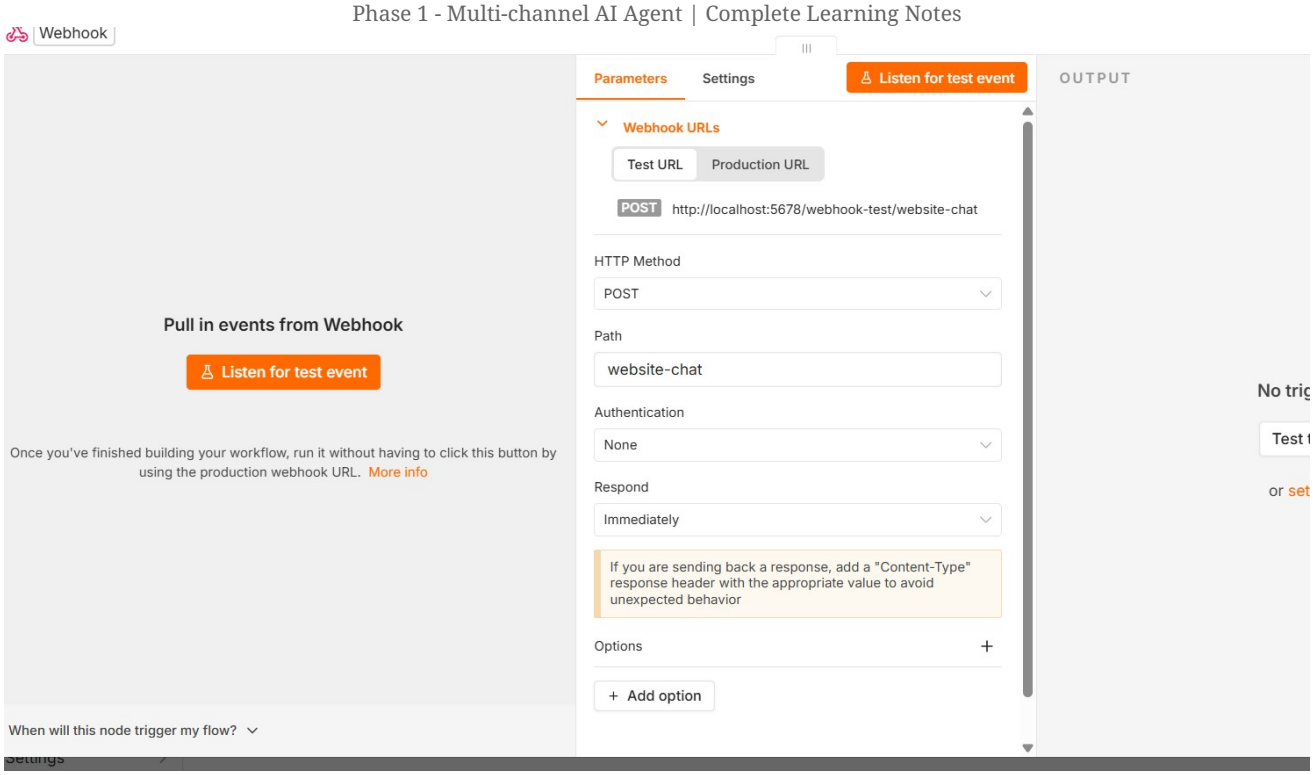
WhatsApp simulation ka successful output.

17. “Listen for test event” aur Webhook Testing

n8n ka test webhook temporary listening mode use karta hai. Jab tum “Listen for test event” click karte ho to n8n kuch time ke liye incoming test request ka wait karta hai.

1. Workflow open
2. Webhook node open
3. Listen for test event
4. CMD me curl command run
5. Output panel me JSON verify

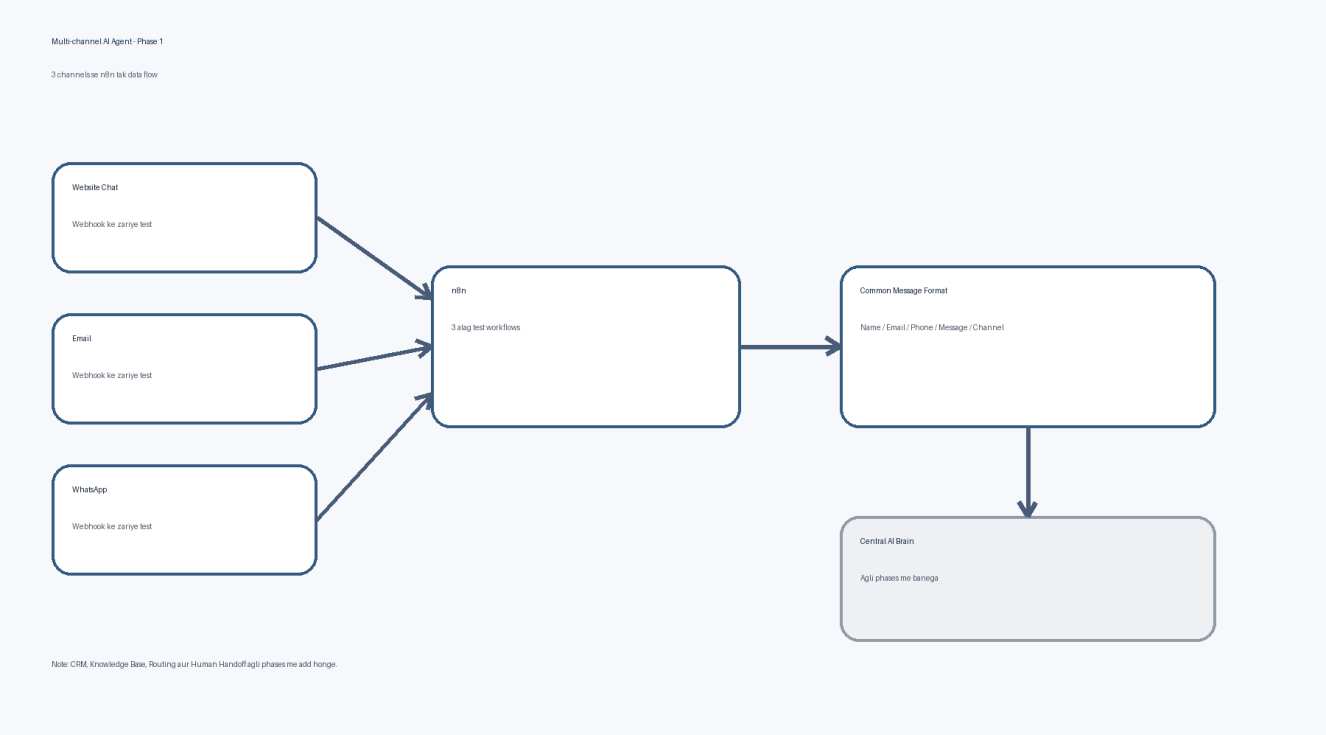
Note: Agar test URL par “webhook not registered” aaye to listener active nahi hai ya timeout ho gaya. Dobara “Listen for test event” click karke request foran bhejo.



Website Chat webhook settings ka example.

18. Architecture aur Centralized AI Brain

Architecture system ka high-level map hota hai. Phase 1 me humne channels se n8n aur future centralized AI tak flow design kiya.



Phase 1 architecture diagram.

Centralized AI ka faida

Agar har channel ka alag AI ho to logic duplicate ho sakta hai. Centralized AI me Website, Email aur WhatsApp sab ek hi business knowledge, rules aur customer context use kar sakte hain.

```
Website ─┐
Email ───┼──> ONE AI BRAIN
WhatsApp ┘
```

Future components

- Message standardization
- Central AI understanding
- CRM / customer history
- Knowledge Base
- Intelligent routing
- Automatic response
- Human handoff/escalation

Note: Ye future components Phase 1 me implement nahi hue. Phase 1 ne sirf inke liye foundation banayi.

19. Real Integration vs Simulation

Channel	Phase 1 me kya hua	Real integration abhi?
Website Chat	Webhook se fake/test message	Nahi
Email	Webhook se email data simulate	Nahi
WhatsApp	Webhook se WhatsApp data simulate	Nahi

Simulation ka purpose flow ko samajhna aur test karna tha. Later real provider/API connect ki ja sakti hai, lekin assignment ke Phase 1 ke liye simulation valid thi.

20. Troubleshooting aur Errors

Issue	Fix / Learning
Virtualization support not detected	BIOS/UEFI me Intel Virtualization Technology enable ki.
WSL/Virtual Machine Platform issue	Windows Features me WSL aur Virtual Machine Platform enable kiye, restart kiya.
Ubuntu password mismatch/token error	Ubuntu first-run setup ko dobara complete kiya.
n8n CLI direct install issue	npm route chhor kar Docker approach use ki.
hello-world image typo/pull issue	Correct docker run hello-world command use ki.
PowerShell syntax CMD me run hui	$body wali syntax CMD me nahi chalti; curl command use ki.
Webhook not registered	Listen for test event dobara click karke request immediately send ki.
ERR_EMPTY_RESPONSE	Container start ke baad n8n ko startup time diya.
Cannot GET / temporary behavior	Application startup/config response ko recheck kiya; baad me UI open ho gayi.

Troubleshooting ka skill development ka important hissa hai: error read karo → cause isolate karo → small fix test karo → result verify karo.

21. Documentation, Screenshots aur LinkedIn Showcase

Documentation kyun?

- Baad me revision easy hoti hai.
- Internship proof milta hai.
- Commands aur errors ka record rehta hai.
- Portfolio me project explain kar sakte ho.

Screenshots kyun?

Screenshots se environment, workflow configuration aur successful output ka evidence milta hai. Submission me proof clear hota hai.

LinkedIn showcase se kya seekha?

Technical work ke saath usko professionally explain karna bhi important skill hai. Weekly showcase se learning journey public portfolio me convert hoti hai.

22. Phase 1 ke baad tumhe khud kya aana chahiye?

- Docker Desktop open aur container start karna
- localhost:5678 se n8n open karna
- New workflow create karna
- Webhook node add karna
- HTTP Method POST select karna
- Webhook Path set karna
- Listen for test event use karna
- CMD se curl request bhejna
- JSON data ko output me verify karna
- Basic error ko identify karke small troubleshooting karna

Agar tum ek naya “Contact Form - Practice” webhook khud bana kar test kar lo, to Phase 1 ki learning sirf copied steps nahi rahegi - actual skill ban jayegi.

23. Quick Revision Glossary

Term	Easy meaning
Docker	Software ko isolated containers me run karne ka platform.
Image	Software ka ready-made template.
Container	Image ka running instance.
WSL 2	Windows ke andar Linux environment support.
Ubuntu	Linux distribution jo WSL me run hui.
Virtualization	CPU feature jo virtual environments ko support karta hai.
localhost	Apna computer.

Port 5678	n8n service ka local access port.
Volume	Persistent storage jo data ko container ke bahar save rakhti hai.
Environment Variable	Startup/configuration setting.
n8n	Workflow automation platform.
Workflow	Automation steps ka connected flow.
Webhook	Incoming HTTP data receive karne ka endpoint.
POST	Server ko data bhejne ka HTTP method.
GET	Server se data mangne ka HTTP method.
JSON	Structured key-value data format.
curl	Terminal se HTTP request bhejne ka tool.
Multi-channel	Multiple communication sources.
Centralized AI	Sab channels ke liye ek common AI logic/brain.
CRM	Customer records/history manage karne ka system.

24. Self-test Questions

7. Docker Image aur Container me kya farq hai?
8. localhost:5678 me localhost aur 5678 kya represent karte hain?
9. Webhook kis kaam aata hai?
10. POST aur GET me basic difference kya hai?
11. JSON structured data kyun useful hai?
12. Listen for test event kyun click karna padta hai?
13. Agar webhook not registered error aaye to pehla check kya karoge?
14. Real WhatsApp API aur webhook simulation me kya difference hai?
15. Phase 1 me AI brain actually implement hua tha ya sirf architecture me plan hua?
16. Website, Email aur WhatsApp ko centralized AI me lane ka faida kya hai?

Note: In 10 sawalon ke jawab bina notes dekhe de sakte ho to Phase 1 ki understanding achi hai.

25. Phase 1 Final Checklist aur Next Phase Preview

Item	Status
Docker + WSL 2 + Ubuntu environment	Done
n8n Docker image	Done
n8n container	Done
localhost access	Done
Fictional business	Done
3 communication channels	Done
Website Chat webhook	Done
Email webhook	Done

WhatsApp webhook	Done
Separate test workflows	Done
Independent testing	Done
Architecture diagram	Done
Documentation/screenshots	Done

PHASE 1 COMPLETE

Next Phase me kya hoga?

Next learning step different channels ke incoming data ko ek common standardized format me convert karna hoga. Example fields: name, email, phone, message, channel, timestamp aur unique message ID. Is se AI aur CRM ko har channel ka data same structure me milega.

```
Website data ─┐
Email data ──────┼──> Standard Format ──> AI / CRM
WhatsApp data ─┘
```

One-page Final Revision

```
Customer
  ↓
Communication Channel
  ↓
HTTP POST Request
  ↓
Webhook
  ↓
n8n Workflow
  ↓
JSON Data
  ↓
[Next Phase: Standardization]
  ↓
[Later: AI → CRM → Routing → Reply → Human Handoff]
```

Abhi hum Customer → Channel → POST → Webhook → n8n → JSON tak practically pohanch chuke hain.

Ye Phase 1 ki complete learning foundation hai. Isko revise karte waqt sirf definitions yaad na karo; har concept ko apne actual project flow se connect karo.

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 2

Multi-channel Message Collection and Standardization

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	COMPLETE

Normalize every channel into one seven-field message contract before CRM/AI processing.

1. Phase 2 Objective - Standardize Every Channel

Phase 2 converted raw Website, Email and WhatsApp payloads into one common message schema. This prevents later CRM and AI workflows from needing three different input formats.

```
Website raw data \
Email raw data ---> Edit Fields / Standardization ---> Common schema ---> CRM / AI
WhatsApp raw data /
```

Final common schema

Field	Meaning
message_id	Unique ID for every incoming message
sender_name	Customer name
sender_email	Email where available
sender_phone	Phone where available
message	Customer message text
channel	website_chat / email / whatsapp
timestamp	ISO timestamp

Example standardized Website message

```
{
"message_id": "web-16",
"sender_name": "Ali Khan",
"sender_email": "ali@example.com",
"sender_phone": "",
"message": "I need a website for my business",
"channel": "website_chat",
"timestamp": "...ISO..."
}
```

Unique ID approach

• Website IDs used a web- prefix.

• Email IDs used an email- prefix.

• WhatsApp IDs used a wa- prefix.

• n8n execution IDs were used during simulation to keep IDs unique.

Testing requirement

Each channel was tested with at least five fictional customer messages. This proved that the schema stayed identical while only the channel-specific values changed.

Key learning

• Standardization is a data contract between channels and future CRM/AI logic.

• Edit Fields is the normalization layer.

• Channel-specific raw properties should be converted before data reaches downstream logic.

Phase 2 final result: Website Chat, Email and WhatsApp all output the same seven fields and are ready for centralized CRM/AI processing. The original 25-page detailed notes and screenshots are appended.

Multi-channel AI Agent

Phase 2 - Multi-channel Message Collection

Complete Detailed Learning + Implementation + Troubleshooting Notes

Project: Multi-channel AI Agent
Business: NexaTech Solutions
Platform: n8n (local Docker setup)
Channels: Website Chat, Email, WhatsApp
Date: 24 September 2026

Is document me Phase 2 ka poora flow, standardized data format, n8n field mapping, test cases, PowerShell commands, actual screenshots, errors, reasons aur unke solutions sab documented hain.

1. Phase 2 ka Asal Maqsad

Phase 1 me humne verify kiya tha ke Website Chat, Email aur WhatsApp ke messages n8n tak aa sakte hain. Phase 2 ka maqsad sirf message receive karna nahi, balki har channel ke alag raw data ko **same standardized format** me convert karna hai.

```
Website Chat --\
Email ----> Raw Data --> Standardization --> Same Format --> Future AI
WhatsApp --/
```

Standardization is liye zaroori hai kyun ke future AI system ko 3 alag-alag schemas samajhne ki bajaye sirf ek common schema process karna hoga.

Phase 2 ke 10 required steps:

Step	Requirement
1	Website Chat / simulated website webhook workflow use karna
2	Incoming Email workflow use karna
3	WhatsApp / simulated WhatsApp webhook workflow use karna
4	Sender name extract karna
5	Sender email extract karna jahan available ho
6	Phone number extract karna jahan available ho
7	Message content extract karna
8	Channel name aur timestamp add/extract karna
9	Har incoming message ko unique message ID dena
10	Har channel ko kam az kam 5 fictional customer messages se test karna

2. Standardized Data Format

Humne tino workflows me field names same rakhe. Channel ke hisaab se value change ho sakti hai, lekin schema same rahega.

```
message_id
sender_name
sender_email
sender_phone
message
channel
timestamp
```

Example standardized output:

```
{
"message_id": "web-16",
"sender_name": "Ali Khan",
"sender_email": "ali@example.com",
"sender_phone": "",
"message": "I need a website for my business",
"channel": "website_chat",
"timestamp": "2026-09-24T04:13:05.225+05:00"
}
```

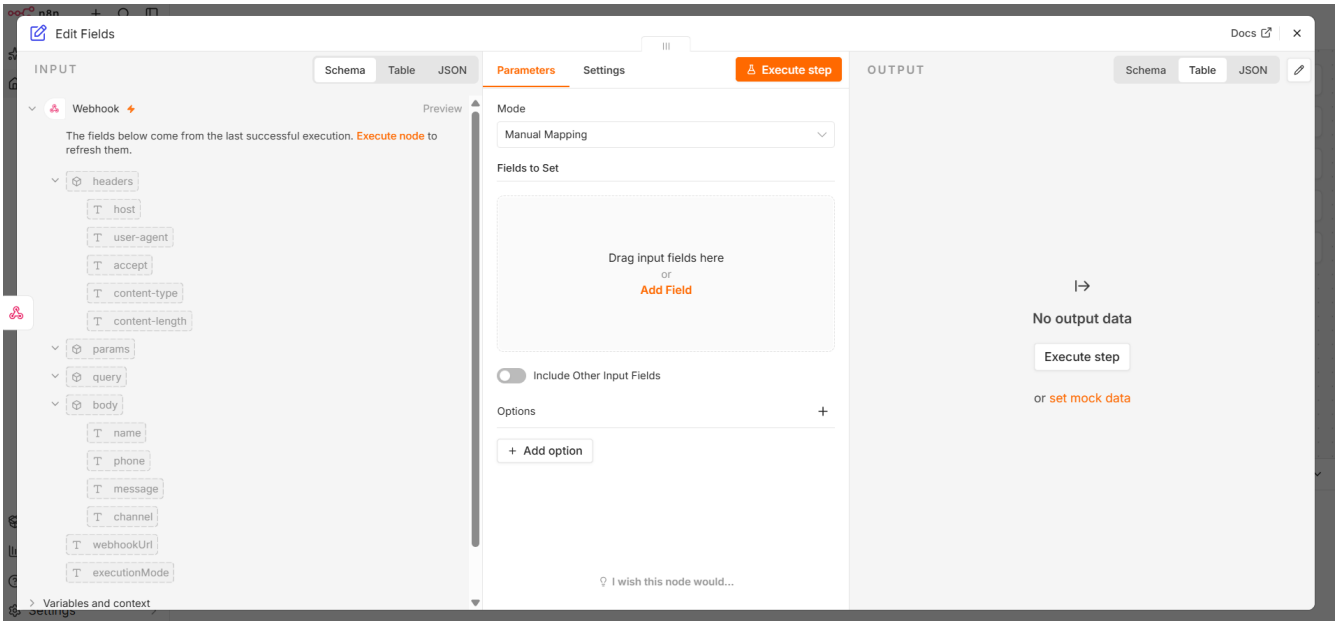
Important: missing data ko fake value se fill nahi kiya gaya. Jahan phone/email available nahi tha, empty value rakhi gayi.

3. Important Concepts jo Phase 2 me Seekhe

Concept	Easy Meaning
Extract	Incoming raw JSON me se required value read karna, jaise body.name.
Standardize	Different source field ko ek common naam dena, jaise name -> sender_name.
Edit Fields / Set	n8n node jahan hum output ka final structure manually define karte hain.
Timestamp	$now.toISO() current date/time ko ISO standard me convert karta hai.
Execution ID	$execution.id har workflow run ki n8n execution ID deta hai.
Message ID	Channel prefix + execution ID. Website: web-..., Email: email-..., WhatsApp: wa-...
Missing field handling	Expression me || '' use kiya taake missing field error na de aur empty value aaye.
Channel field	Source ko identify karta hai: website_chat, email, whatsapp.

4. Website Chat - Edit Fields Node Build Karna

Webhook ke baad **Edit Fields** node add kiya gaya. Is node ka kaam raw webhook body ko clean standardized output me convert karna tha.

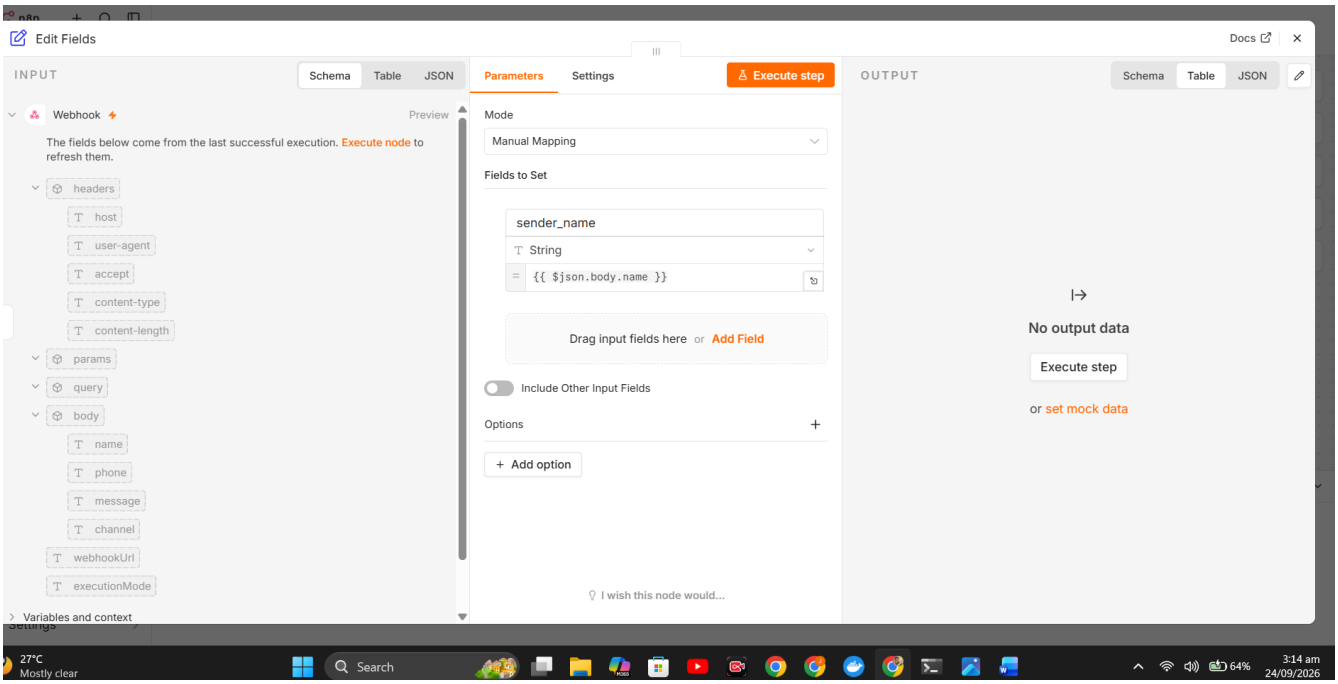


Screenshot: Edit Fields node add hone ke baad Manual Mapping screen. Incoming Webhook fields left side me visible hain.

5. sender_name Field Mapping

```
sender_name = {{ $json.body.name || '' }}
```

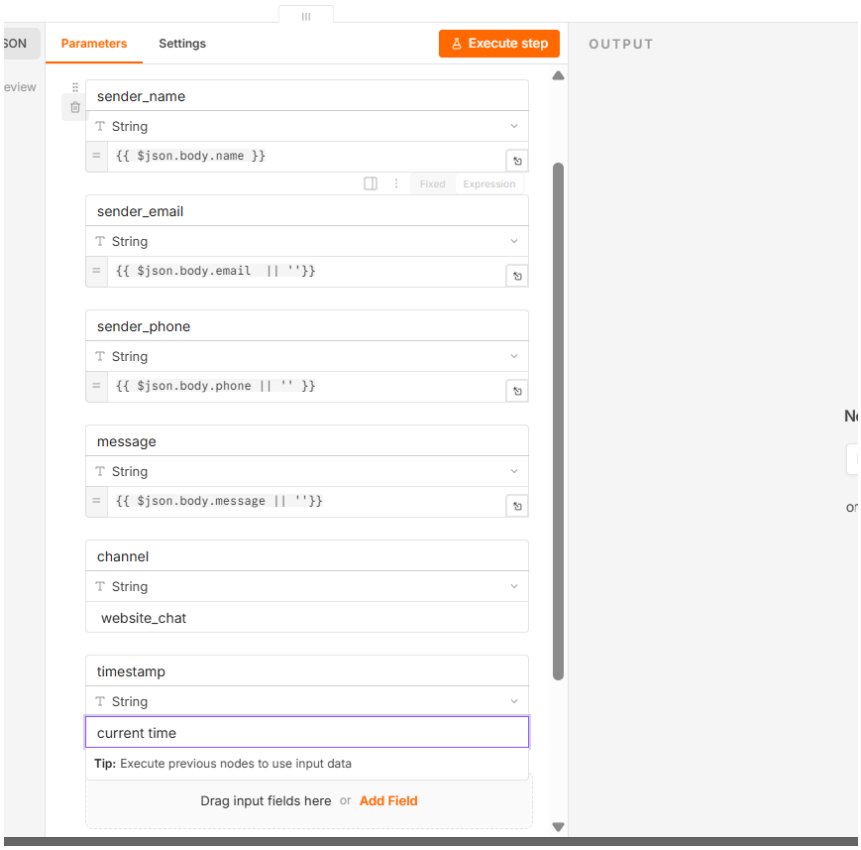
Meaning: Webhook ke body me jo name aaye usko standardized field **sender_name** me save karo. Agar name missing ho to empty string use karo.



Screenshot: sender_name field ko body.name expression ke sath map kiya gaya.

6. Website Standard Fields aur Pehli Correction

Fields set karte waqt ek duplicate timestamp aur naming mistake hui. Ye learning ka important part tha.



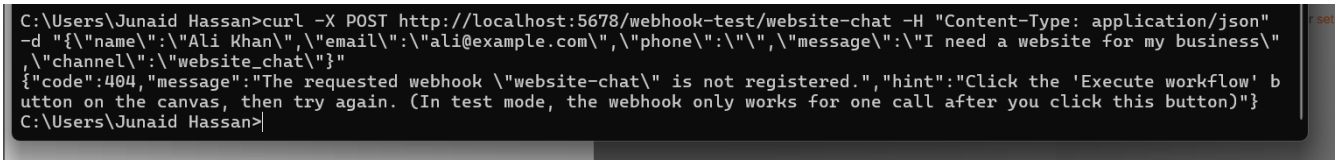
Screenshot: sender fields, message, channel ke sath timestamp aur unique ID add karte waqt duplicate/incorrect field naming nazar aa rahi thi.

Correct final expressions:

```
message_id = {{ 'web-' + $execution.id }}
sender_name = {{ $json.body.name || '' }}
sender_email = {{ $json.body.email || '' }}
sender_phone = {{ $json.body.phone || '' }}
message = {{ $json.body.message || '' }}
channel = website_chat
timestamp = {{ $now.toISO() }}
```

7. Troubleshooting - Webhook Not Registered (404)

Website test request bhejte waqt 404 error aaya: **The requested webhook is not registered.**



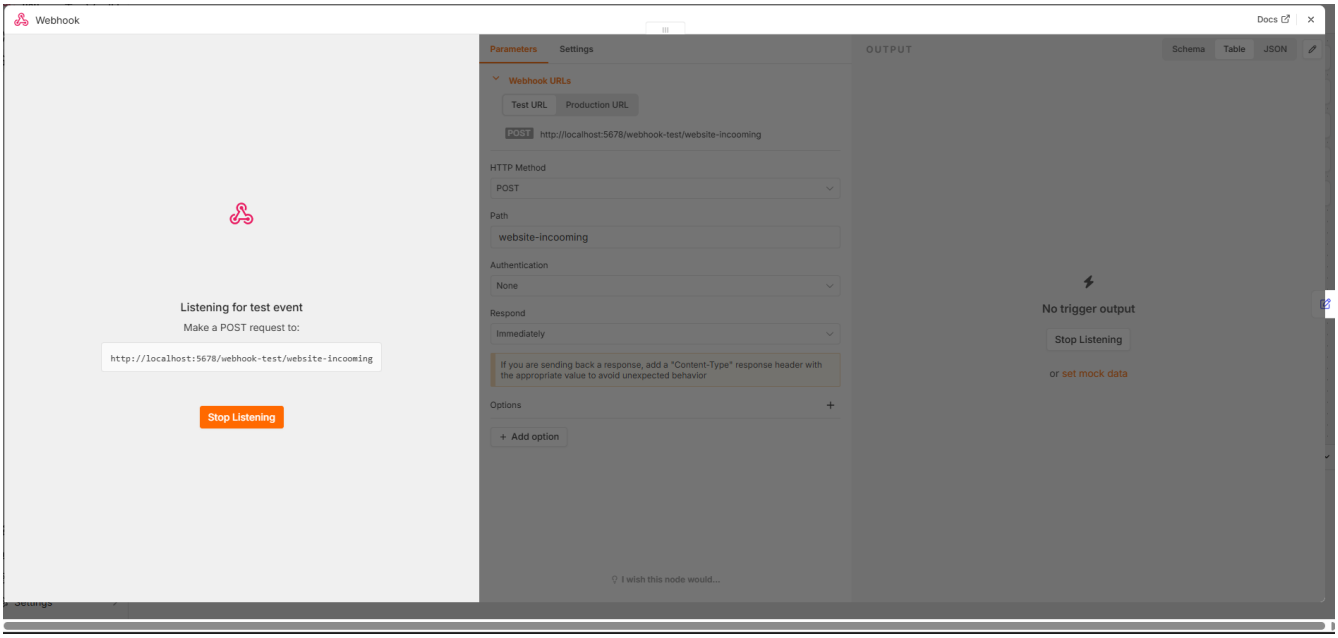
Screenshot: CMD/terminal me 404 webhook not registered error.

Reason: Test URL sirf tab active hota hai jab n8n me **Listen for test event** active ho. Test webhook temporary hota hai aur usually ek test call receive karta hai.

Solution: Webhook node me Listen for test event click kiya, phir request foran bheji.

8. Troubleshooting - Webhook Path Mismatch / Typo

Ek aur issue path spelling ka tha. n8n me path aur PowerShell request me path exactly same hona zaroori hai.



Screenshot: n8n test listener active tha aur path me website-incooming spelling visible thi.

Galat/similar paths jo debugging me saamne aaye:

```
website-chat
website-incoming
website-incooming
```

Lesson: ek extra letter bhi webhook ko different endpoint bana deta hai. Final request aur n8n path ko same rakha gaya.

9. Troubleshooting - PowerShell me curl Alias Issue

PowerShell me **curl** command actually Invoke-WebRequest ka alias ho sakti hai. Isi wajah se CMD wali -H / -d syntax PowerShell me fail hui.

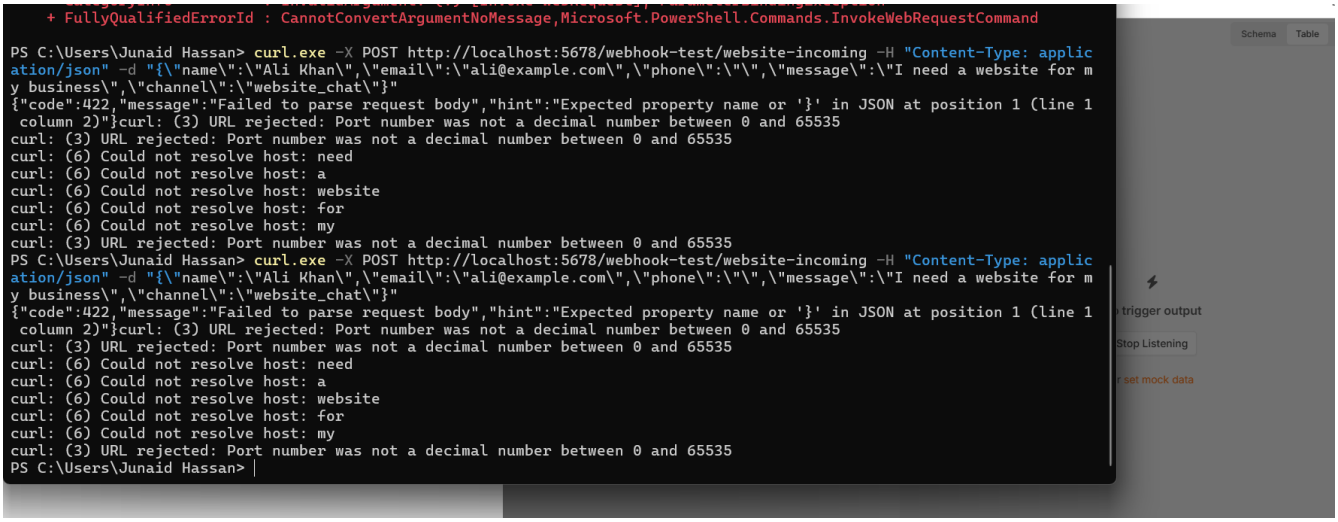
```
PS C:\Users\Junaid Hassan> http://localhost:5678/webhook-test/website-incooming
http://localhost:5678/webhook-test/website-incooming : The term 'http://localhost:5678/webhook-test/website-incooming'
is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name,
or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ http://localhost:5678/webhook-test/website-incooming
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (http://localhos...bsite-incooming:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Junaid Hassan> curl -X POST http://localhost:5678/webhook-test/website-incoming -H "Content-Type: applicat
n/json" -d "{\"name\":\"Ali Khan\",\"email\":\"ali@example.com\",\"phone\":\"\",\"message\":\"I need a website for my
siness\",\"channel\":\"website_chat\"}"
Invoke-WebRequest : Cannot bind parameter 'Headers'. Cannot convert the "Content-Type: application/json" value of type
"System.String" to type "System.Collections.IDictionary".
At line:1 char:69
+ ... ook-test/website-incoming -H "Content-Type: application/json" -d "{\" ...
+                               ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [Invoke-WebRequest], ParameterBindingException
    + FullyQualifiedErrorId : CannotConvertArgumentNoMessage,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\Users\Junaid Hassan>
```

Screenshot: PowerShell me curl use karne par Headers parameter conversion error.

Phir **curl.exe** try kiya gaya, lekin JSON quoting/escaping PowerShell me break ho gayi aur body parse error aaya.



Screenshot: curl.exe request me malformed JSON / URL parsing errors.

Final reliable PowerShell method:

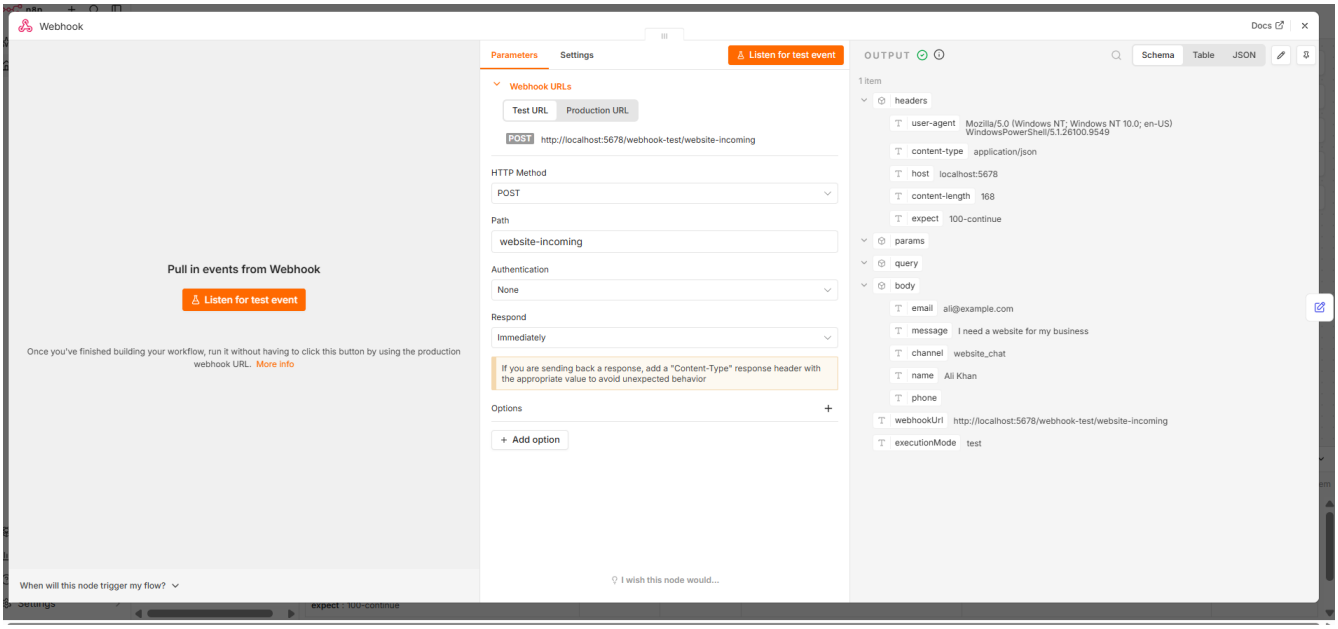
```
$body = @{
name = "Ali Khan"
email = "ali@example.com"
phone = ""
message = "I need a website for my business"
channel = "website_chat"
} | ConvertTo-Json

Invoke-RestMethod `
-Uri "http://localhost:5678/webhook-test/website-incoming" `
-Method POST `
-ContentType "application/json" `
-Body $body
```

Ye method sabse clean raha: PowerShell hashtable -> ConvertTo-Json -> Invoke-RestMethod.

10. Website Chat Raw Webhook - Successful Receive

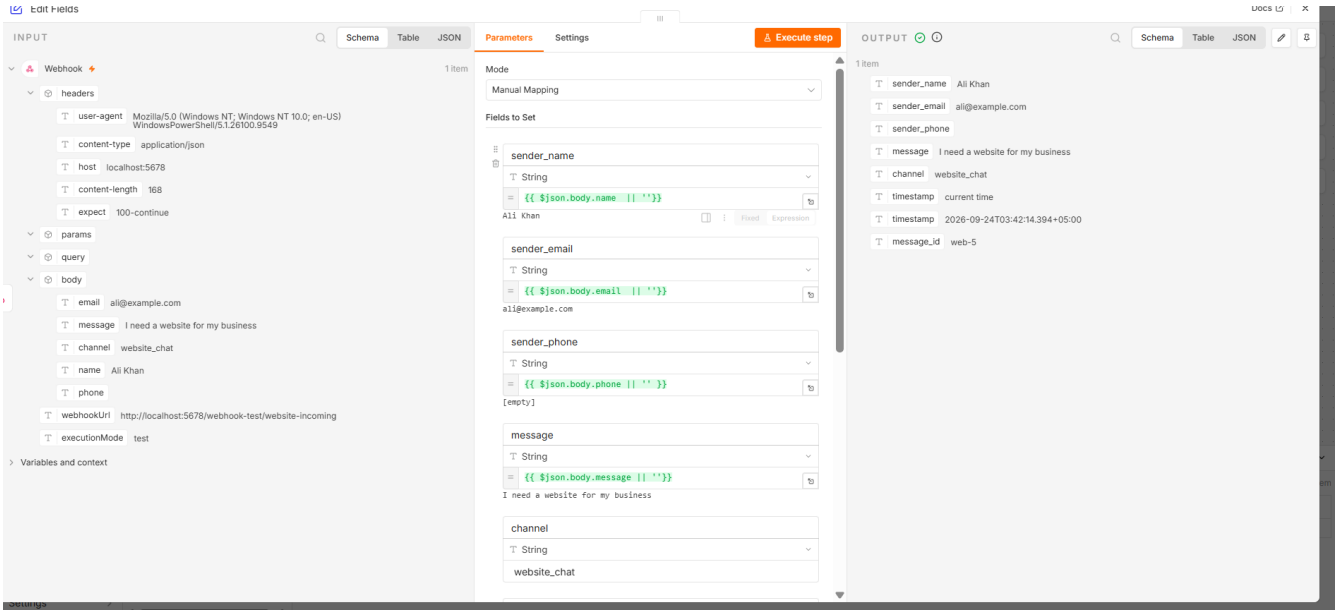
Correct path + active listener + PowerShell Invoke-RestMethod use karne ke baad raw Website Chat data n8n me successfully receive hua.



Screenshot: Webhook body me Ali Khan ka name, email, message aur website_chat channel successfully receive hua.

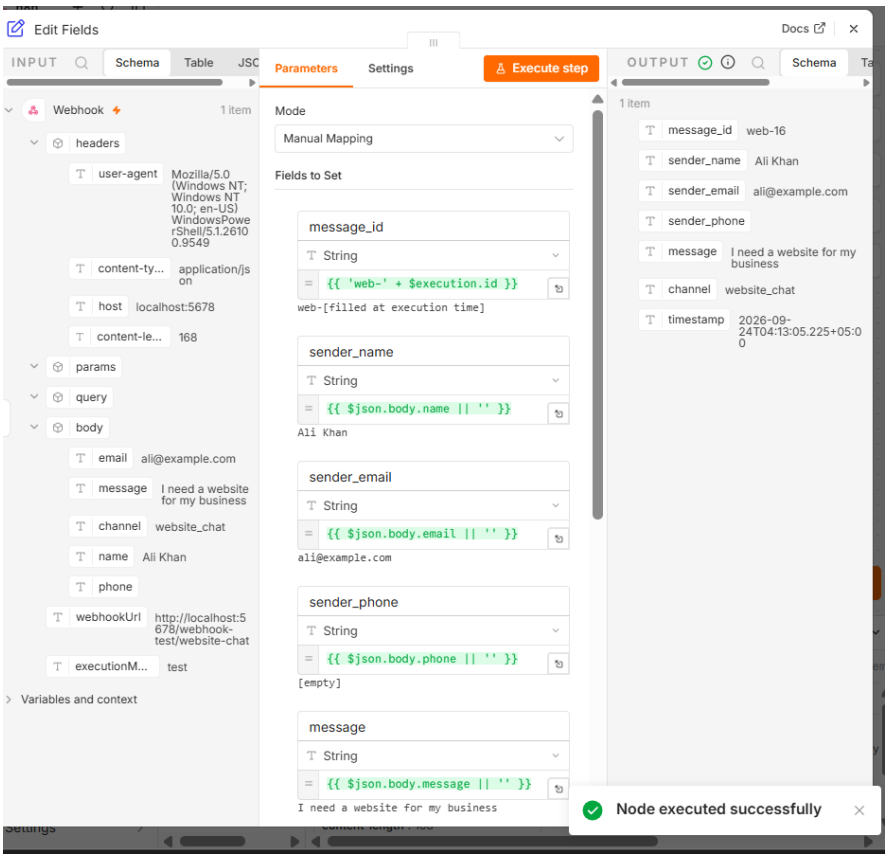
11. Website Chat Standardized Output - Correction aur Final

Pehle execution me standardized output sahi tha lekin timestamp do baar aa raha tha: ek literal "current time" aur ek actual ISO timestamp. Literal duplicate field remove kiya gaya.



Screenshot: Website standardized output me duplicate timestamp issue visible tha.

Final corrected Website output:

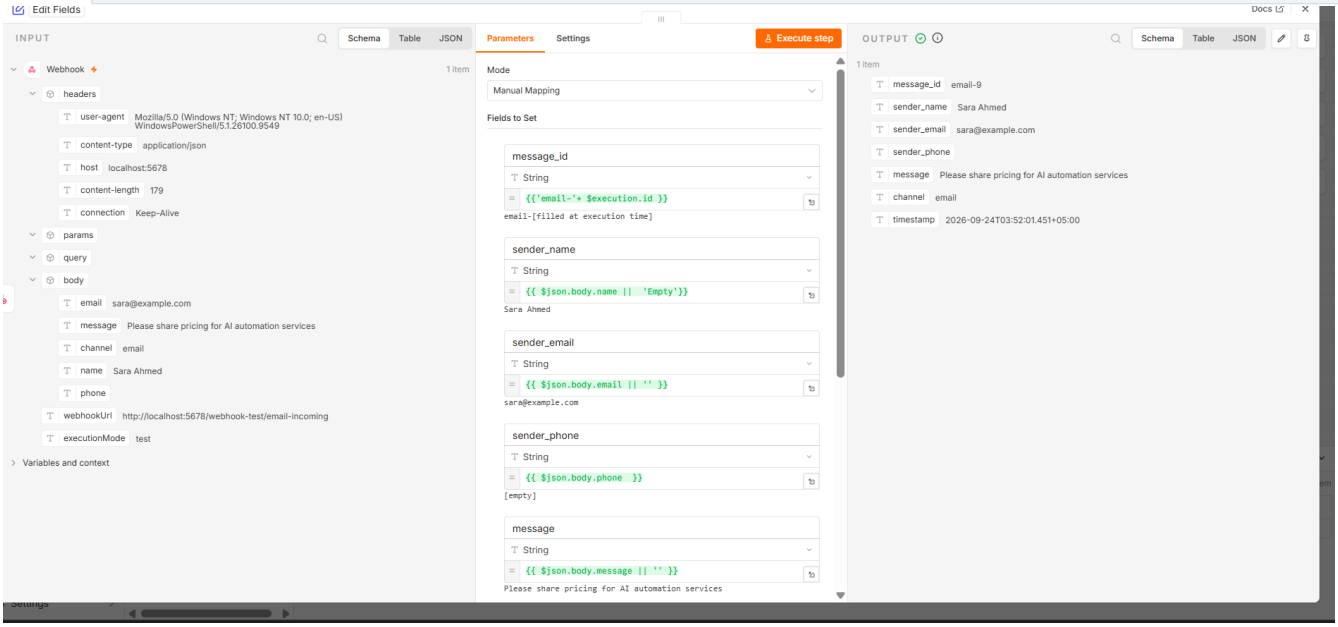


Screenshot: Website Chat final output - message_id web-16, Ali Khan, email, empty phone, message, website_chat aur ISO timestamp.

12. Email Workflow - Standardization

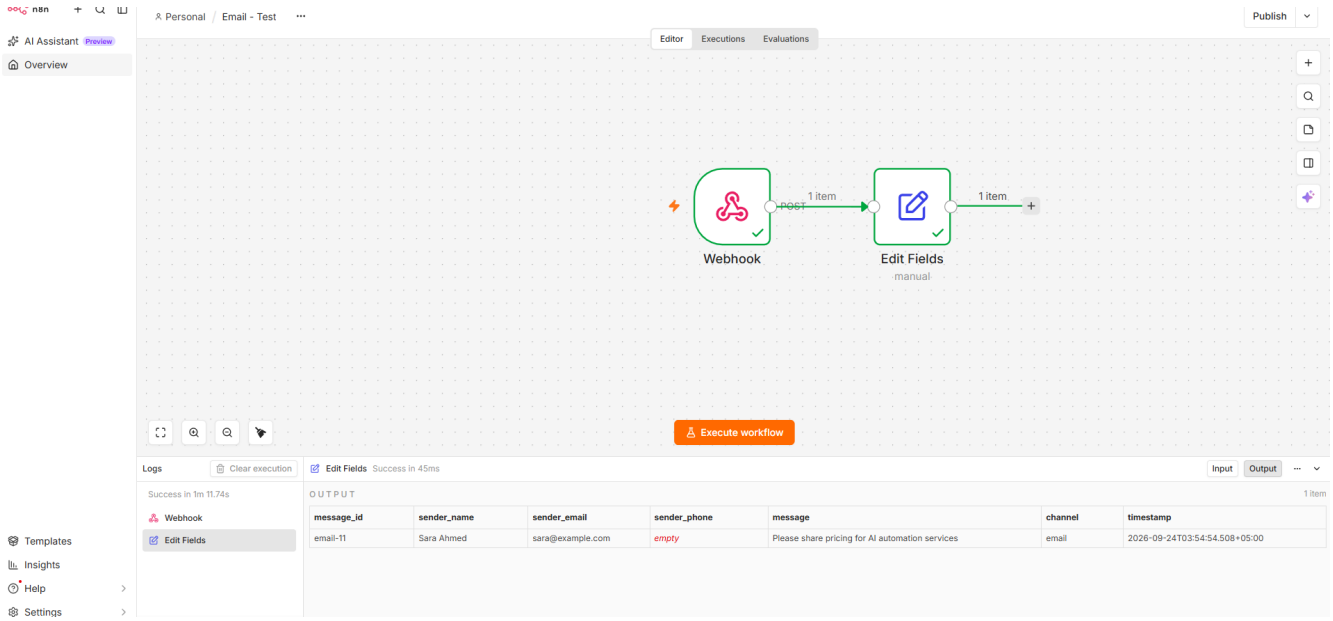
Email workflow me bhi exactly same 7 standardized fields rakhe gaye. Sirf message_id prefix aur channel value change hui.

```
message_id = {{ 'email-' + $execution.id }}
sender_name = {{ $json.body.name || '' }}
sender_email = {{ $json.body.email || '' }}
sender_phone = {{ $json.body.phone || '' }}
message = {{ $json.body.message || '' }}
channel = email
timestamp = {{ $now.toISO() }}
```



Screenshot: Email standardized output - Sara Ahmed, sara@example.com, empty phone, AI automation pricing message, channel email, email message ID aur ISO timestamp.

13. Email Workflow - Final Canvas

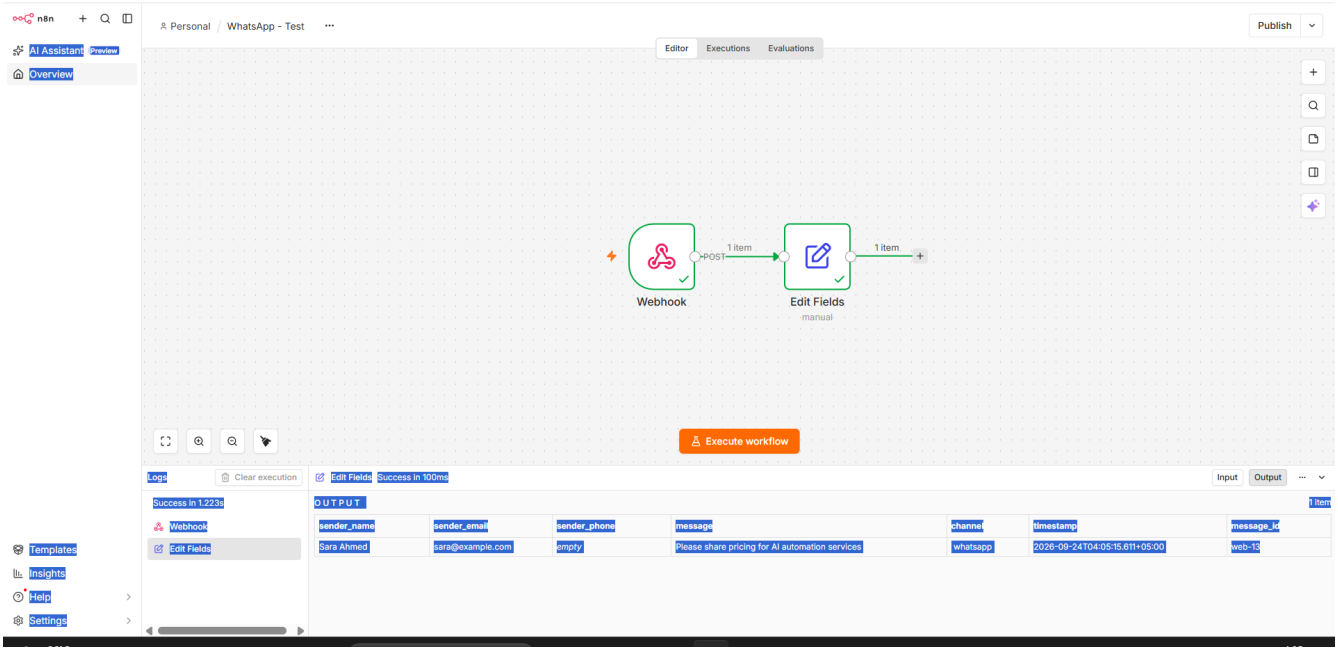


Screenshot: Email - Test workflow final flow: Webhook -> Edit Fields. Output table me standardized fields visible hain.

Email result: receive, extract, standardize aur test successful.

14. WhatsApp Workflow - Field Setup aur Data Correction

WhatsApp workflow me same schema use hua. Is channel me phone important field tha. Initial screenshot me purana Email/Website test data reh gaya tha, isliye actual WhatsApp test dobara run kiya gaya.



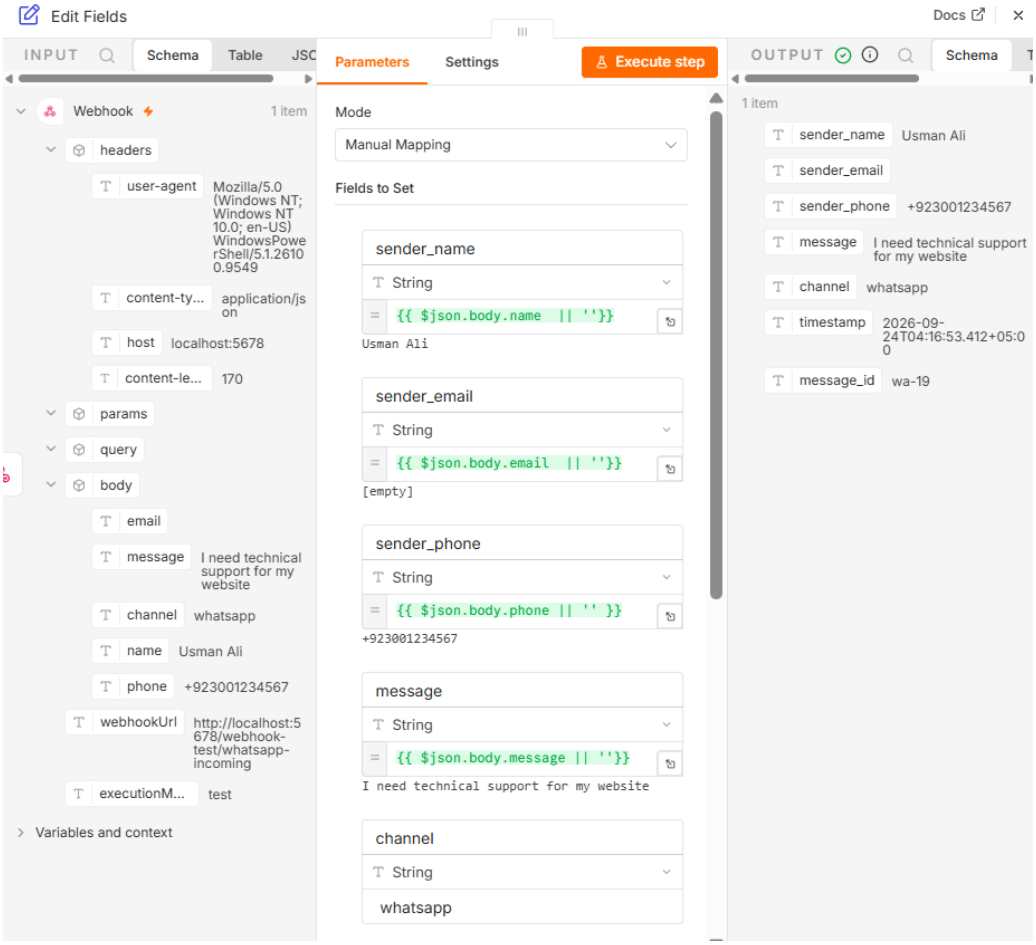
Screenshot: WhatsApp workflow ke andar old test data dikh raha tha; isne verify kiya ke node configuration aur input execution data ko alag samajhna zaroori hai.

WhatsApp final field expressions:

```
message_id = {{ 'wa-' + $execution.id }}
sender_name = {{ $json.body.name || '' }}
sender_email = {{ $json.body.email || '' }}
sender_phone = {{ $json.body.phone || '' }}
message = {{ $json.body.message || '' }}
channel = whatsapp
timestamp = {{ $now.toISO() }}
```

15. WhatsApp Final Standardized Test

Actual WhatsApp fictional customer message send kiya gaya: Usman Ali, fake/test phone number, technical support message.



Screenshot: WhatsApp final output - Usman Ali, phone +923001234567, empty email, technical support message, channel whatsapp, wa-19 ID aur ISO timestamp.

WhatsApp result: receive, extract, standardize aur test successful.

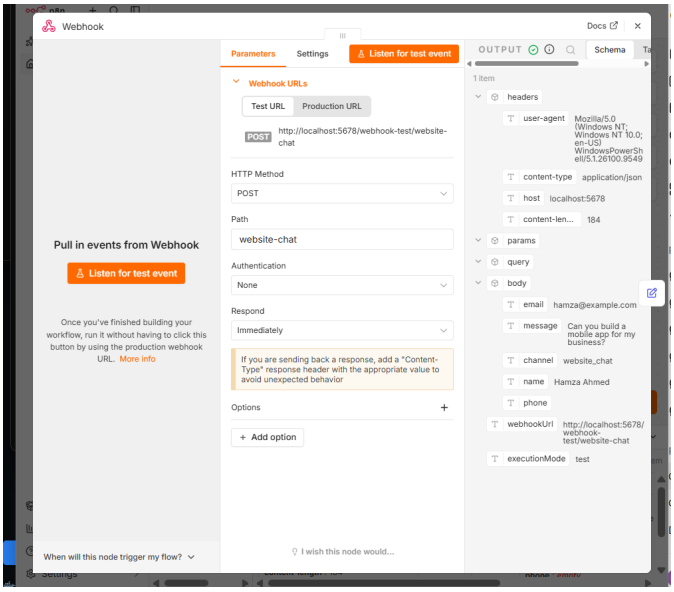
16. Step 10 - 15 Fictional Customer Messages Testing

Assignment requirement ke mutabiq har channel ko kam az kam 5 fictional customer messages se test kiya gaya. Total 15 messages.

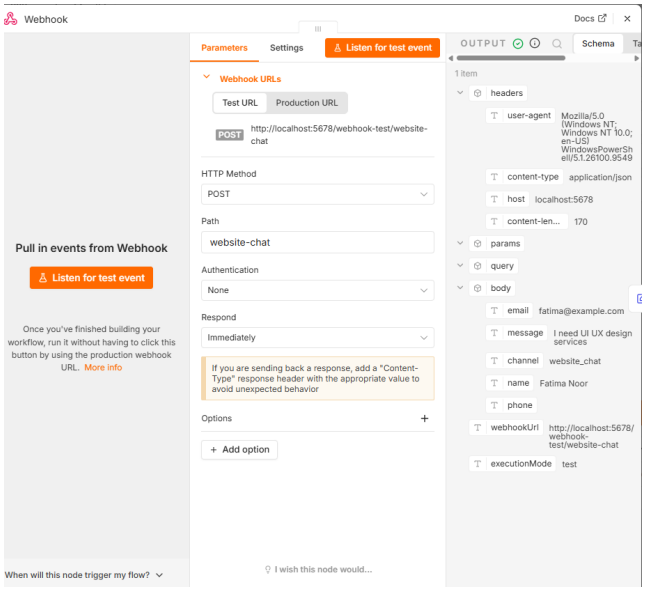
Test	Name	Email	Phone	Message	Channel
Website 1	Ali Khan	ali@example.com	-	I need a website for my business	website_chat
Website 2	Ayesha Khan	ayesha@example.com	-	I need an e-commerce website	website_chat
Website 3	Hamza Ahmed	hamza@example.com	-	Can you build a mobile app for my business?	website_chat
Website 4	Fatima Noor	fatima@example.com	-	I need UI UX design services	website_chat
Website 5	Bilal Hassan	bilal@example.com	-	I need a CRM system for my company	website_chat
Email 1	Sara Ahmed	sara@example.com	-	Please share pricing for AI automation services	email
Email 2	Zain Ali	zain@example.com	-	I need API integration services	email
Email 3	Hira Malik	hira@example.com	-	Please share cloud hosting packages	email
Email 4	Ahmed Raza	ahmed@example.com	-	I need a cybersecurity consultation	email
Email 5	Maham Khan	maham@example.com	-	I need software maintenance support	email
WhatsApp 1	Usman Ali	-	+923001234567	I need technical support for my website	whatsapp
WhatsApp 2	Umar Farooq	-	+923000000002	My website is not loading	whatsapp
WhatsApp 3	Mariam Ali	-	+923000000003	I need an AI chatbot for my business	whatsapp
WhatsApp 4	Saad Khan	-	+923000000004	Can you automate my business workflows?	whatsapp
WhatsApp 5	Noor Ahmed	-	+923000000005	I need help with my mobile application	whatsapp

17. Website Chat - Multi-message Testing Proof

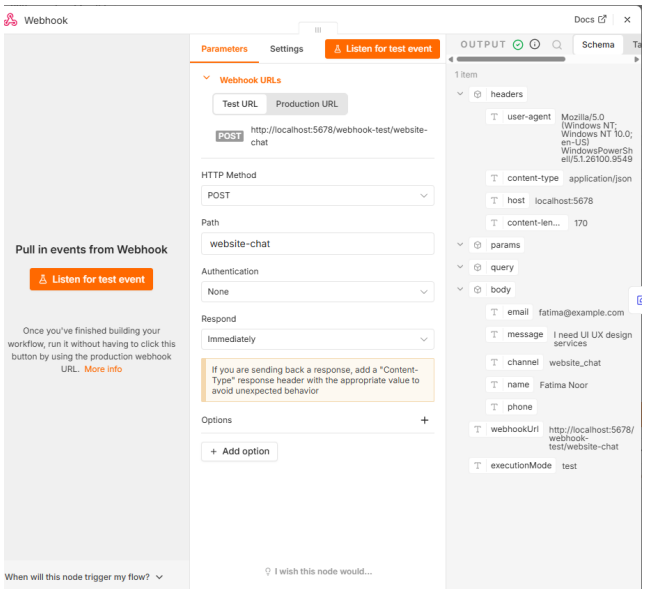
Website Chat ke multiple fictional customers ke screenshots neeche hain. Inse clear hai ke workflow sirf ek hardcoded customer ke liye nahi chal raha.



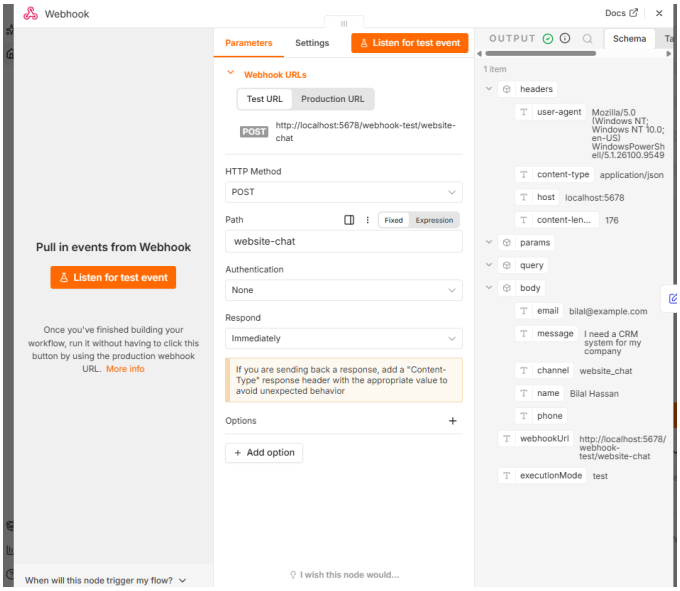
Website test: Hamza Ahmed - mobile app request.



Website test: Fatima Noor - UI/UX design request.



Website test screenshot - another successful website_chat execution.



Website test: Bilal Hassan - CRM system request.

18. Testing Commands - Reusable PowerShell Pattern

Har test se pehle Webhook node me **Listen for test event** click karna hota hai. Uske baad same PowerShell pattern me sirf customer values aur URL change ki jati hai.

```
$body = @{
name = "Customer Name"
email = "customer@example.com"
phone = ""
message = "Customer message"
channel = "website_chat"
} | ConvertTo-Json

Invoke-RestMethod `
-Uri "http://localhost:5678/webhook-test/website-chat" `
-Method POST `
-ContentType "application/json" `
-Body $body
```

Email URL: **/webhook-test/email-incoming**
WhatsApp URL: **/webhook-test/whatsapp-incoming**

19. Is Phase me Kya Seekha

#	Learning
1	Raw webhook JSON ko clean standardized format me convert karna
2	Edit Fields / Set node ki Manual Mapping
3	$json.body.field se nested data read karna
4	Missing field ke liye || '' fallback use karna
5	$now.toISO() se standard timestamp banana
6	$execution.id se current workflow run ki ID lena
7	Channel prefix use karke human-readable message IDs banana
8	Website, Email aur WhatsApp ko same schema me lana
9	PowerShell me JSON safely send karna using ConvertTo-Json + Invoke-RestMethod
10	Webhook path spelling aur test listener ki importance
11	Different fictional customers se hardcoding detect/test karna
12	Errors ko reason + solution ke sath document karna

20. Complete Troubleshooting Summary

Issue	Kyun Hua	Solution
404 webhook not registered	Test listener active nahi tha	Listen for test event click karke request foran send ki
Path mismatch	Request path aur n8n path different thay	Dono ko exact same kiya
website-incooming typo	Extra o ki wajah se different endpoint bana	Path spelling correct ki
PowerShell direct URL	URL ko command samjha gaya	Invoke-RestMethod use kiya
curl Headers error	PowerShell curl alias Invoke-WebRequest tha	PowerShell-native method use ki
curl.exe JSON parse error	Escaping/quotes break ho rahi thi	Hashtable + ConvertTo-Json use kiya
Duplicate timestamp	Literal current time + ISO timestamp dono fields thay	Literal duplicate field delete ki
Old data in WhatsApp output	Previous execution data visible tha	Fresh WhatsApp test event execute kiya
Missing phone/email	Source me value available nahi thi	Empty value acceptable rakhi

21. Final Phase 2 Status

Area	Requirement	Status
Website Chat	Webhook receive	Done
Website Chat	7-field standardization	Done
Website Chat	5 fictional messages	Done
Email	Webhook receive	Done
Email	7-field standardization	Done
Email	5 fictional messages	Done
WhatsApp	Webhook receive	Done
WhatsApp	7-field standardization	Done
WhatsApp	5 fictional messages	Done
Common Schema	Same field names across all channels	Done
Unique IDs	web-/email-/wa- + execution ID	Done
Timestamp	ISO timestamp per message	Done

Phase 2 Result: Website Chat, Email aur WhatsApp se aane wale messages ab ek same standardized format me convert ho rahe hain. Ye data next phase me centralized CRM / AI processing ke liye ready hai.

22. Phase 2 ka Future Project me Faida

Ab future AI ko channel-specific raw data samajhne ki zaroorat nahi. Har message same keys ke sath milega:

```
message_id -> sender_name -> sender_email -> sender_phone -> message -> channel -> timestamp
```

Is foundation par next phases me CRM, customer identification, conversation history, AI intent classification, routing, auto-response aur human handoff build kiya ja sakta hai.

Simple flow:

```
Website / Email / WhatsApp
|
v
Webhook
|
v
Edit Fields
|
v
Standardized Message
|
v
Future CRM + AI System
```

23. Submission Proof Checklist

Proof	Kya Dikhata Hai
Website Chat workflow screenshot	Webhook + Edit Fields flow / output
Email workflow screenshot	Webhook + Edit Fields flow / standardized output
WhatsApp workflow screenshot	Webhook + Edit Fields / phone extraction
Website multi-message screenshots	Different fictional customers
Error screenshots	Troubleshooting evidence
Final standardized output	message_id, sender fields, message, channel, timestamp

Sensitive credentials, API keys, passwords ya real customer data is document me include nahi kiya gaya. Test data fictional hai.

PHASE 2 - COMPLETE

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 3

Unified Customer Profile and Simulated CRM

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	COMPLETE

One Customer ID, Google Sheets CRM, duplicate prevention and centralized conversation storage.

1. Phase 3 Objective - One Customer, One CRM Profile

Phase 3 introduced a simulated CRM using Google Sheets. The system identifies an existing customer using email and/or phone, updates the same profile instead of creating duplicates, and stores every message as a separate conversation record.

Google Sheets data model

Customers sheet	Conversations sheet
Customer ID	Conversation ID
Name	Customer ID
Email	Channel
Phone	Message ID
Company	Message
Preferred Channel	Sender Type
First Interaction	Timestamp
Last Interaction	Intent
Customer Status	Response

Final CRM workflow

```
Webhook -> Edit Fields -> Get row(s) in Customers -> IF
TRUE -> Update existing customer --\
FALSE -> Append new customer -----+-> Store Conversation
```

Identity logic

- Email is a strong identity when website/email channels provide it.
- Phone is essential for WhatsApp and cross-channel matching.
- First contact creates the customer; later contacts update the same Customer ID.
- Every message creates a new row in Conversations.

Duplicate-customer debugging

The workflow was tested with the same fictional customer across Website Chat and WhatsApp. Duplicate creation problems were debugged by correcting lookup values and branch logic.

Phase 3 final result: customer identity is centralized, while message history remains one-to-many through the Conversations sheet. The original Phase 3 notes and screenshots are appended.

Multi-channel AI Agent

Phase 3 - Unified Customer Profile and Simulated CRM

Internship Day	Day 10
Date	25 September 2026
CRM	Google Sheets - Multi-channel AI CRM
Automation	n8n local workflow
Channels tested	Website Chat + WhatsApp
Final status	Core Phase 3 workflow working

Goal: Website, email/phone identity and WhatsApp messages ko ek centralized customer profile aur conversation history ke saath manage karna.

1. Aaj ka Submission Description

Aaj Multi-channel AI Agent ke Phase 3 me centralized CRM workflow complete kiya gaya. Google Sheets me **Customers** aur **Conversations** tables use ki gayin. n8n incoming message ko standard fields me receive karta hai, existing customer ko Email ya Phone se search karta hai, new customer ko append karta hai aur existing customer ko update karta hai. Har incoming message ko separate conversation record ke taur par store kiya jata hai. Website Chat aur WhatsApp ke through same fictional customer ko test karke verify kiya gaya ke customer profile single rehti hai aur multiple messages Conversations table me save hote hain.

Aaj kya implement hua

- Multi-channel AI CRM Google Sheet configured.
- Customers table with Customer ID, Name, Email, Phone, Company, Preferred Channel, First Interaction, Last Interaction and Customer Status.
- Conversations table with Conversation ID, Customer ID, Channel, Message ID, Message, Sender Type, Timestamp, Intent and Response.
- Google Sheets OAuth connection configured for n8n.
- Existing customer lookup by Email and Phone.
- IF routing: existing customer -> Update Row; new customer -> Append Row.
- Store Conversation node added after both branches.
- Website Chat and WhatsApp cross-channel testing completed.
- Duplicate-customer problems debugged and fixed.

Final CRM logic

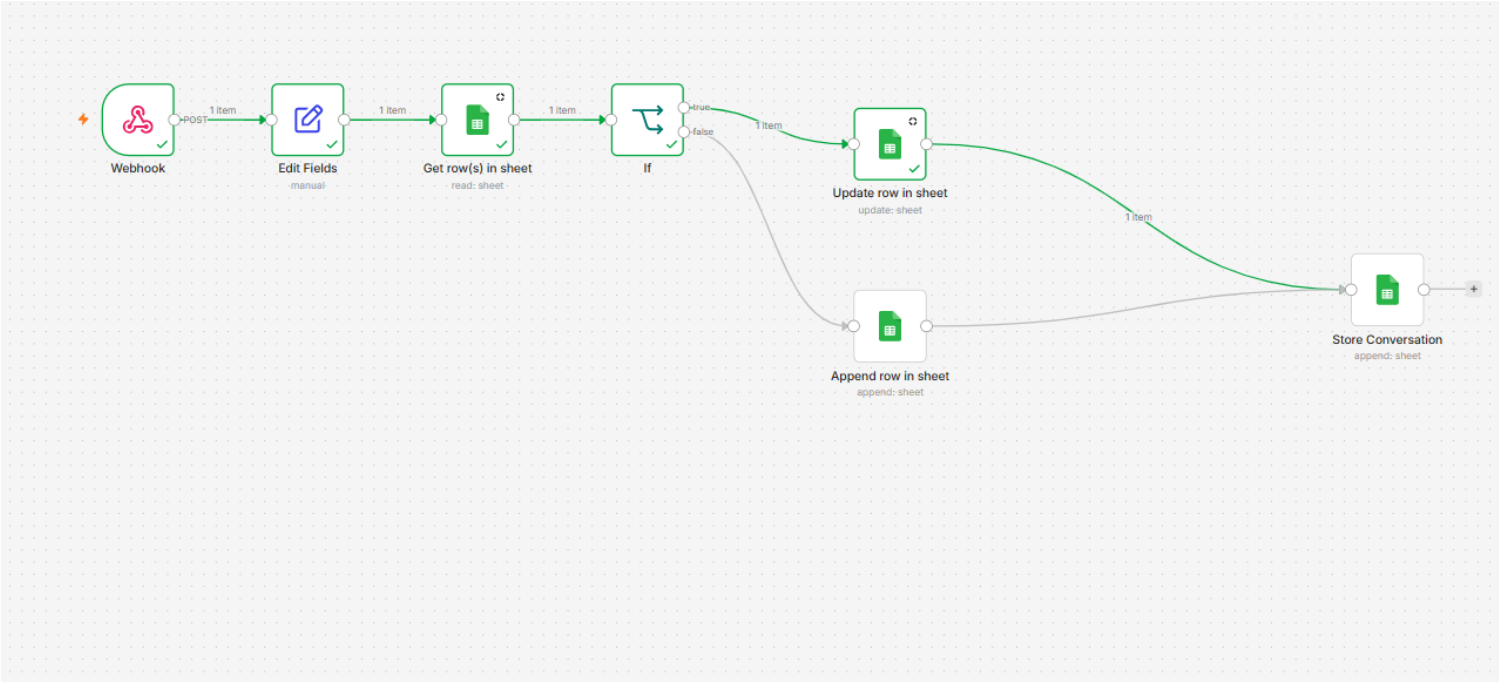
First contact -> customer not found -> Append customer. Next contact with same identity -> customer found -> Update same record. Every message -> append a new Conversations row.

2. CRM Data Model

Customers table field	Purpose
Customer ID	Unique CRM identity, e.g. CUST-83
Name	Customer name
Email	Primary identity when available
Phone	Cross-channel identity, especially WhatsApp
Company	Optional company information
Preferred Channel	Latest/preferred contact channel
First Interaction	Original first contact time
Last Interaction	Latest contact time
Customer Status	Active / other future statuses

Conversations table field	Purpose
Conversation ID	Unique conversation record ID
Customer ID	Links message to one customer profile
Channel	website_chat / email / whatsapp
Message ID	Unique incoming message ID
Message	Incoming customer text
Sender Type	customer
Timestamp	Message time
Intent	Reserved for AI classification in later phase
Response	Reserved for AI response in later phase

3. Website Chat CRM Workflow



Final Website Chat flow: Webhook -> Edit Fields -> Get Row(s) -> IF -> Update/Append -> Store Conversation.

Node-by-node role

- **Webhook:** simulated website chat POST request receive karta hai.
- **Edit Fields:** sender_name, sender_email, sender_phone, message, channel, timestamp aur message_id standard format me banata hai.
- **Get Row(s):** Customers sheet me identity search karta hai.
- **IF:** existing row mili to TRUE, warna FALSE.
- **Update Row:** existing customer ka Last Interaction / channel update karta hai.
- **Append Row:** first-time customer ka naya profile create karta hai.
- **Store Conversation:** har incoming message ko Conversations sheet me append karta hai.

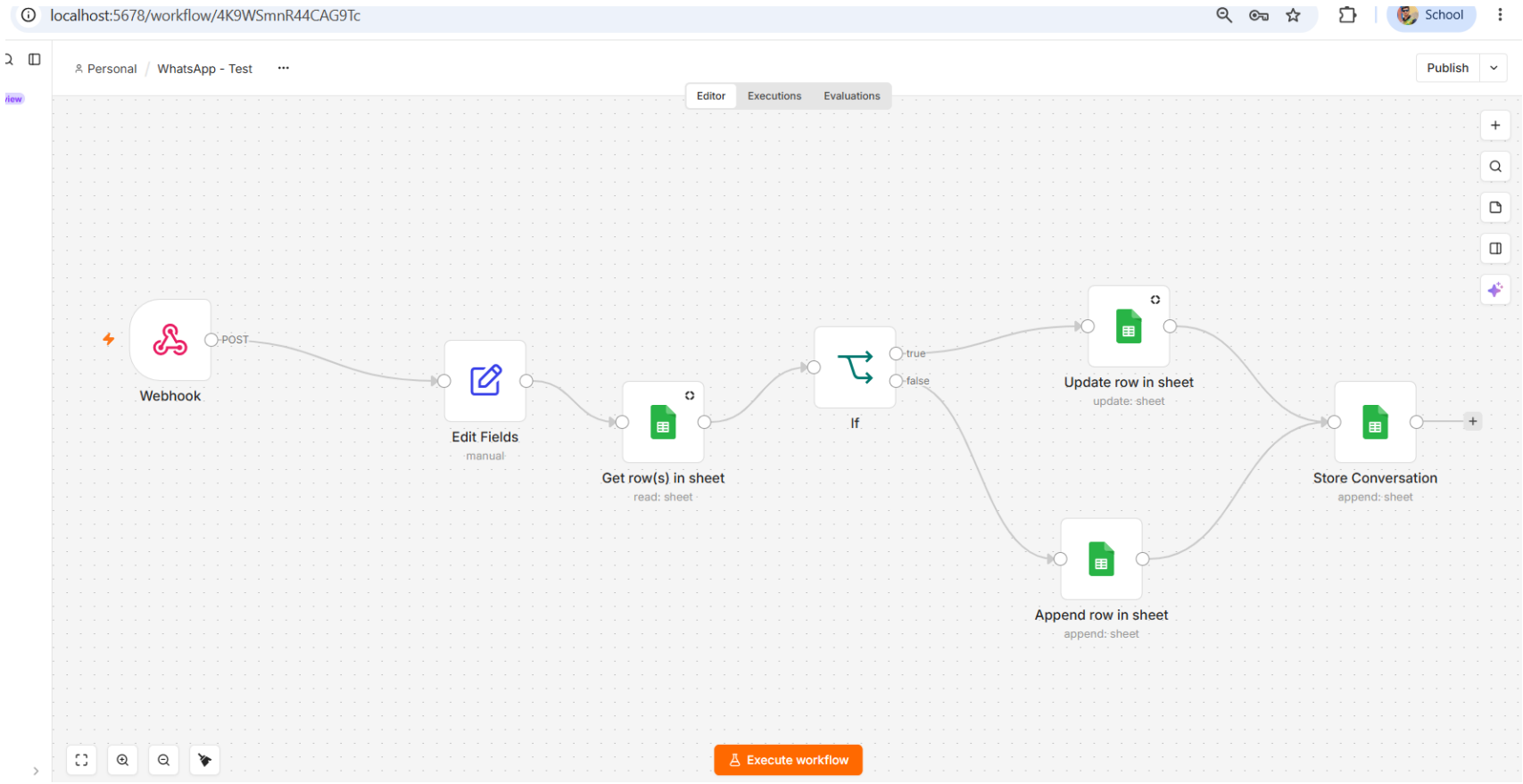
4. Duplicate Customer Problem - Debugging and Fix

Testing ke dauran same customer ke multiple CUST IDs ban rahe thay. Problem ek single bug nahi thi; multiple configuration issues ek saath identify hue.

Problem	Reason	Fix
Existing email find nahi ho rahi thi	Get Row(s) me Column galti se Customer ID select tha	Column ko Email set kiya
IF FALSE ja raha tha	IF node me extra blank condition lagi hui thi	Sirf one clean condition rakhi
Manual test sahi, full run wrong	Old/stale node input use ho raha tha	Previous nodes re-execute karke fresh data use kiya
WhatsApp par duplicate customer	Phone formatting + sign/API representation mismatch	Phone ko consistent searchable format me compare kiya
Blank Email OR filter false matches	Blank email rows bhi match ho rahi thin	Blank email ke liye sentinel value __NO_EMAIL__ use ki
Copied workflow me reference issue	Some expressions copied node context ko resolve nahi kar rahe thay	Direct/current item references verify kiye

Correct identity rule: same customer ko repeatedly run karne par new customer row create nahi honi chahiye. Customer profile one; conversation records many.

5. WhatsApp Cross-channel Workflow



WhatsApp - Test workflow copied from the working CRM pattern and connected to the same Google Sheets CRM.

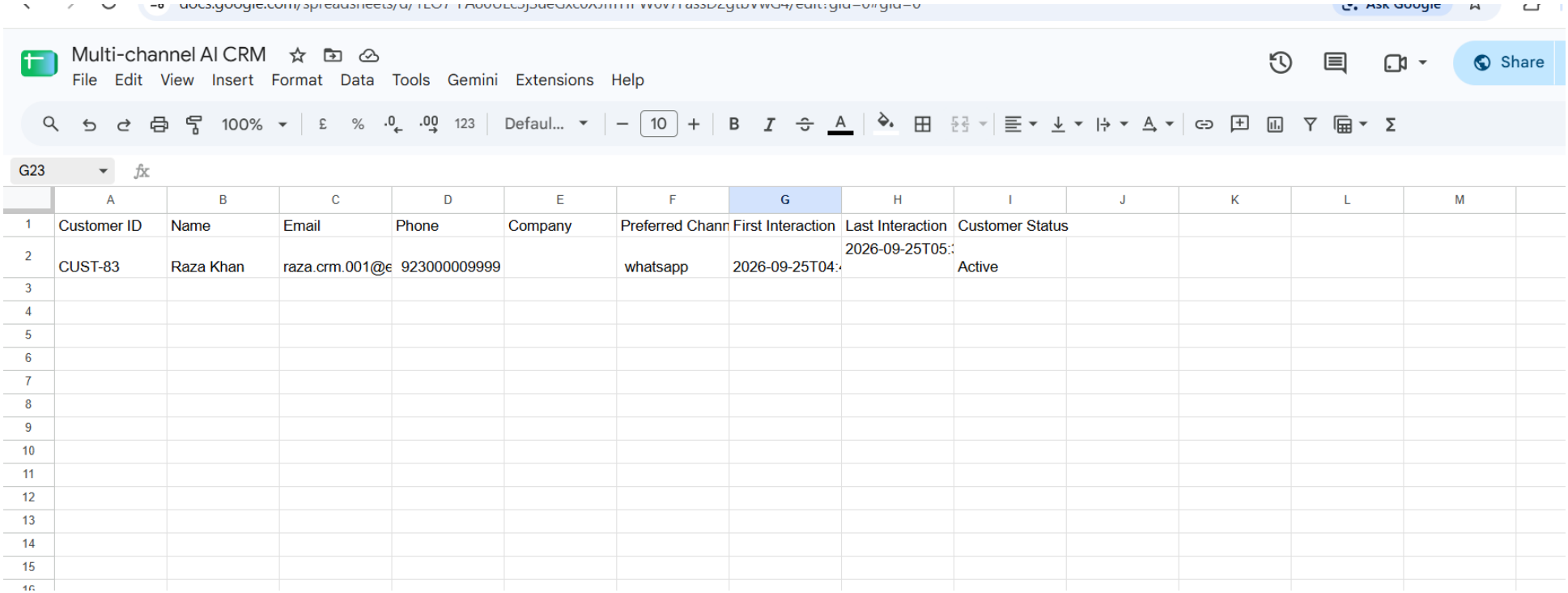
Cross-channel identity

WhatsApp message me email empty ho sakti hai. Is liye phone number ko second identity key banaya gaya. Lookup rule practical form me: Email match OR Phone match -> same existing customer.

```
Important test case
```

```
Website: Raza Khan -> same customer profile WhatsApp: same phone -> same Customer ID Result: customer duplicate nahi; sirf Preferred Channel / Last Interaction update.
```

6. Final Customers Proof



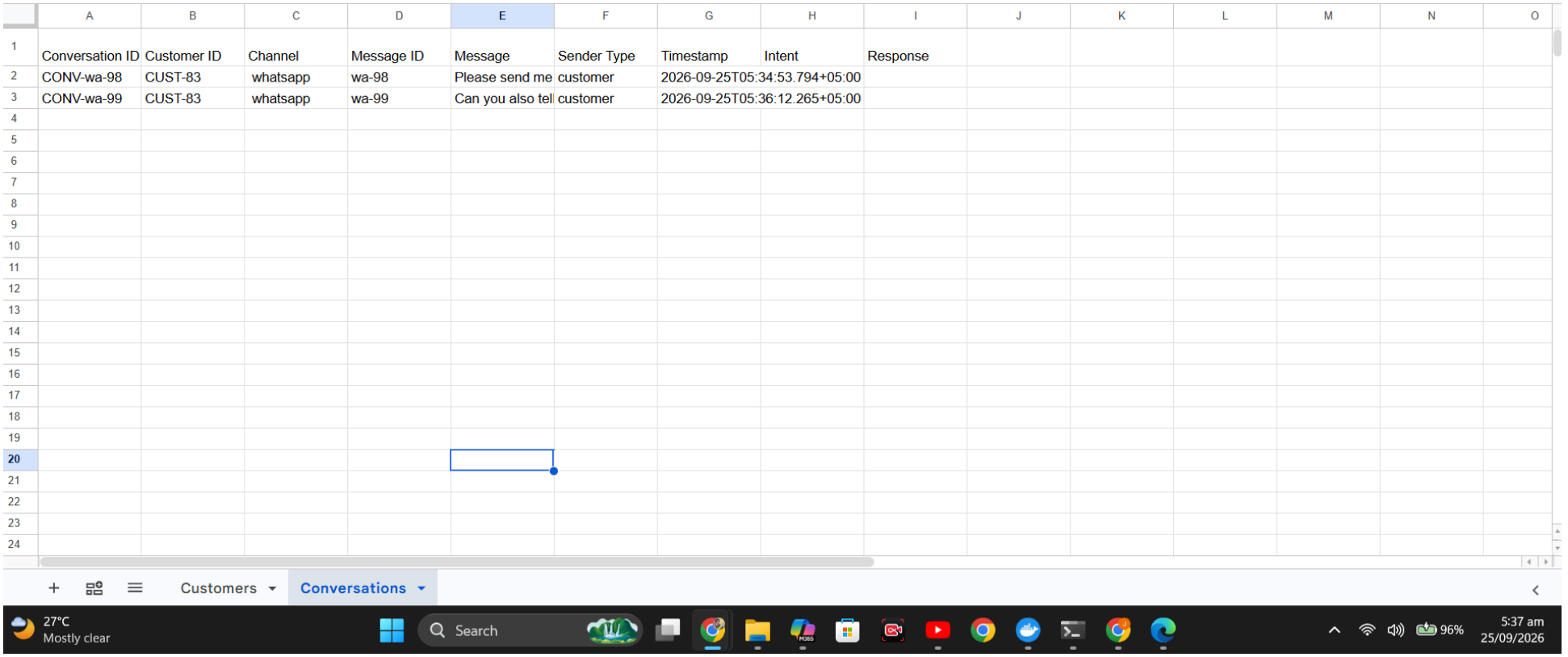
	A	B	C	D	E	F	G	H	I
1	Customer ID	Name	Email	Phone	Company	Preferred Chann	First Interaction	Last Interaction	Customer Status
2	CUST-83	Raza Khan	raza.crm.001@e	923000009999		whatsapp	2026-09-25T04:	2026-09-25T05:	Active

Final Customers sheet proof: Raza Khan remains one centralized customer profile (CUST-83). Preferred Channel updated to WhatsApp.

Verification

- Customer ID remains the same: CUST-83.
- Same customer ka duplicate profile final state me create nahi hua.
- Preferred Channel WhatsApp update hua.
- First Interaction preserved; Last Interaction latest contact se update hua.

7. Final Conversations Proof



	A	B	C	D	E	F	G	H	I
1	Conversation ID	Customer ID	Channel	Message ID	Message	Sender Type	Timestamp	Intent	Response
2	CONV-wa-98	CUST-83	whatsapp	wa-98	Please send me	customer	2026-09-25T05:34:53.794+05:00		
3	CONV-wa-99	CUST-83	whatsapp	wa-99	Can you also tel	customer	2026-09-25T05:36:12.265+05:00		

Conversations sheet proof: multiple WhatsApp messages stored separately, both linked to CUST-83.

Why this design matters

Customers table identity ko deduplicate karta hai, jabke Conversations table complete history rakhta hai. Is se AI later phase me previous customer context, intent aur response history use kar sakta hai.

Intent aur Response blank hona abhi correct hai. Ye fields later centralized AI understanding / response phases me populate hongi.

8. Testing Commands and Submission Checklist

Example Website Chat test

```
curl -X POST http://localhost:5678/webhook-test/website-chat -H "Content-Type: application/json" -d "{\"name\":\"Raza
Khan\",\"email\":\"raza.crm.001@example.com\",\"phone\":\"+923000009999\",\"message\":\"I need a business
website\",\"channel\":\"website_chat\"}"
```

Example WhatsApp test

```
curl -X POST http://localhost:5678/webhook-test/whatsapp-incoming -H "Content-Type: application/json" -d "{\"name\":\"Raza
Khan\",\"email\":\"\",\"phone\":\"+923000009999\",\"message\":\"Can you also tell me the expected project completion
time?\",\"channel\":\"whatsapp\"}"
```

Submission screenshots checklist

- Full Website Chat workflow.
- Full WhatsApp workflow.
- Get Row(s) filters showing Email + Phone logic.
- IF routing showing TRUE -> Update and FALSE -> Append.
- Customers sheet showing one customer profile.
- Conversations sheet showing multiple messages linked to same Customer ID.

Security note: OAuth Client ID/Secret screenshots ko submission me expose na karein.

9. Learning Summary and Final Copy-paste Description

What I learned

- Multi-channel systems me channel-specific input ko standardized format me convert karna.
- CRM deduplication ke liye reliable identity keys ka use.
- Google Sheets ko lightweight simulated CRM/database ke taur par integrate karna.
- IF branching se create-vs-update logic implement karna.
- Customer profile aur conversation history ko separate tables me maintain karna.
- Cross-channel customer identity ko Email/Phone ke zariye unify karna.
- n8n debugging me node input/output ko exact execution ke context me inspect karna.

Internship submission description (copy/paste)

```
Implemented Phase 3 of the Multi-channel AI Agent by creating a simulated centralized CRM in Google Sheets and connecting it with n8n. I created Customers and Conversations tables, added customer search logic, and implemented automatic create/update routing. New customers are added only once, while existing customers are updated instead of duplicated. Every incoming message is stored as a separate conversation record. I also tested the same fictional customer across Website Chat and WhatsApp using email/phone identity matching. The final system maintains one centralized customer profile with multiple conversation records, and I troubleshot duplicate-record, IF-condition, lookup-filter and phone-formatting issues during implementation.
```

Phase 3 Day 10 status: COMPLETE for the assigned requirements.

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 4

Centralized AI Understanding

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	COMPLETE AND TESTED

One reusable Central AI Brain for Website Chat, Email and WhatsApp with structured intent/decision output.

1. Phase 4 Objective - Centralized AI Understanding

Phase 4 created one reusable Central AI Brain. Website Chat, Email and WhatsApp no longer need separate AI logic; they call one sub-workflow with the same input schema.

```
Website Chat --\
WhatsApp ------> Central AI Brain -> LLM -> Structured Output
Email --------/
```

Central AI Brain input schema

Field	Purpose
customer_id	CRM customer identifier
channel	website_chat / whatsapp / email
message	Current customer message
customer_name	Customer name
email	Customer email if available
phone	Customer phone if available
previous_context	Previous conversation context / summary

AI components

• When Executed by Another Workflow trigger defined the reusable sub-workflow contract.

• Basic LLM Chain held the prompt and business instructions.

• Google Gemini Chat Model was connected as the model provider.

• Structured Output Parser forced machine-readable, consistent fields.

Intent categories

• Product Inquiry

• Pricing

• Technical Support

• Order Information

• Complaint

• Appointment

• General Question

• Human Assistance

Structured decision output

• intent and urgency

• required_information

• preferred_action

• response_type

• human_escalation_required

Caller workflow integration

```
Store Conversation -> Execute Workflow (Central AI Brain)
Source: Database
Mode: Run once with all items
```

Important fixes

• Wrong node type in caller workflow: Execute Workflow must call the Central AI Brain; the caller should not use the sub-workflow trigger.

• sender_name / sender_email / sender_phone payload names were mapped correctly after empty/undefined fields appeared.

• Same pricing question was tested through multiple channels to prove centralized logic.

Phase 4 was completed across Day 13 and Day 14. Both original implementation documents are appended after this summary.

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

Day 13 - Phase 4: Centralized AI Understanding

Student Notes + Implementation Summary + Submission Proof Guide

Status: IMPLEMENTED AND TESTED SUCCESSFULLY

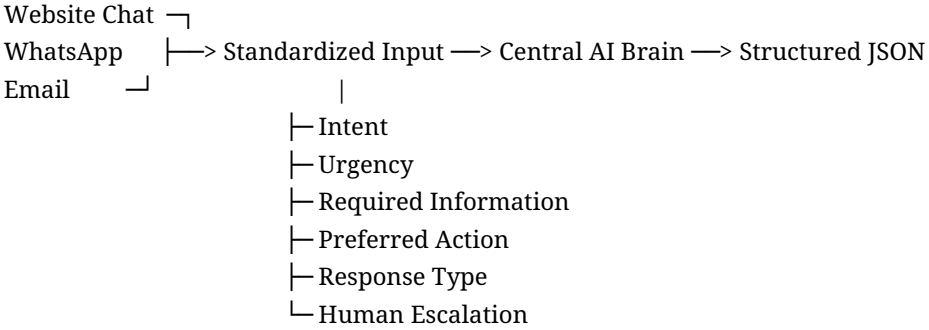
Core Stack: n8n + Google Gemini + Structured Output Parser + Multi-channel CRM Context

1. Aaj ka Task Kya Tha?

Phase 4 ka objective ek centralized AI brain banana tha jo Website Chat, WhatsApp aur Email se aane wale messages ko same logic se samjhe. Har channel ke liye separate chatbot banane ke bajaye, ek shared AI processing workflow use kiya gaya.

- Approved LLM provider select karna.
- AI credentials securely n8n me configure karna.
- Standardized customer/message input format banana.
- Central AI ko message + customer context dena.
- System prompt se AI ka role define karna.
- Customer intent aur urgency identify karwana.
- Preferred action aur missing/required information nikalna.
- Structured JSON output generate karwana.
- Same question ko different channels se test karna.

2. Final Architecture



Important concept: channel change ho sakta hai, lekin AI logic aur output schema same rehta hai. Isi wajah se system genuinely centralized hai.

3. Step-by-Step Implementation

Step 1 - Central AI Brain workflow create kiya

Existing Multi-channel AI Agent project ke andar ek new workflow banaya:

> Workflow Name: Central AI Brain

Purpose: Website Chat, WhatsApp aur Email workflows ki AI processing ko ek common workflow me centralize karna.

Step 2 - LLM Provider aur Credential

- Provider selected: Google Gemini.
- Google Gemini API credential n8n Credentials me securely store kiya.
- API key workflow text/prompt me hard-code nahi ki gayi.
- Gemini Chat Model ko Basic LLM Chain ke Model connector ke sath attach kiya.

Step 3 - Standardized Workflow Input Schema

"When Executed by Another Workflow" trigger use kiya aur ye fields define kiye:

Field	Type	Purpose
customer_id	String	CRM customer identifier
channel	String	website_chat / whatsapp / email
message	String	Current customer message
customer_name	String	Customer name
email	String	Customer email if available
phone	String	Customer phone if available
previous_context	String	Previous conversation context/summary

Standardized input ka faida: har channel same field names aur same structure me data AI ko deta hai.

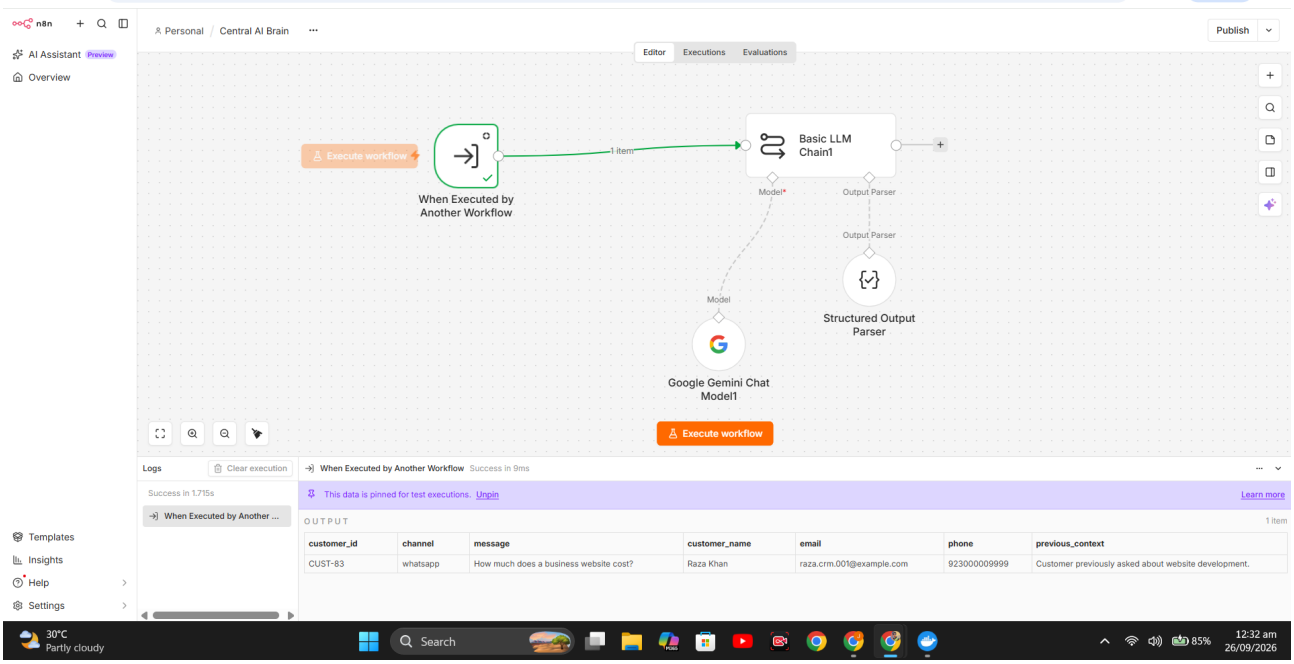


Figure 1 - Pinned/mock standardized input used for AI testing.

Step 4 - Basic LLM Chain configure kiya

Basic LLM Chain ko main AI processing node ke taur par use kiya. Prompt source ko "Define below" kiya, kyun ke input Chat Trigger se nahi balki another workflow se aa raha tha.

> Customer ID: {{ $json.customer_id }}
> Channel: {{ $json.channel }}
> Customer Name: {{ $json.customer_name }}
> Email: {{ $json.email }}
> Phone: {{ $json.phone }}
>
> Previous Conversation Context:
> {{ $json.previous_context }}
>
> Current Customer Message:
> {{ $json.message }}

Step 5 - System Prompt

System prompt me AI ko NexaTech Solutions ka centralized multi-channel business assistant define kiya gaya.

- Website Chat, WhatsApp aur Email messages ko same business logic se process karna.
- Current message + previous conversation context dono use karna.
- Missing information identify karna.
- Human escalation tab karna jab customer explicitly human maange, serious complaint ho, ya issue automation se safely handle na ho.
- Cross-channel classification consistent rakhna.

Defined intent categories:

Product Inquiry
Pricing
Technical Support
Order Information
Complaint
Appointment
General Question
Human Assistance

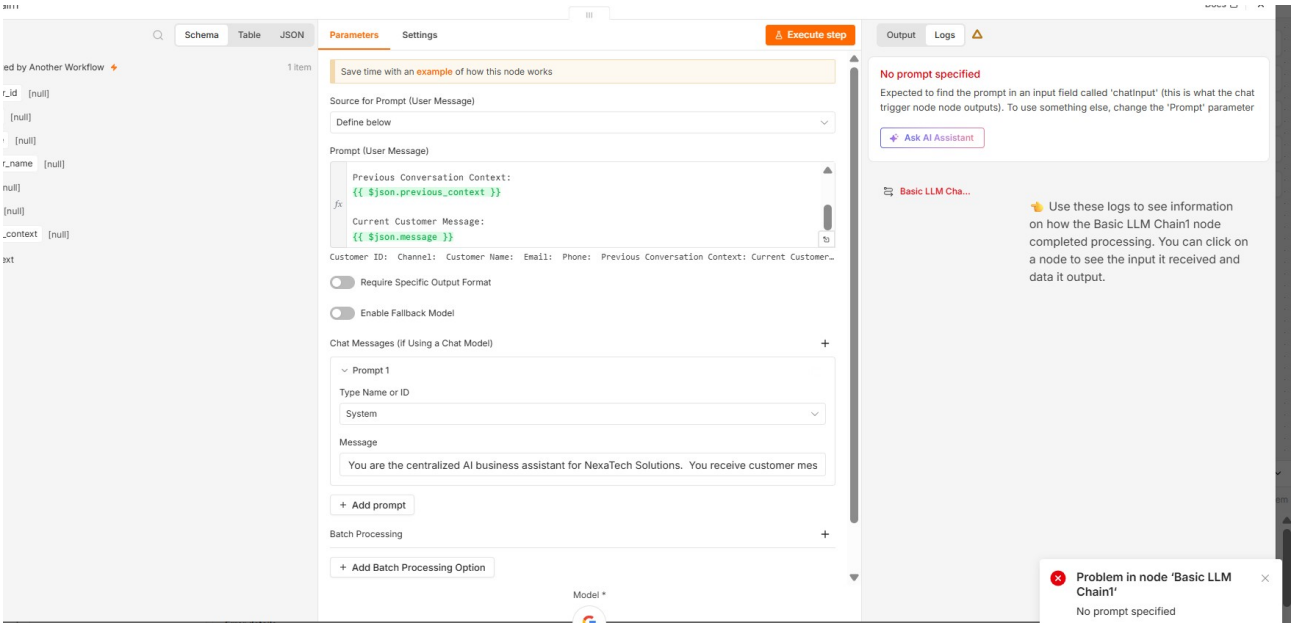


Figure 2 - Basic LLM Chain with standardized prompt and System message configuration.

Step 6 - Structured Output Parser

"Require Specific Output Format" enable karke Structured Output Parser attach kiya. JSON example:

```
{
  "intent": "Pricing",
  "urgency": "normal",
  "required_information": ["website type", "number of pages"],
  "preferred_action": "Provide pricing information",
  "response_type": "automatic",
  "human_escalation_required": false
}
```

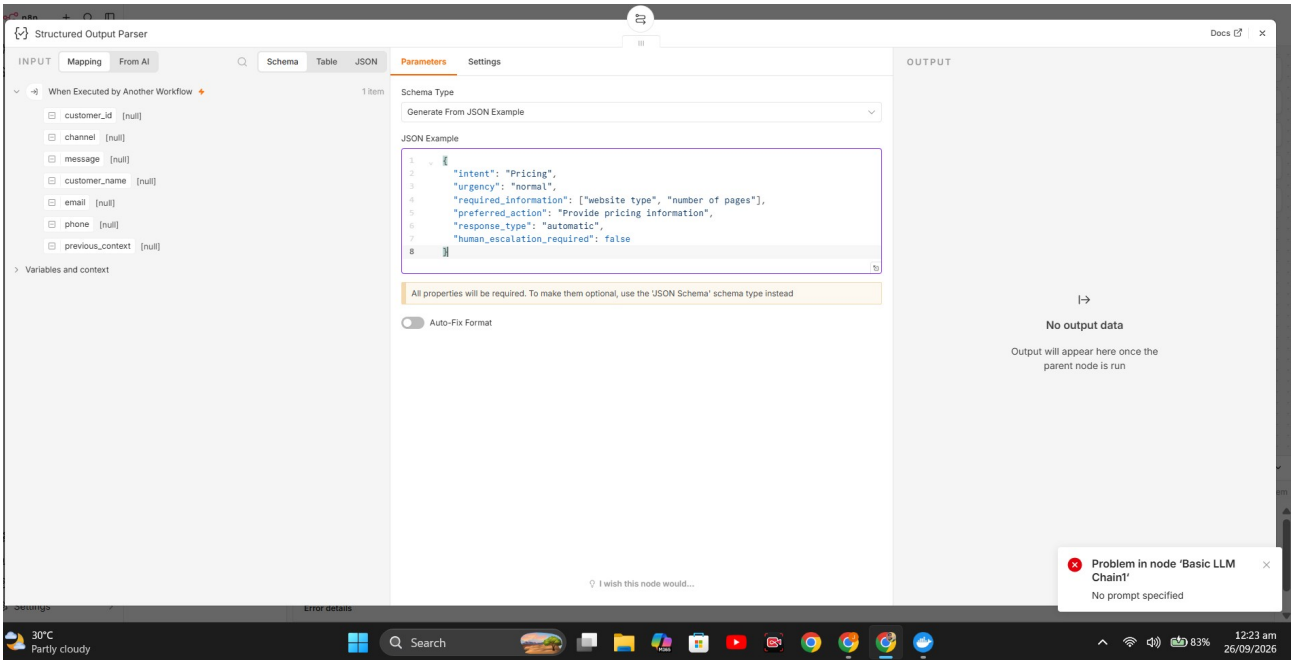


Figure 3 - Structured Output Parser schema used to force clean machine-readable AI output.

Step 7 - Final Central AI Brain Flow

When Executed by Another Workflow
|
v
Basic LLM Chain
|
+-- Google Gemini Chat Model
+-- Structured Output Parser

Is structure ka result: same centralized AI logic ko multiple source workflows reuse kar sakte hain.

4. Testing and Results

Test A - WhatsApp

Test input:

customer_id: CUST-83
channel: whatsapp
message: How much does a business website cost?
customer_name: Raza Khan
previous_context: Customer previously asked about website development.

Successful AI result me intent Pricing classify hua, urgency identify hui, required information nikli aur human escalation false return hua.

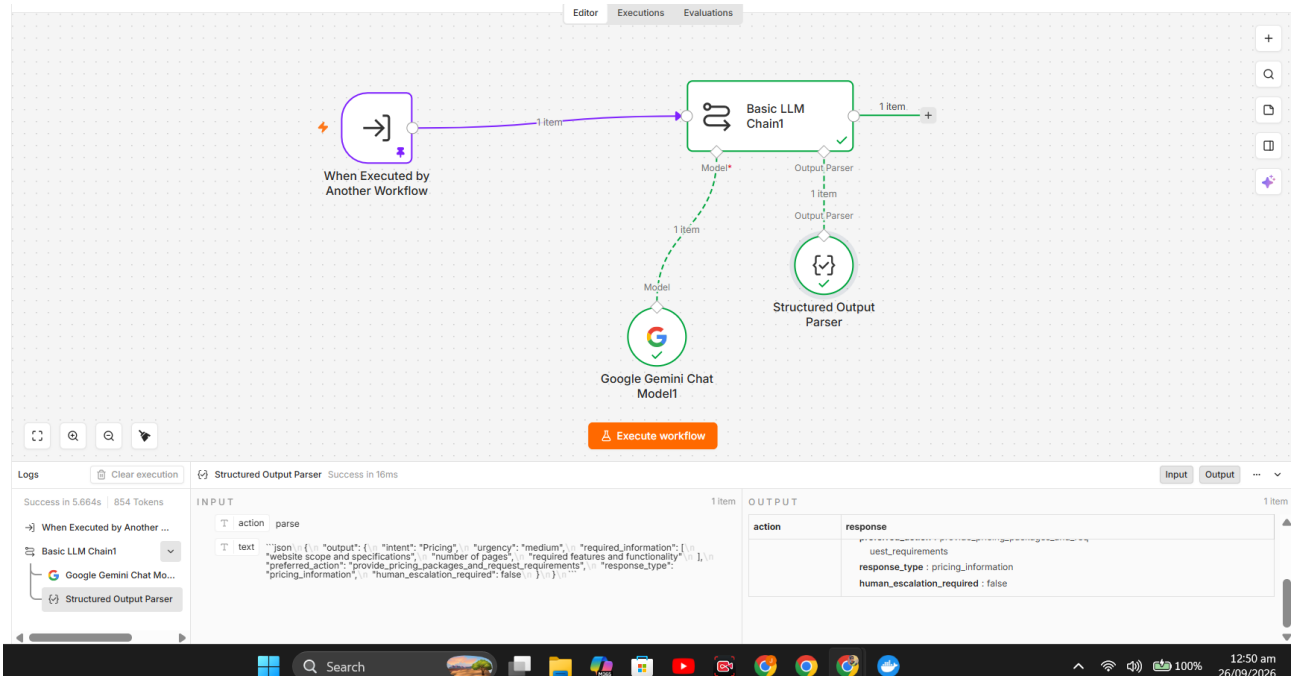


Figure 4 - Successful WhatsApp-channel AI classification with structured output.

Test B - Website Chat

Cross-channel consistency test me same customer question rakha aur sirf channel change kiya:

channel: website_chat
message: How much does a business website cost?

Result: AI ne dobara Pricing intent classify kiya aur same centralized logic apply ki. Isse Phase 4 ka "one AI brain" objective verify hua.

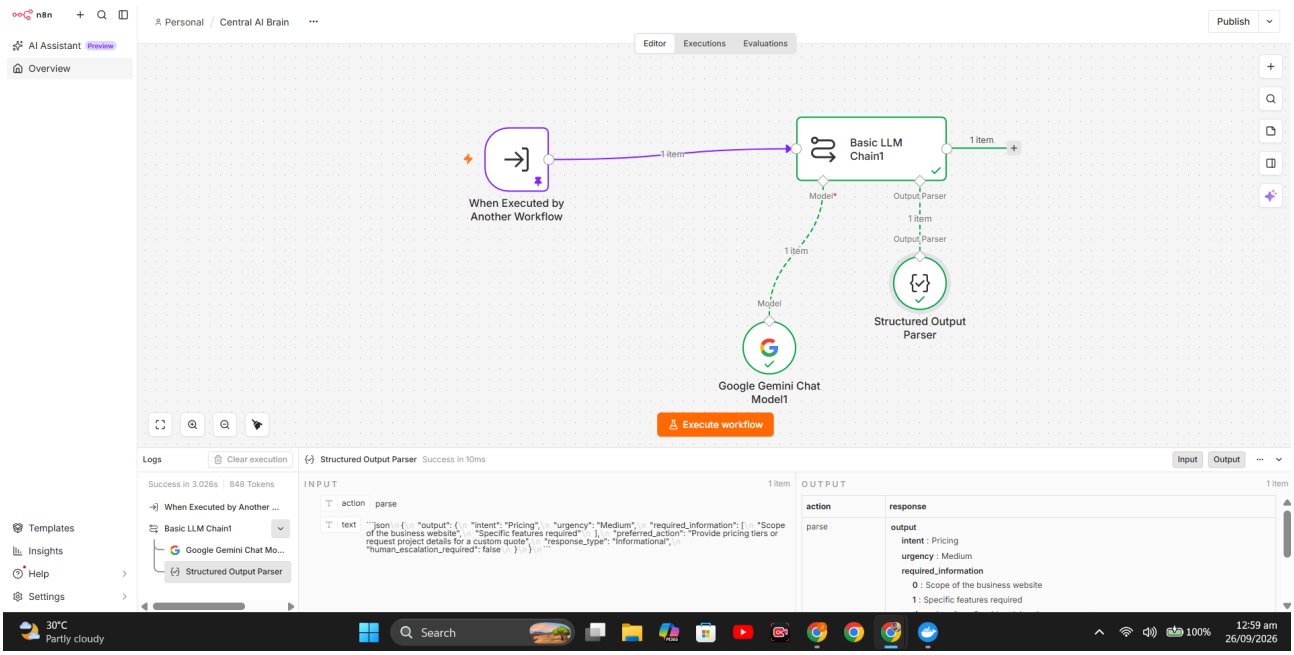


Figure 5 - Successful Website Chat test using the same centralized AI logic.

5. Errors Faced and How They Were Solved

Problem	Reason	Solution
No prompt specified	Basic LLM Chain was expecting chatInput from a Chat Trigger.	Prompt source changed to Define below and standardized fields were used.
Output parser required	Require Specific Output Format was enabled but parser was missing.	Structured Output Parser added and JSON schema defined.
403 - Project has been denied access	Initial Gemini project/account credential did not have usable access.	A working Gemini account/API key credential was configured.
503 - Service unavailable / high demand	Selected Gemini Flash model was temporarily overloaded.	Retried and used an available lighter/stable model when needed.
404 - resource/model not found	A Veo video-generation model was accidentally selected for text generation.	Switched back to a Gemini text Chat Model supported by generateContent.

6. Phase 4 Requirement Checklist

- Step 1: LLM provider selected - Google Gemini - DONE
- Step 2: API credential configured securely in n8n - DONE
- Step 3: Standardized message format created - DONE
- Step 4: Standardized data sent to AI model - DONE
- Step 5: Multi-channel business assistant system prompt created - DONE
- Step 6: Customer intent identification implemented - DONE
- Step 7: Required intent categories defined - DONE
- Step 8: Preferred action and important/required information identified - DONE
- Step 9: Structured JSON output implemented - DONE
- Step 10: Same question tested through WhatsApp and Website Chat - DONE

7. Screenshots to Submit

1. Full Central AI Brain workflow showing Trigger -> Basic LLM Chain with Gemini + Structured Output Parser.
2. Workflow Input Schema showing customer_id, channel, message, customer_name, email, phone, previous_context.
3. Basic LLM Chain prompt + System prompt configuration.
4. Structured Output Parser JSON schema.
5. Successful WhatsApp AI output.
6. Successful Website Chat AI output with the same Pricing intent logic.

8. Ready-to-Paste Submission Description

Implemented Phase 4 - Centralized AI Understanding for my Multi-channel AI Agent project. I created a reusable Central AI Brain workflow in n8n using Google Gemini. The workflow accepts a standardized input containing customer ID, channel, message, customer information, and previous conversation context. A centralized system prompt classifies customer intent across Product Inquiry, Pricing, Technical Support, Order Information, Complaint, Appointment, General Question, and Human Assistance. The AI also identifies urgency, required information, preferred action, response type, and whether human escalation is required. I connected a Structured Output Parser so the AI returns consistent JSON. I successfully tested the same pricing question through WhatsApp and Website Chat and verified that both channels were processed by the same centralized AI logic. During implementation I also troubleshot prompt configuration, structured output parsing, Gemini project/API access, temporary model overload, and an incorrect model selection.

9. Important Learning

- Centralized AI architecture is easier to maintain than separate AI bots for every channel.
- Standardized data contracts are important because every channel can provide data differently.
- System prompts define business rules; output parsers define machine-readable response structure.
- Previous conversation context improves understanding of short follow-up messages.
- API/model errors can be external; workflow logic should be debugged separately from provider availability.
- Cross-channel testing is necessary to prove that the AI logic is actually centralized.

10. Final Status

DAY 13 / PHASE 4 IMPLEMENTATION: COMPLETE

Next internship phase should only be started when it is officially assigned. Existing Phase 1-4 workflows should remain connected and preserved.

Multi-channel AI Agent

Day 14 - Phase 4 Remaining Functionality

Centralized AI Understanding - Integration, Testing, and Completion

Project	Multi-channel AI Agent
Phase	Phase 4 - Centralized AI Understanding
Day 14 Goal	Connect remaining channel workflows to the existing Central AI Brain and test complete flows.
Final Status	WhatsApp, Website Chat, and Email successfully processed through one shared Central AI Brain.

1. What Day 14 Required

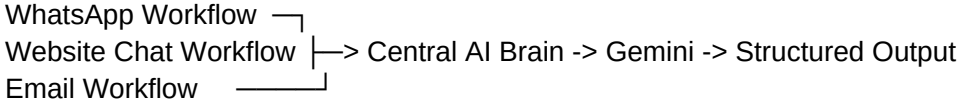
- Continue the same Phase 4 implementation from Day 13 - no new project and no rebuild from scratch.
- Complete the remaining functionality by connecting the previously created Central AI Brain with the existing channel workflows.
- Verify data flow, credentials, AI processing, structured output, and end-to-end workflow execution.
- Test using meaningful data and capture screenshots that prove the implementation works.
- Do not start Phase 5.

2. Day 13 Work Reused

- Google Gemini configured as the LLM provider inside n8n.
- Central AI Brain workflow created using When Executed by Another Workflow -> Basic LLM Chain -> Google Gemini Chat Model -> Structured Output Parser.
- Standard input schema defined: customer_id, channel, message, customer_name, email, phone, previous_context.
- System prompt defined the AI as a multi-channel business assistant.
- Intent categories included Product Inquiry, Pricing, Technical Support, Order Information, Complaint, Appointment, General Question, and Human Assistance.
- Structured AI output included intent, urgency, required_information, preferred_action, response_type, and human_escalation_required.

3. Day 14 Architecture

Final logical design:



All three channels reuse the same AI workflow. No separate AI chatbot or separate AI logic was created for each channel.

4. Central AI Brain Input Schema

Field	Type
customer_id	String

channel	String
message	String
customer_name	String
email	String
phone	String
previous_context	String

5. Execute Workflow Integration

Each channel workflow calls the existing Central AI Brain using the Execute Workflow node after the conversation is stored in Google Sheets CRM.

- Source: Database
- Workflow: Central AI Brain
- Mode: Run once with all items
- The same standardized fields are mapped into the Central AI Brain.

6. WhatsApp Integration and Test

- Existing flow: Webhook -> Edit Fields -> Get row(s) -> IF -> Update/Append Customer -> Store Conversation -> Call Central AI Brain.
- Webhook test path used: /webhook-test/whatsapp-incoming.
- Test message: How much does a business website cost?
- Channel value: whatsapp.
- Final customer values reached the AI workflow correctly: Customer ID, channel, message, name, email, and phone.
- AI result: intent = Pricing; structured required information and preferred action returned; human escalation = false.

7. WhatsApp Issue and Fix

Initial issue: customer_name and email were empty and phone was undefined in the Central AI Brain input.

Root cause: the Webhook payload used sender_name, sender_email, and sender_phone, while Edit Fields originally tried to read body.name, body.email, and body.phone.

- sender_name -> {{ $json.body.sender_name || '' }}
- sender_email -> {{ $json.body.sender_email || '' }}
- sender_phone -> {{ $json.body.sender_phone || '' }}
- For the AI call, phone was finally mapped directly from Webhook body: {{ $('Webhook').item.json.body.sender_phone }}

8. Website Chat Integration and Test

- Existing Website Chat workflow was connected to the same Central AI Brain using Execute Workflow.
- The stored conversation provided Customer ID, channel, and message.
- Customer name, email, and phone were mapped from the Website Chat Webhook body.
- Test question: How much does a business website cost?
- Channel: website_chat.
- AI result: intent = Pricing; urgency = Low; structured required information returned; response type = Informational; human escalation = false.

9. Email Integration and Test

- Existing Email workflow was connected to the same Central AI Brain.
- Webhook test path used: /webhook-test/email-incomin (as configured in the workflow).
- Test payload used channel = email.
- Customer ID, email channel, message, customer email, and phone were passed successfully.
- AI result: intent = Pricing; urgency = Low; required information detected; preferred action returned; response type = Informational; human escalation = false.

10. Command-Line Testing Clarification

CMD/PowerShell/curl was used only to simulate incoming channel data during development. This is a testing method, not the final user experience.

- Real Website: website chat/form -> webhook/API -> same n8n workflow.
- Real WhatsApp: WhatsApp provider/API -> same n8n workflow.
- Real Email: Gmail/IMAP/Email trigger -> same n8n workflow.
- The CRM, Central AI Brain, Gemini prompt, structured parser, and routing logic will not need to be rebuilt when live channels are connected.

11. Troubleshooting Performed

Issue	Reason	Fix
Wrong trigger added to caller workflow	When Executed by Another Workflow was added instead of Execute Workflow	Removed trigger and used Execute Workflow to call Central AI Brain.
Customer details empty	Incorrect Webhook property names	Mapped sender_name, sender_email, sender_phone correctly.
Phone undefined	Source expression did not resolve correctly	Mapped phone directly from Webhook body.
curl -H failed in PowerShell	PowerShell interpreted curl as Invoke-WebRequest	Used curl.exe or PowerShell-compatible request syntax.
JSON file not found	CMD working directory did not contain the file	Used inline JSON or correct file path.

12. Final Day 14 Verification

Check	Status
WhatsApp -> Central AI Brain	PASSED
Website Chat -> Central AI Brain	PASSED
Email -> Central AI Brain	PASSED
Same centralized Gemini logic used across channels	PASSED
Structured output returned	PASSED
Intent classification	PASSED
Human escalation field returned	PASSED
CRM conversation stored before AI processing	PASSED

13. Screenshots to Submit

1. Full WhatsApp workflow showing Store Conversation -> Call Central AI Brain and successful green execution.

2. WhatsApp Call Central AI Brain screen showing mapped customer data and structured AI output (Pricing).
3. Full Website Chat workflow showing Store Conversation -> Call Central AI Brain.
4. Website Chat Call Central AI Brain screen showing channel = website_chat and successful structured output.
5. Email Call Central AI Brain screen showing channel = email and successful structured output.
6. Central AI Brain workflow canvas showing trigger -> Basic LLM Chain -> Gemini -> Structured Output Parser.
7. Optional: one screenshot of the Central AI Brain input schema with all 7 fields.

Best submission set: 5-7 screenshots. Prefer screens that show actual values and successful outputs, not only empty configuration panels.

14. Ready-to-Paste Submission Description

Day 14 - Phase 4 Completion

I completed the remaining Phase 4 functionality of my Multi-channel AI Agent by extending the existing Central AI Brain instead of rebuilding the project. I connected the WhatsApp, Website Chat, and Email workflows to the same centralized n8n AI workflow using Execute Workflow. Each channel passes a standardized message containing customer ID, channel, message, customer information, and context to Google Gemini. The AI returns structured output including intent, urgency, required information, preferred action, response type, and human escalation requirement. I tested the same pricing question through multiple channels and verified that all channels were processed using the same centralized AI logic. I also fixed field-mapping and command-line testing issues during implementation. The final channel-to-AI flows executed successfully and Phase 4 is now integrated and tested.

15. Final Status

PHASE 4 - COMPLETED AND TESTED

The current implementation proves centralized AI understanding across all three project channels. Live production channel providers can be connected later without rebuilding the core CRM or AI Brain.

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 5

Centralized Knowledge Base and Grounded AI Responses

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	IMPLEMENTED / DAY 15 PROOF INCLUDED

Intent-based routing to approved business knowledge, channel rules and grounded final customer responses.

1. Phase 5 Objective - Centralized Knowledge Base and Grounded Responses

Phase 5 extended the Central AI Brain from “understand and classify” into “retrieve approved business knowledge and generate grounded customer-facing responses”. The architecture used the structured intent from Phase 4 to route each request into the correct knowledge and response branch.

Routing architecture

```
Central AI Brain structured intent
|
Switch
+-----+-----+-----+-----+-----+-----+-----+-----+
| | | | | | | |
Product Pricing Tech Order Complaint Appt General Human
| | | | | | | |
Relevant Google Sheets knowledge + Channel Rules
|
Final Grounded AI Response
```

Knowledge sources used

• Services / product knowledge for Product Inquiry.

• Pricing / packages / commercial information for Pricing.

• Troubleshooting / support knowledge for Technical Support.

• Order-information data for Order Information.

• Complaint, policy, payment/refund and escalation knowledge for Complaint.

• Appointment information for Appointment.

• General knowledge rules for General Question.

• Routing and escalation rules for Human Assistance.

• Channel-specific response rules so WhatsApp, Website and Email can use different response style constraints while keeping the same facts.

Core nodes and pattern

```
Switch -> Get row(s) in sheet -> Aggregate -> Get more approved rows -> Aggregate -> Channel Rules ->
Final AI Response
```

The exact number of knowledge lookups varied by branch. Aggregation nodes combined approved rows before the final response prompt so the model could answer using known business data instead of inventing unsupported facts.

Grounding and safety rules

• Use only approved facts from knowledge sheets.

• Do not invent prices, policies, contacts, delivery promises or outcomes.

• Technical Support asks for safe diagnostic details and never requests passwords/API keys.

• Complaint flow does not automatically approve refunds; it follows policy and escalation rules.

• Human Assistance identifies the correct department and avoids fake handoff confirmations.

Test evidence from Day 15

• Complete Central AI Brain architecture screenshot.

• Product Inquiry final grounded response.

• Pricing final grounded response.

• Technical Support secure troubleshooting response.

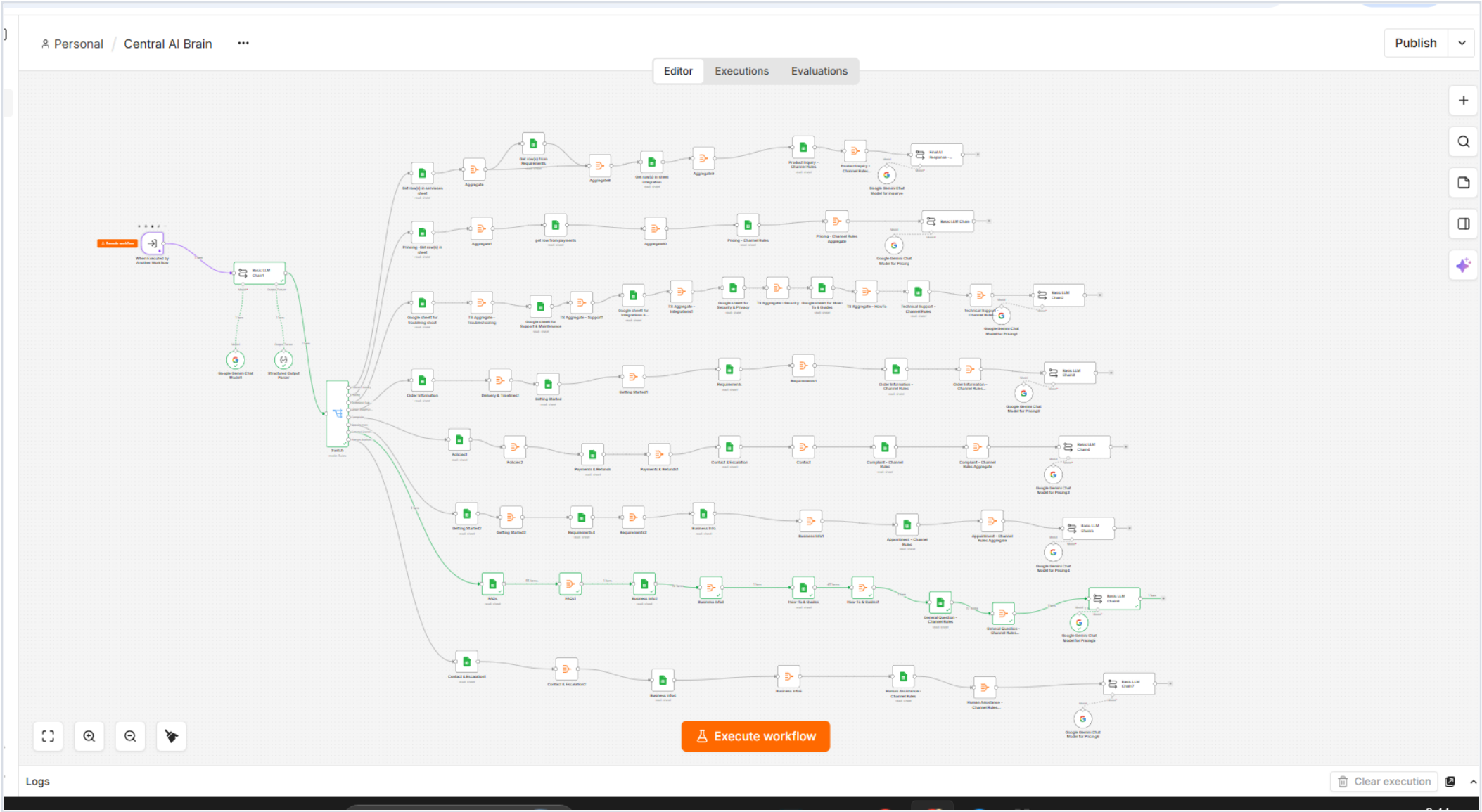
• Complaint policy/escalation response.

• Human Assistance routing/escalation response.

Phase 5 result: intent classification now drives knowledge retrieval and grounded responses. The six selected Day 15 proof screenshots are appended after these notes.

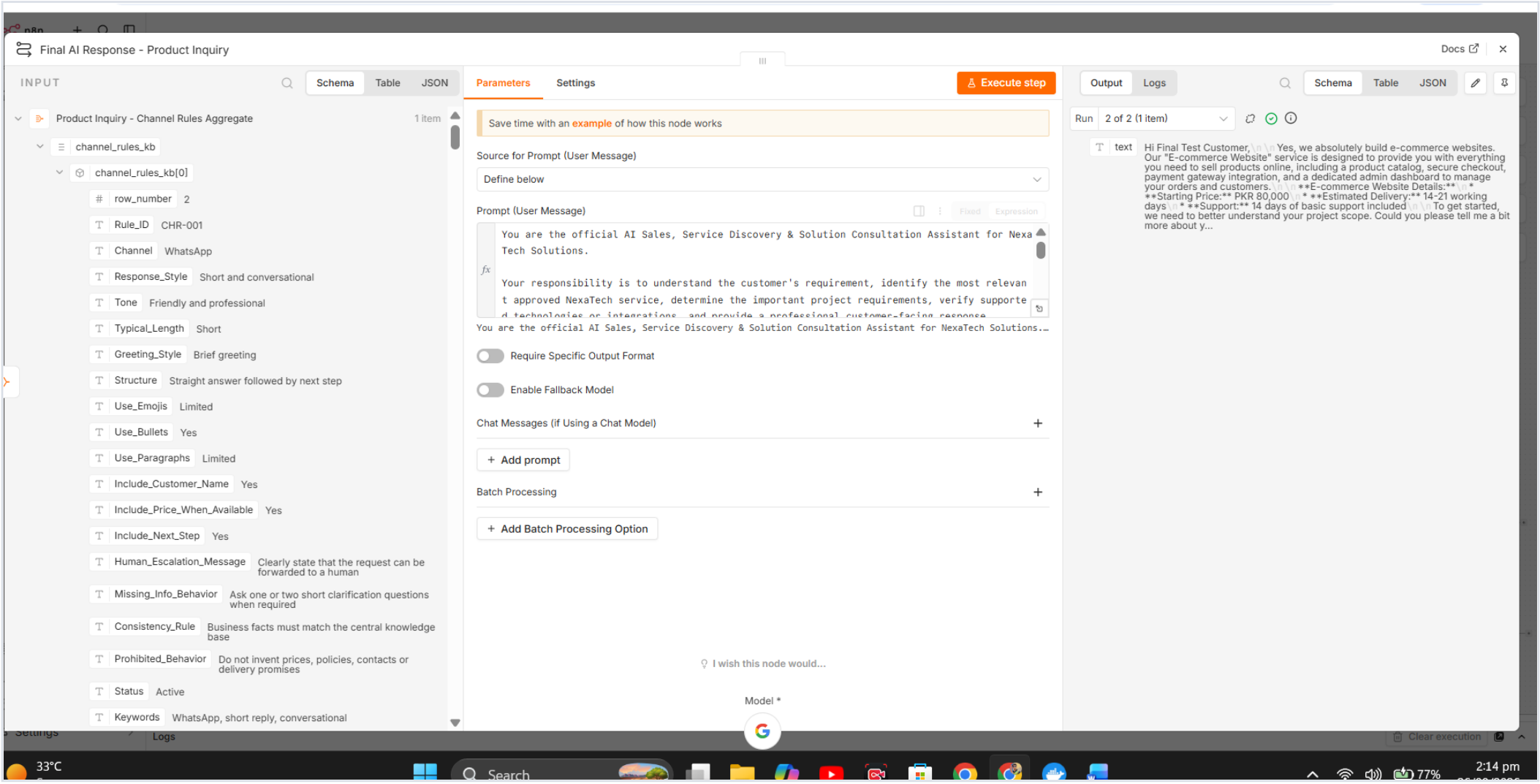
1. Complete Central AI Brain Architecture

Final n8n workflow showing centralized intent classification, 8 routed intents, relevant knowledge-base retrieval, channel response rules, and grounded Gemini response branches.



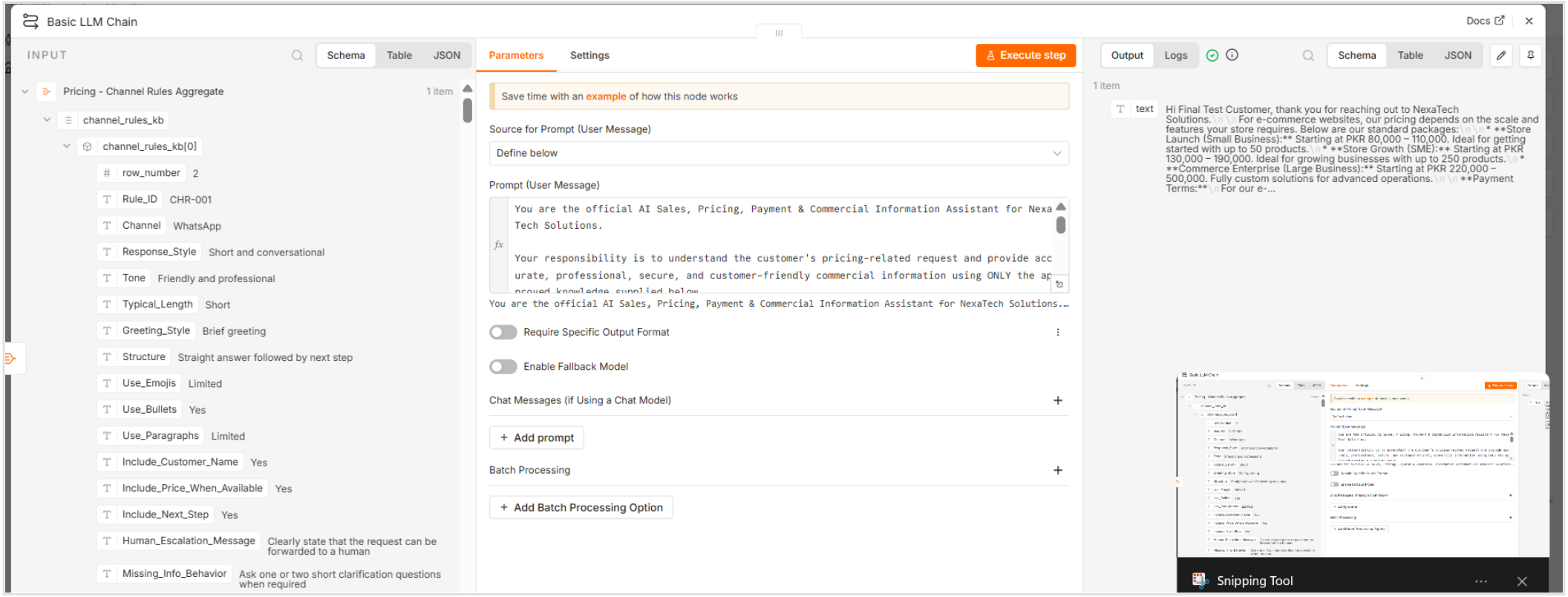
2. Product Inquiry - Final Grounded Response

Product Inquiry test successfully uses approved service, requirement, integration, and channel-rule context to generate a customer-facing answer.



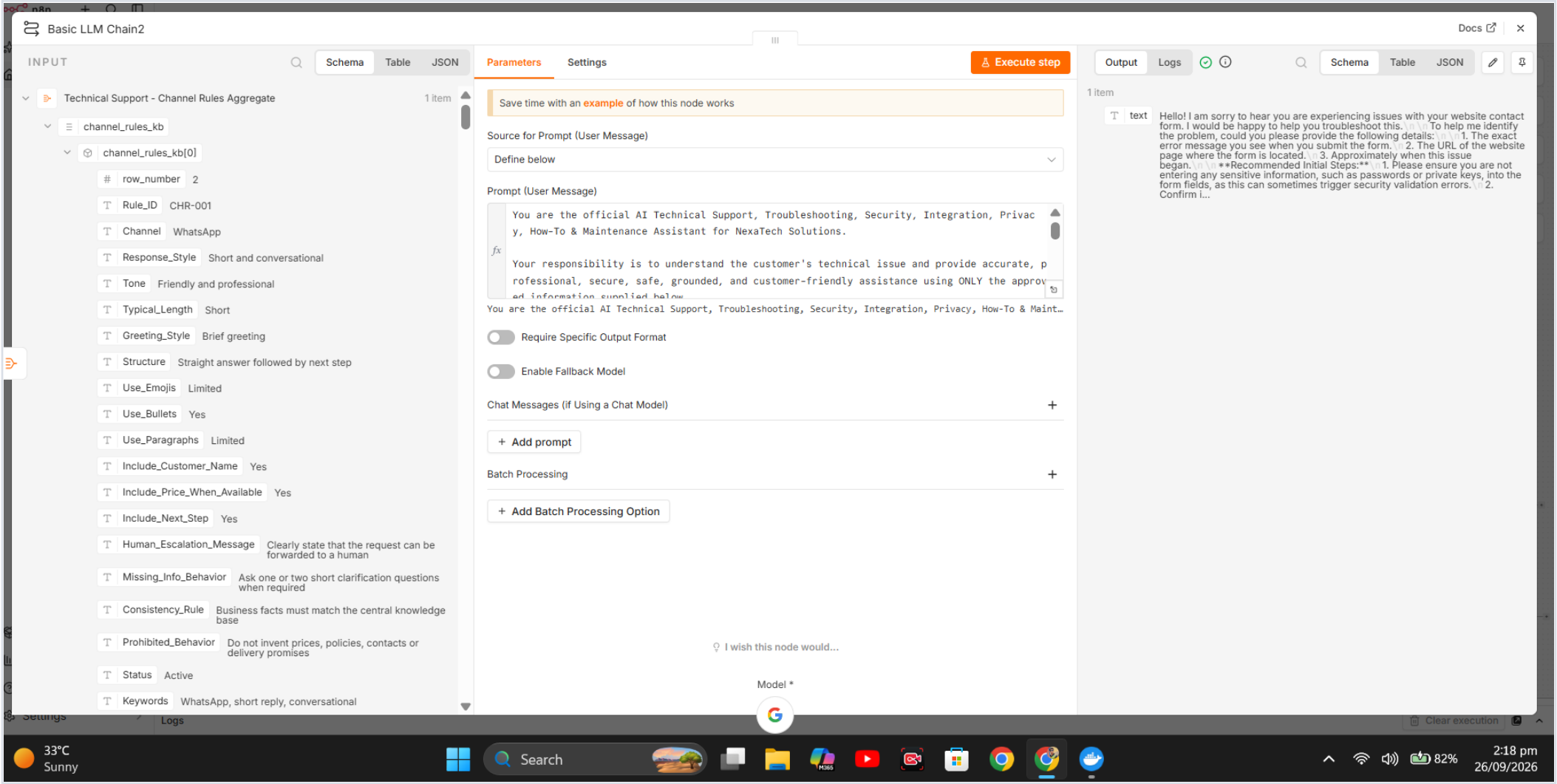
3. Pricing - Final Grounded Response

Pricing branch returns approved package ranges and commercial information while preserving no-hallucination and channel-consistency rules.



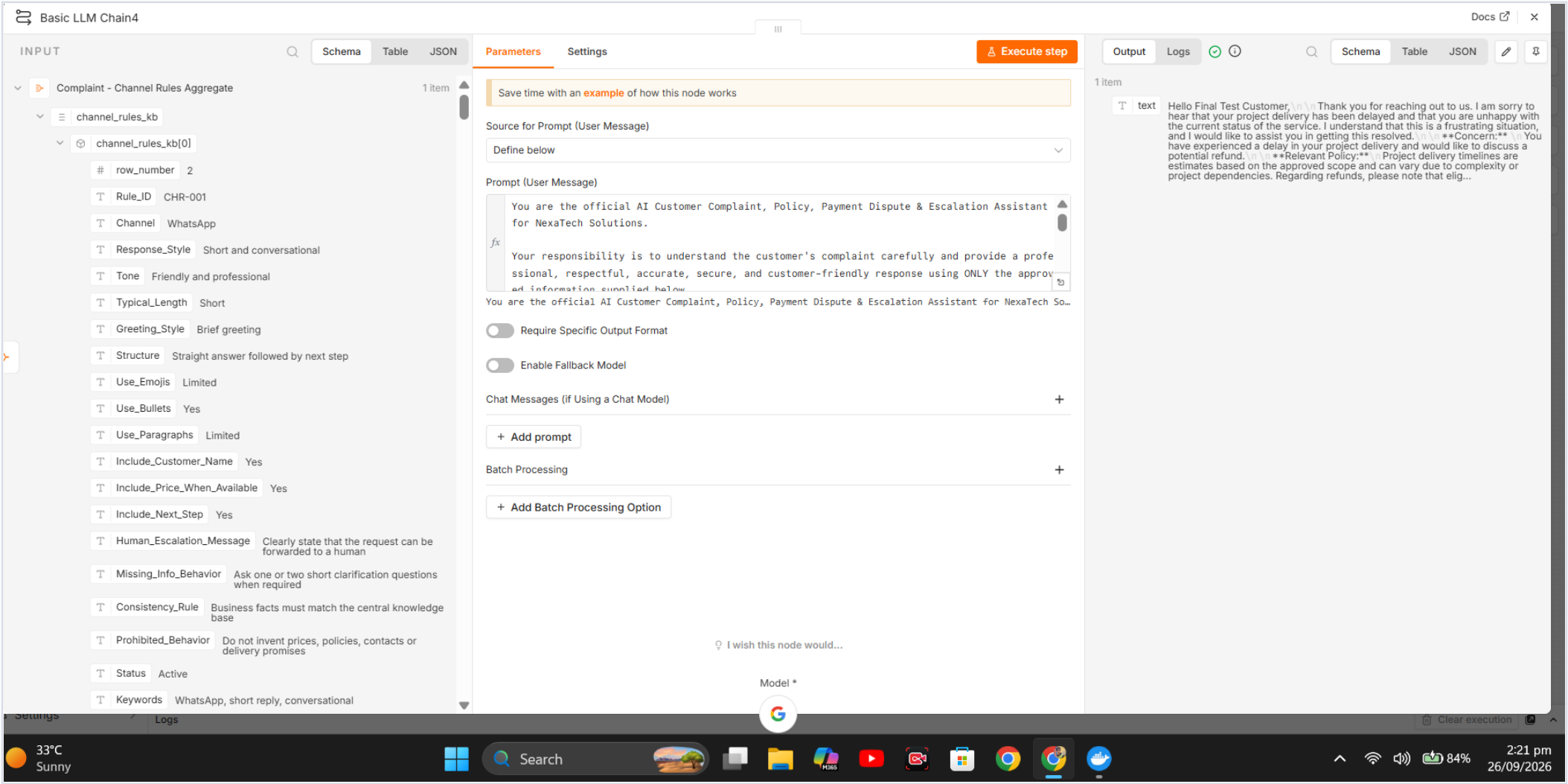
4. Technical Support - Secure Troubleshooting

Technical Support branch asks for safe diagnostic details, avoids inventing a root cause, and protects passwords, keys, and other sensitive credentials.



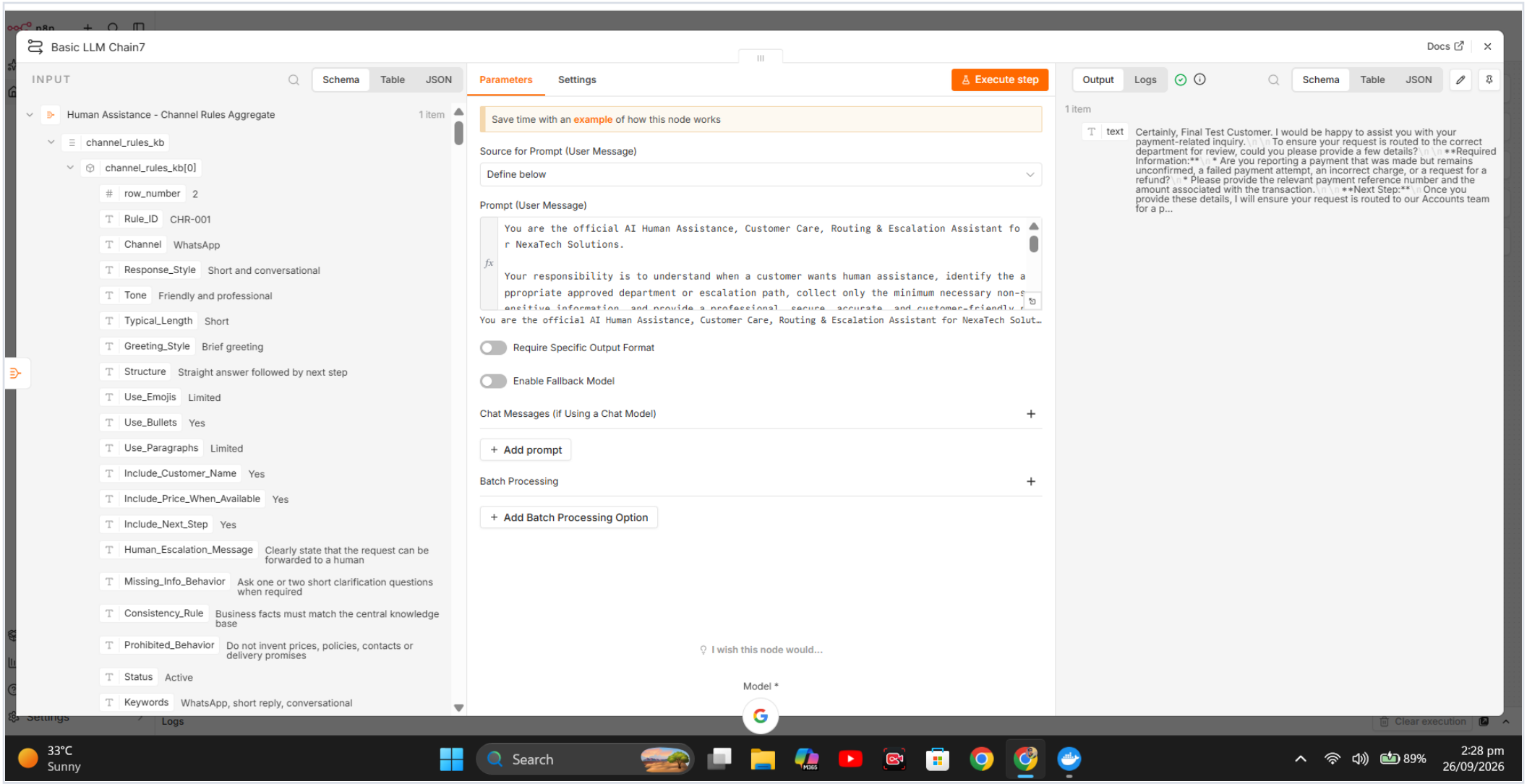
5. Complaint - Policy & Escalation Handling

Complaint branch uses approved policy, payment/refund, escalation, and channel rules without automatically approving refunds or inventing outcomes.



6. Human Assistance - Correct Routing & Escalation

Human Assistance branch identifies the correct department, requests only non-sensitive information, and avoids fake employee/contact or handoff confirmations.



BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 6

Live WhatsApp Integration and Automated Response

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	FUNCTIONAL PROTOTYPE / LIVE CHANNEL TESTED

Real Meta WhatsApp Cloud API channel connected to the existing CRM and Central AI Brain; inbound/outbound flow documented.

1. Phase 6 Implementation Archive - Real WhatsApp Channel and Automated Reply

Before Phase 7, the project moved from simulated WhatsApp input toward a real Meta WhatsApp Cloud API channel. The same CRM and Central AI Brain were reused; the main work was public webhook exposure, Meta verification/subscription, inbound event handling, customer/conversation storage and outbound WhatsApp reply.

Live WhatsApp architecture

```
WhatsApp User -> Meta Cloud API -> Public HTTPS Tunnel -> n8n POST Webhook
-> If actual text message -> Edit Fields -> CRM lookup/update/append
-> Store Conversation -> Central AI Brain -> Limit 1 -> Send WhatsApp Reply
```

Why a public tunnel was required

Meta cannot call localhost directly. The local n8n instance at http://localhost:5678 therefore needed a public HTTPS endpoint. Cloudflare Quick Tunnel was first used for testing; later a fixed ngrok development domain was configured in Docker Compose.

GET verification + POST messages

```
GET /webhook/whatsapp-meta -> Respond to Webhook -> hub.challenge
POST /webhook/whatsapp-meta -> receive real Meta message/status events
```

GET verification response

```
Respond With: Text
Response Body: {{ $json.query["hub.challenge"] }}
Response Code: 200
```

WABA callback override

```
POST <WABA_ID>/subscribed_apps
{
"override_callback_uri": "https://<public-domain>/webhook/whatsapp-meta",
"verify_token": "<your-verify-token>"
}
Expected: { "success": true }
```

Duplicate/status event handling

Meta produces message events and later sent/delivered/read status events. An IF node was added immediately after the POST webhook so only actual text messages continue into the CRM/AI flow.

```
{{ $json.body?.entry?.[0]?.changes?.[0]?.value?.messages?.[0]?.text?.body ?? "" }}
Operator: is not empty
```

Outbound reply fixes

• The Central AI Brain final WhatsApp payload includes to_phone and response.

• A Limit node with Max Items = 1 prevents duplicate sends when an upstream AI branch returns more than one item.

• Send Message recipient points to the inbound sender phone; Text Body uses the final response text.

• Authorization/credential failures were separated from webhook failures during debugging.

Docker Compose deployment

```
services:
n8n: n8nio/n8n:latest
ngrok/cloudflared tunnel service
persistent bind mount -> /home/node/.n8n
TZ / GENERIC_TIMEZONE = Asia/Karachi
```

Major troubleshooting lessons

• If Meta shows events but n8n gets nothing, test the public URL directly before changing workflow logic.

• A fixed ngrok domain can be configured, but the ngrok agent still needs outbound TCP 443 connectivity.

• Docker containers can be Up while ngrok public connectivity is broken.

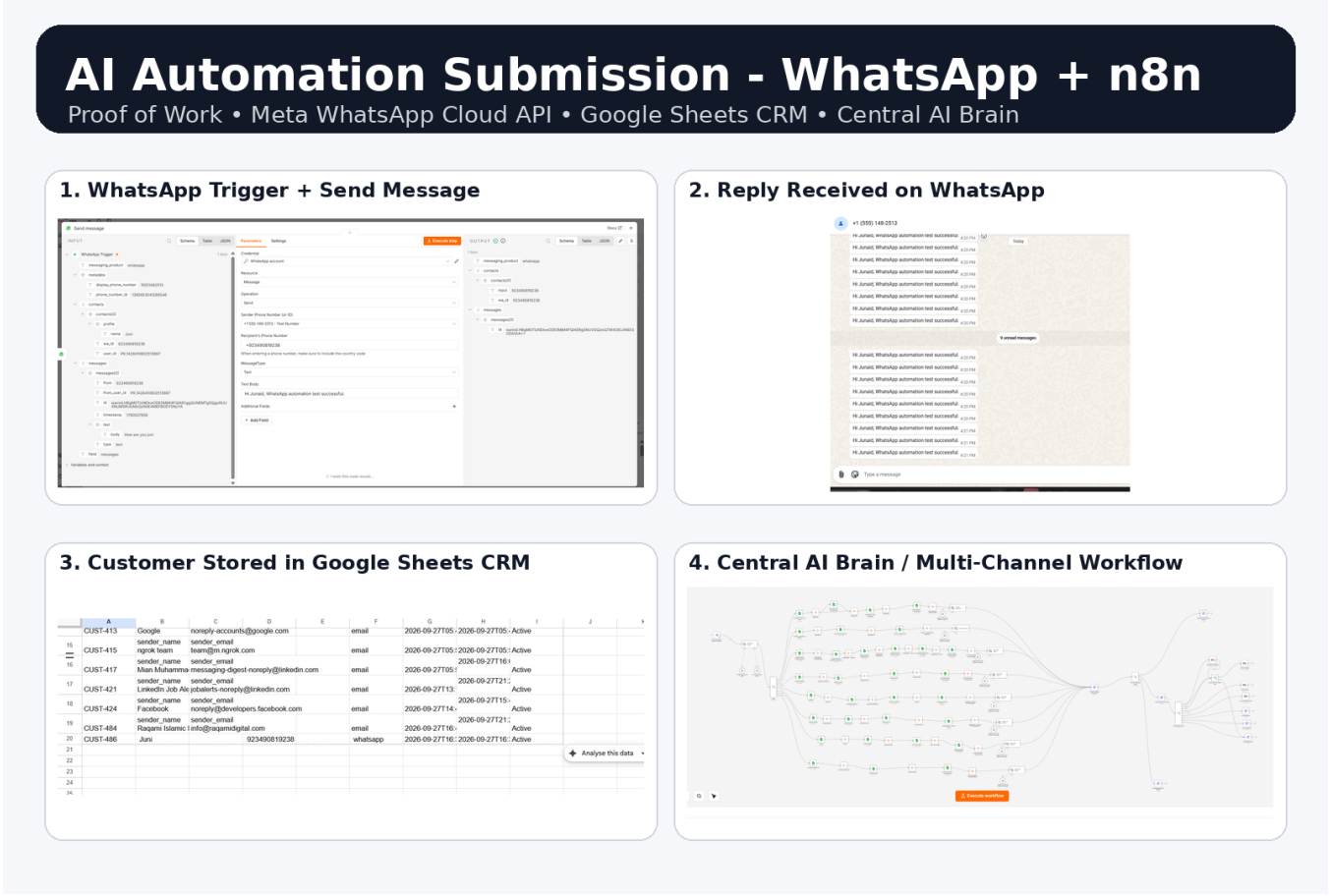
• Use Test-NetConnection connect.ngrok-agent.com -Port 443 to detect a network block; switching to a working hotspot restored the tunnel.

• Direct GET challenge test returning 12345 proves the verification route works.

The original 7-page WhatsApp automation submission and 19-page Meta+n8n connection guide are appended as Phase 6 evidence and implementation detail.

AI Automation Project Submission

WhatsApp Cloud API + n8n + Google Sheets CRM + Central AI Brain



Project Goal: Build a WhatsApp automation workflow that captures incoming customer messages, stores customer and conversation data, passes the message to a centralized AI workflow, and sends an automated reply back through WhatsApp.

Current Status: Functional prototype validated with Meta's WhatsApp test number. Inbound trigger data, customer CRM storage, conversation logging, AI-brain handoff, and outbound WhatsApp replies were demonstrated. Production webhook activation requires a stable public tunnel/endpoint.

1. Solution Overview

The workflow is designed as a reusable multi-channel automation. WhatsApp is one communication channel feeding the same customer and conversation pipeline. The automation extracts structured data from each message, checks whether the customer already exists, updates or creates the CRM record, logs the conversation, calls a centralized AI sub-workflow, and then sends a response.

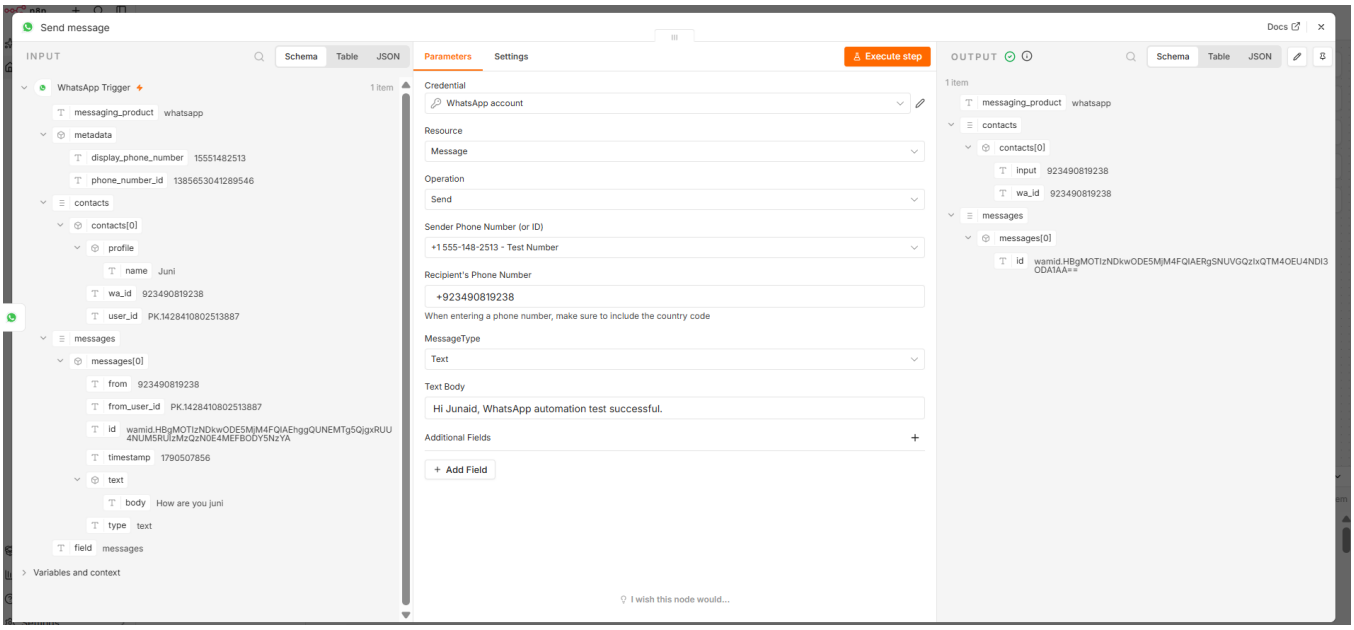
Stage	Purpose
WhatsApp Trigger	Receives inbound WhatsApp message events from Meta Cloud API.
Edit Fields	Normalizes user ID, phone, name, message, channel and timestamp.
Google Sheets Lookup	Searches the Customers sheet using the sender phone number.
IF + Update / Append	Updates an existing customer or creates a new customer record.
Store Conversation	Saves message ID, channel, sender type, message and timestamp.
Central AI Brain	Passes normalized context into the centralized AI workflow.
Send Message	Sends the generated response back to the WhatsApp user.

2. Technologies Used

n8n for workflow orchestration; **Meta WhatsApp Cloud API** for messaging; **Google Sheets** as a lightweight CRM and conversation store; a **Central AI Brain** sub-workflow for shared AI decision logic; and **ngrok** for exposing the local n8n instance during webhook testing.

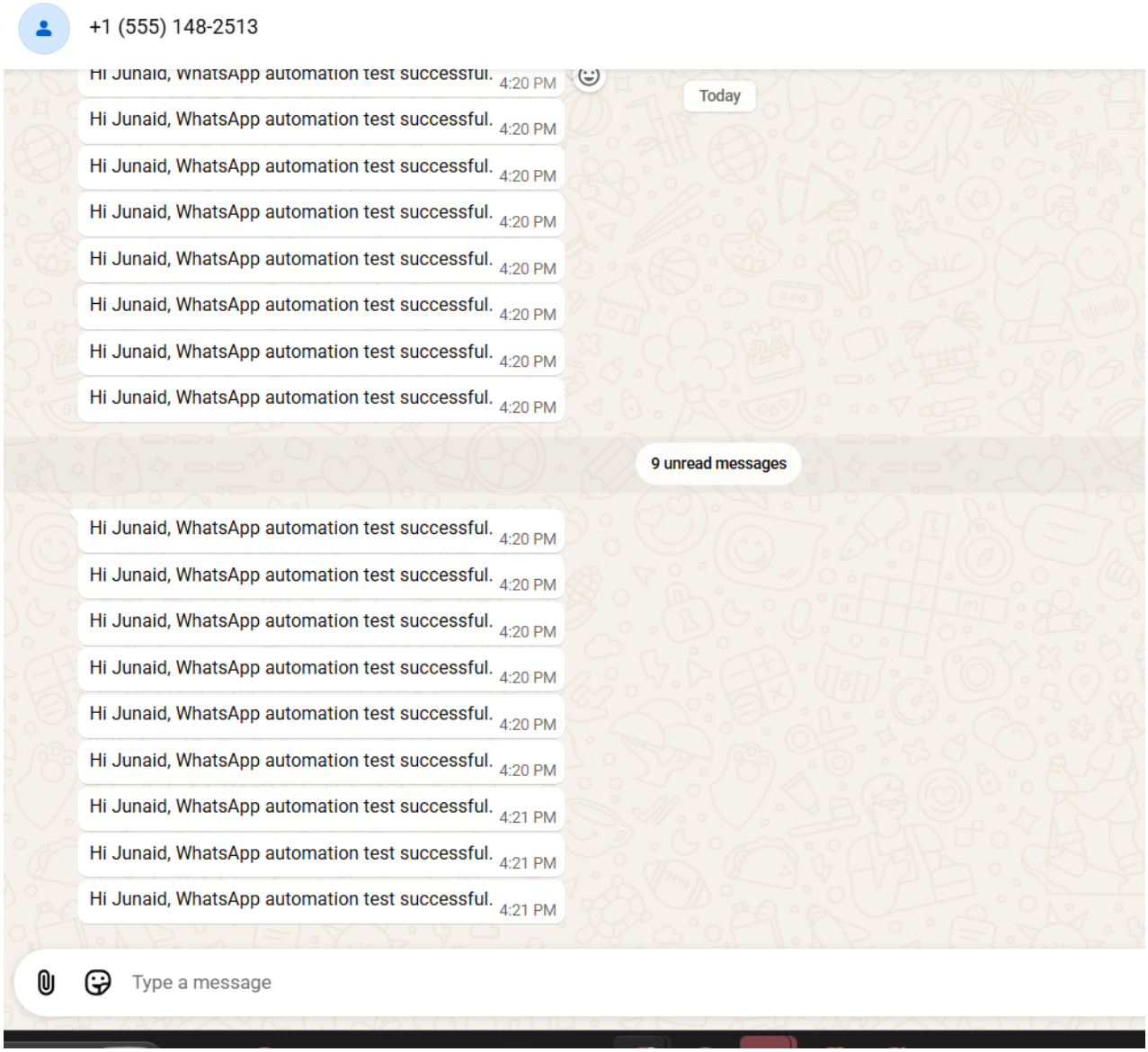
Evidence A - WhatsApp Trigger and Outbound Send

The n8n WhatsApp node received structured WhatsApp data including sender profile, phone number, message ID and text. The Send Message node was also configured to reply to the same recipient.



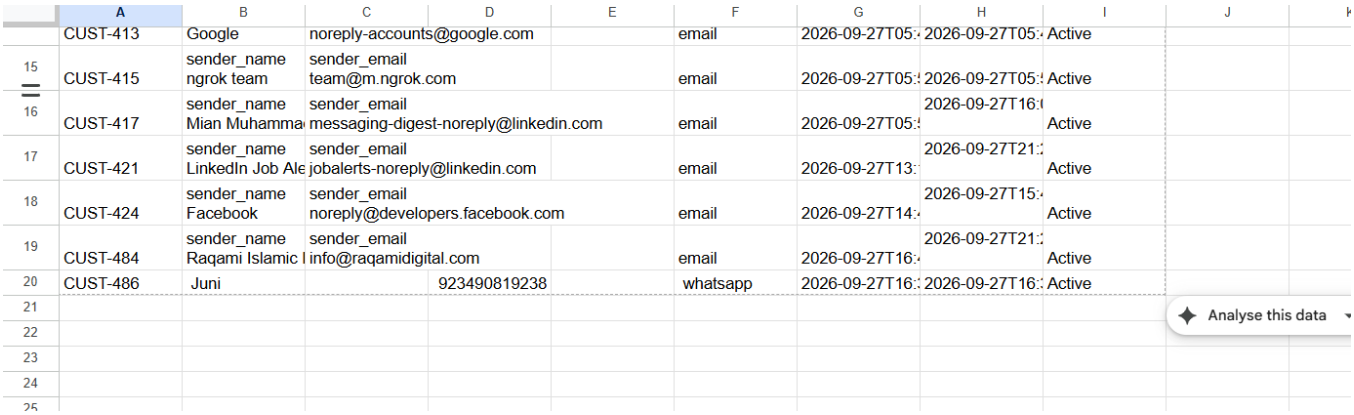
Evidence B - WhatsApp Reply Received

The Meta WhatsApp test number successfully delivered automated messages back to the recipient, confirming outbound messaging from the workflow.



Evidence C - Customer CRM Record

A WhatsApp customer was added to the Customers sheet with a generated customer ID, name, phone number, preferred channel and active status.

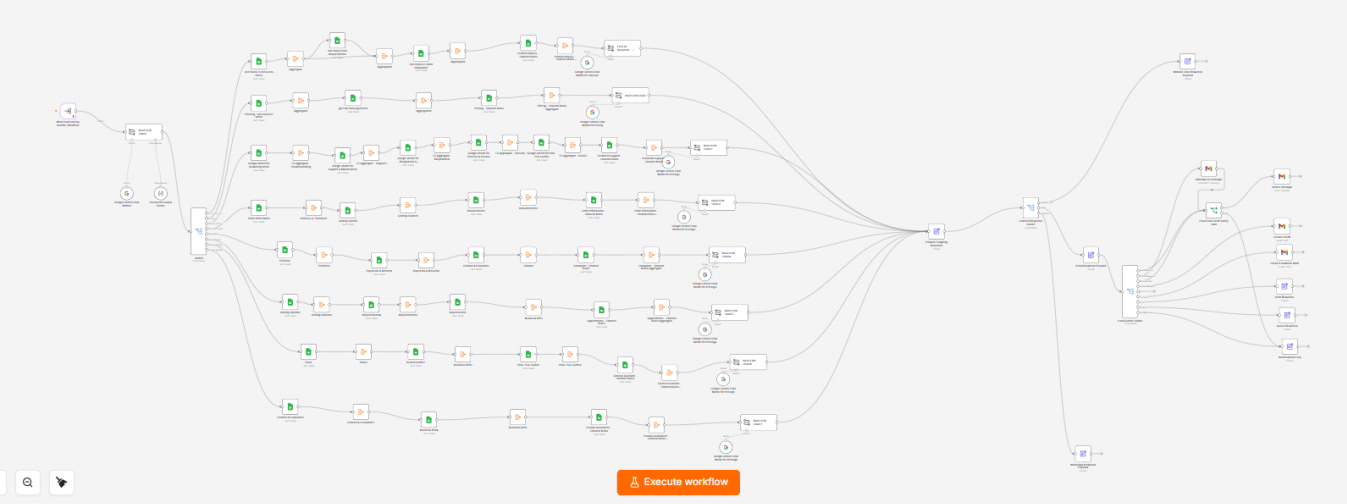


	A	B	C	D	E	F	G	H	I	J
	CUST-413	Google	noreply-accounts@google.com			email	2026-09-27T05:	2026-09-27T05:	Active	
15	CUST-415	sender_name ngrok team	sender_email team@m.ngrok.com			email	2026-09-27T05:	2026-09-27T05:	Active	
16	CUST-417	sender_name Mian Muhamma	sender_email messaging-digest-noreply@linkedin.com			email	2026-09-27T05:	2026-09-27T16:	Active	
17	CUST-421	sender_name LinkedIn Job Ale	sender_email jobalerts-noreply@linkedin.com			email	2026-09-27T13:	2026-09-27T21:	Active	
18	CUST-424	sender_name Facebook	sender_email noreply@developers.facebook.com			email	2026-09-27T14:	2026-09-27T15:	Active	
19	CUST-484	sender_name Raqami Islamic I	sender_email info@raqamidigital.com			email	2026-09-27T16:	2026-09-27T21:	Active	
20	CUST-486	Juni		923490819238		whatsapp	2026-09-27T16:	2026-09-27T16:	Active	
21										
22										
23										
24										

Analyse this data

Evidence D - Central AI Brain Architecture

The centralized AI workflow shows the larger multi-channel routing architecture used to consolidate processing and response logic.



3. Key Learning Outcomes

This task provided practical experience with OAuth/API credentials, webhook-based triggers, WhatsApp Cloud API events, data mapping, conditional routing, Google Sheets CRUD operations, sub-workflows, conversation persistence, and automated customer replies.

4. Submission Summary

The project demonstrates a working WhatsApp automation prototype in n8n. It receives messages, identifies or creates the customer, stores the conversation, forwards the context to a centralized AI workflow, and can send a response back through WhatsApp. The same architecture is designed to support additional channels through the shared Central AI Brain.

Production note: The workflow was validated in test mode. A stable externally reachable webhook endpoint is required for continuous production operation.

WhatsApp + Meta + n8n Full Connection & Troubleshooting Guide

Free development setup using Cloudflare Quick Tunnel and n8n Webhooks

WhatsApp User	Meta Cloud API	Cloudflare Tunnel	n8n Webhook	AI / CRM / Reply

Current verified milestone: real phone message -> Meta -> Cloudflare -> n8n POST Webhook triggers automatically.

Privacy note
Access tokens are secrets. Any visible access token in screenshots used in this guide has been redacted. If a token was ever exposed in a screenshot or shared message, regenerate it before using the setup for real work.

Prepared from the actual setup, commands, errors, fixes, and screenshots used during configuration.

1. What this guide covers

- What must be installed and how to check it
- How to run n8n locally with Docker
- Why localhost cannot receive Meta webhooks directly
- How to create a free public HTTPS URL with Cloudflare Quick Tunnel
- How to build GET verification and POST incoming-message webhooks in n8n
- How to verify the callback in Meta
- How to subscribe the WhatsApp Business Account (WABA)
- How to override the WABA callback so Meta sends events to the correct n8n URL
- How to test with curl and then with a real WhatsApp message
- How to map incoming Meta JSON into Edit Fields
- What failed with ngrok, why it failed, and how Cloudflare solved it
- Troubleshooting checklist and commands

Important distinction
GET and POST can use the same path (for example, /webhook/whatsapp-meta) because they are different HTTP methods. GET is used by Meta for verification; POST is used for actual WhatsApp events.

2. Final architecture

```
WhatsApp User
   |
   v
Meta WhatsApp Cloud API
   |  (HTTPS webhook event)
   v
Cloudflare Quick Tunnel URL
   |
   v
localhost:5678 (n8n)
   |
   +--> GET /webhook/whatsapp-meta  -> Respond with hub.challenge (verification)
   |
   +--> POST /webhook/whatsapp-meta -> Edit Fields -> Sheets/CRM -> AI Brain -> Reply
```

3. Prerequisites and installation

3.1 Docker Desktop (if Docker is not installed)

1. Install Docker Desktop for Windows. Use the normal Windows installer and the WSL 2 backend if available.
2. Start Docker Desktop and wait until Docker Engine is running.
3. Verify from PowerShell or CMD:

```
docker --version
docker ps
```

If docker is not recognized
Close and reopen the terminal after installation. Also make sure Docker Desktop is running. Official Windows install guidance: https://docs.docker.com/desktop/setup/install/windows-install/

3.2 n8n (if n8n is not already running)

If you already have an n8n Docker container
Do not recreate or delete it. First use docker ps -a to find the exact container name. Recreating a container without checking volumes can risk your local data.

```
docker ps -a
```

4. If there is no n8n container at all, a basic persistent Docker setup can be created with:

```
docker volume create n8n_data

docker run -d --name n8n -p 5678:5678 ^
  -e TZ=Asia/Karachi ^
  -e GENERIC_TIMEZONE=Asia/Karachi ^
  -v n8n_data:/home/node/.n8n ^
  n8nio/n8n:latest
```

5. Open n8n locally in the browser:

```
http://localhost:5678
```

6. Useful checks used in this setup:

```
docker ps -a
docker start <EXACT_CONTAINER_NAME>
docker exec <EXACT_CONTAINER_NAME> n8n --version
docker inspect <EXACT_CONTAINER_NAME> --format "{{range .Config.Env}}{{println .}}{{end}}"
```

3.3 cloudflared (if not installed)

7. Check whether cloudflared is already installed:

```
cloudflared --version
```

8. If it is not installed, download the latest Windows 64-bit MSI or executable from Cloudflare Tunnel Downloads, install it, and reopen the terminal.

Official download
https://developers.cloudflare.com/tunnel/downloads/ - Cloudflare notes that Windows installations do

not automatically update, so update the executable manually when needed.

4. Create the free public HTTPS tunnel

9. Make sure n8n is already reachable locally at http://localhost:5678.
10. Start a free Cloudflare Quick Tunnel:

```
cloudflared tunnel --url http://localhost:5678
```

11. Copy the generated URL. It will look similar to:

```
https://random-words.trycloudflare.com
```

12. Keep the terminal window open while testing. If the process stops, the public URL stops working.

Quick Tunnel behavior
Quick Tunnels are free for development/testing and generate a random trycloudflare.com subdomain. The URL can change after a restart. If it changes, update Meta callback settings and the WABA override callback. Cloudflare documents a 200 concurrent-request limit for Quick Tunnels.

5. n8n GET Webhook for Meta verification

13. Create a Webhook node.

- HTTP Method: GET
- Path: whatsapp-meta
- Respond: Using 'Respond to Webhook' Node

14. Add a Respond to Webhook node and connect:

```
Webhook (GET)  ->  Respond to Webhook
```

15. Configure Respond to Webhook:

- Respond With: Text
- Response Code: 200
- Response Body:

```
{{ $json.query["hub.challenge"] }}
```

16. Publish the workflow.

6. Meta callback verification

17. Open Meta Developers -> your WhatsApp app -> Production setup -> Configure Webhooks.
18. Set Callback URL to the Cloudflare production webhook URL:

```
https://<YOUR-TRYCLOUDFLARE-DOMAIN>/webhook/whatsapp-meta
```

19. Set a verify token that you choose, for example:

```
YOUR_VERIFY_TOKEN
```

20. Click Verify and Save.

Why this works
Meta makes a GET request containing hub.challenge. n8n returns the challenge text with HTTP 200, so Meta verifies that the endpoint belongs to you.

7. n8n POST Webhook for real incoming messages

21. Create a second Webhook node in the same workflow.

- HTTP Method: POST
- Path: whatsapp-meta (same path as GET)
- Authentication: None (for Meta webhook receiver)
- Respond: Immediately
- Response Code: 200

22. Publish the workflow again after adding/changing the POST node.

Do not use the Test URL in Meta
For automatic events, Meta must call the production URL containing /webhook/, not the n8n /webhook-test/ URL. Listen for test event is not required once the workflow is published.

8. Confirm WABA subscription in Graph API Explorer

23. Open Meta Graph API Explorer and select the same Meta app used for WhatsApp.
24. Use a token with these permissions:

- whatsapp_business_management
- whatsapp_business_messaging

25. Run this GET request, replacing WABA_ID:

```
GET  <WABA_ID>/subscribed_apps
```

26. Confirm your app appears in the data array.

9. Force the WABA to use the Cloudflare callback

This was the key fix in this setup
curl could reach n8n through Cloudflare, and Meta could see the phone message, but n8n still did not receive Meta POST events. The WABA callback was then explicitly overridden to the Cloudflare production webhook URL. After this, real phone messages triggered n8n automatically.

27. In Graph API Explorer, use POST on the same endpoint:

```
POST  <WABA_ID>/subscribed_apps
```

28. JSON body:

```
{
  "override_callback_uri": "https://<YOUR-TRYCLOUDFLARE-DOMAIN>/webhook/whatsapp-meta",
  "verify_token": "<YOUR_VERIFY_TOKEN>"
}
```

29. Submit. A successful call returns:

```
{
  "success": true
}
```

Reference
Meta's WhatsApp Business Platform collection documents override_callback_uri and verify_token on POST /<WABA-ID>/subscribed_apps for sending webhook notifications to an alternate callback URL.

10. Test in the correct order

10.1 First test Cloudflare -> n8n with curl

```
curl.exe -i -X POST "https://<YOUR-TRYCLOUDFLARE-DOMAIN>/webhook/whatsapp-meta" ^
  -H "Content-Type: application/json" ^
  -d "{\"test\":\"hello\"}"
```

- If a new n8n execution appears, Cloudflare and the POST webhook are working.
- If this works but real Meta messages do not, check WABA subscription and override_callback_uri.

10.2 Confirm Meta receives the phone message

30. Send a message from your real WhatsApp phone to the Meta test number.
31. In Meta -> Try it out -> Check test webhooks, confirm a new row appears with webhook field = messages.
32. Expand the payload and confirm it contains value.messages[0].text.body.

10.3 Final automatic test

33. Leave the n8n workflow Published.
34. Leave the Cloudflare tunnel terminal running.
35. Do not press Execute Workflow or Listen for test event.
36. Send a fresh WhatsApp message from the phone.
37. A new n8n execution should appear automatically, triggered by POST Webhook.

11. Map Meta webhook data into Edit Fields

38. Connect the POST Webhook to Edit Fields:

```
Webhook (POST) -> Edit Fields
```

39. Recommended mappings for normal text messages:

Field	Value / Expression
user_id	{{ $json.body.entry[0].changes[0].value.messages[0].from }}
message	{{ $json.body.entry[0].changes[0].value.messages[0].text.body }}
sender_phone	{{ $json.body.entry[0].changes[0].value.messages[0].from }}
name	{{ $json.body.entry[0].changes[0].value.contacts[0].profile.name }}
channel	whatsapp
timestamp	{{ $now.toISO() }}

12. Recommended message-event filter

Meta also sends status updates. Before CRM/AI processing, add an IF node so only events containing a real incoming message continue.

```
{{ !!$json.body?.entry?.[0]?.changes?.[0]?.value?.messages?.[0] }}
```

True branch -> Edit Fields / CRM / AI. False branch -> stop or ignore status-only notifications.

13. Continue into the existing CRM + AI workflow

```
POST Webhook
  -> IF (is real message?)
  -> Edit Fields
  -> Get row(s) in Customers sheet
  -> IF existing customer
     True  -> Update row
     False -> Append row
  -> Store Conversation
  -> Call Central AI Brain
  -> Send Message / HTTP Request to Meta Graph API
```

Current project status

Incoming webhook delivery is confirmed working automatically. The next build step is to finish data mapping and reconnect the POST webhook to the existing Google Sheets, Central AI Brain, and outbound reply flow.

14. Troubleshooting - errors we actually saw

Symptom	Cause / diagnosis	Fix
n8n command not recognized	n8n was running in Docker, not installed globally with npm.	Use docker ps -a and work with the existing container instead of installing another n8n copy.
No such container: n8n	The real container name was different.	Run docker ps -a and copy the exact NAMES value.
Native WhatsApp Trigger callback 404	Meta callback verification could not reach/register the native trigger correctly in the local/tunnel setup.	Switched to generic GET + POST n8n Webhooks, which are easier to control and verify.
ngrok reconnecting / i/o timeout on :443	DNS and internet worked, but ngrok agent could not establish its cloud session.	Used Cloudflare Quick Tunnel instead.
Meta sees message but n8n receives nothing	Meta incoming test webhook showed messages, and curl to Cloudflare triggered n8n, so the missing link was WABA callback routing.	POST /<WABA_ID>/subscribed_apps with override_callback_uri and verify_token.
curl triggers n8n, phone does not	Public endpoint is healthy, but Meta is not sending to that exact callback.	Check messages subscription, same Meta app, WABA subscribed_apps, and callback override.
Quick Tunnel URL changes	Quick tunnels use random temporary domains.	Update Meta Callback URL and override_callback_uri after every URL change.

15. Diagnostic commands cheat sheet

```
# Docker
docker --version
docker ps -a
docker exec <CONTAINER_NAME> n8n --version

# Internet / DNS
ping google.com
nslookup connect.ngrok-agent.com

# ngrok diagnosis (if you still use ngrok)
ngrok config check
ngrok http 5678 --log=stdout --log-level=debug

# Cloudflare
cloudflared --version
cloudflared tunnel --url http://localhost:5678

# Public POST test
curl.exe -i -X POST "https://<PUBLIC-DOMAIN>/webhook/whatsapp-meta" -H "Content-Type: application/json" -d "{\"test\":\"hello\"}"
```

16. Security and privacy checklist

- Never share an access token or App Secret in screenshots, PDFs, GitHub, or messages.
- If an access token was exposed, regenerate/revoke it and update n8n credentials.
- Use n8n Credentials rather than hard-coding tokens in Code or HTTP Request nodes.
- Quick Tunnel is suitable for development/testing, not a permanent production endpoint.
- For production, use a named Cloudflare Tunnel or hosted n8n with a stable HTTPS domain.
- Do not delete a working Docker container until you confirm its volume/data backup.

17. Free vs production

Component	Development / testing	Production note
n8n self-hosted	Can be run locally with Docker	You manage uptime, backups, security and updates.
Cloudflare Quick Tunnel	Free and fast for testing	Temporary random URL; use a named tunnel/domain for stable production.
Meta test number	Useful for development	Real business messaging is subject to Meta platform rules and pricing.
Google Sheets	Fine for prototype CRM	Larger systems may move to a database.
AI model API	Depends on provider/credits	Usage may have cost depending on the selected model/provider.

18. Verified official references

Cloudflare Quick Tunnels:
https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/do-more-with-tunnels/trycloudflare/

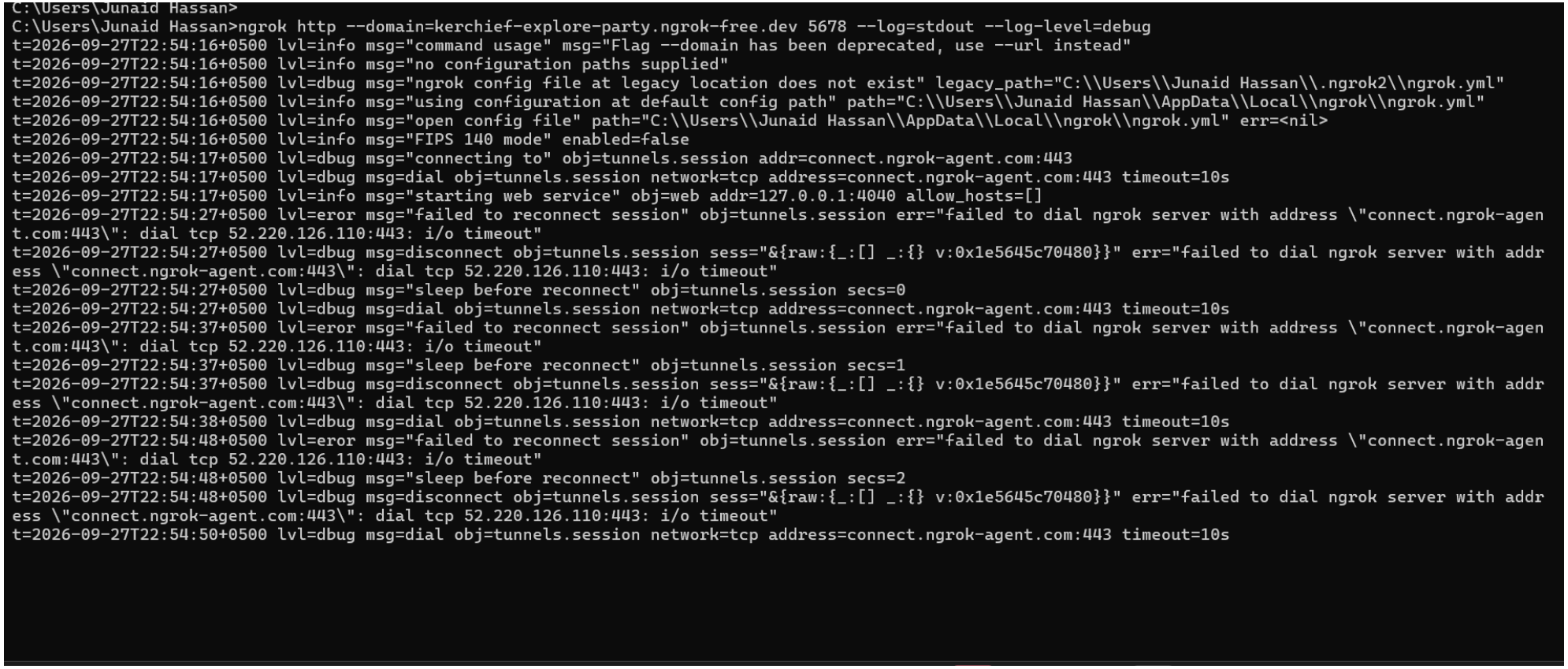
Cloudflare Tunnel downloads: https://developers.cloudflare.com/tunnel/downloads/

Docker Desktop Windows install: https://docs.docker.com/desktop/setup/install/windows-install/

Meta WhatsApp Business Platform - Override Callback URL (official Meta collection on Postman): https://www.postman.com/meta/whatsapp-business-platform/request/un84tul/override-callback-url

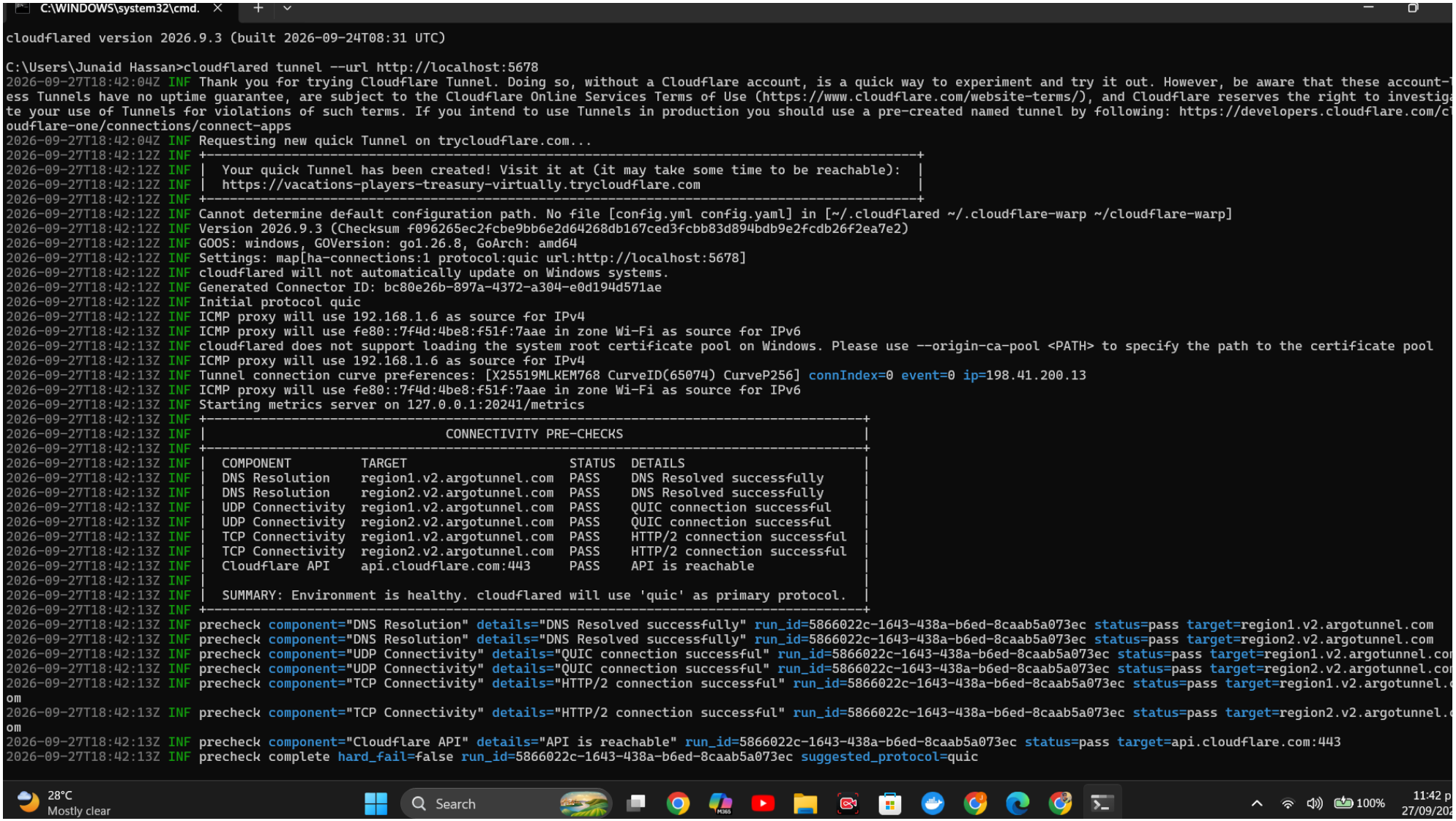
Appendix A - Evidence screenshots from this setup

Note
Screenshots are included as proof and troubleshooting reference. Any access token visible in the original screenshot has been redacted.



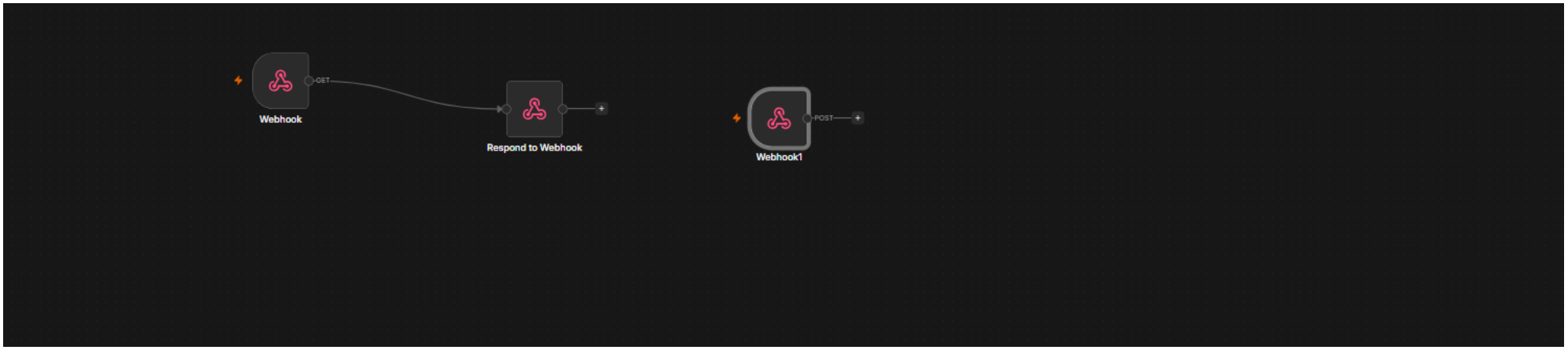
A1. ngrok failure: connection to connect.ngrok-agent.com:443 timed out even though DNS and normal internet were working.

What this proves: The original public-tunnel problem was outside Meta and n8n; ngrok could not establish its cloud session.



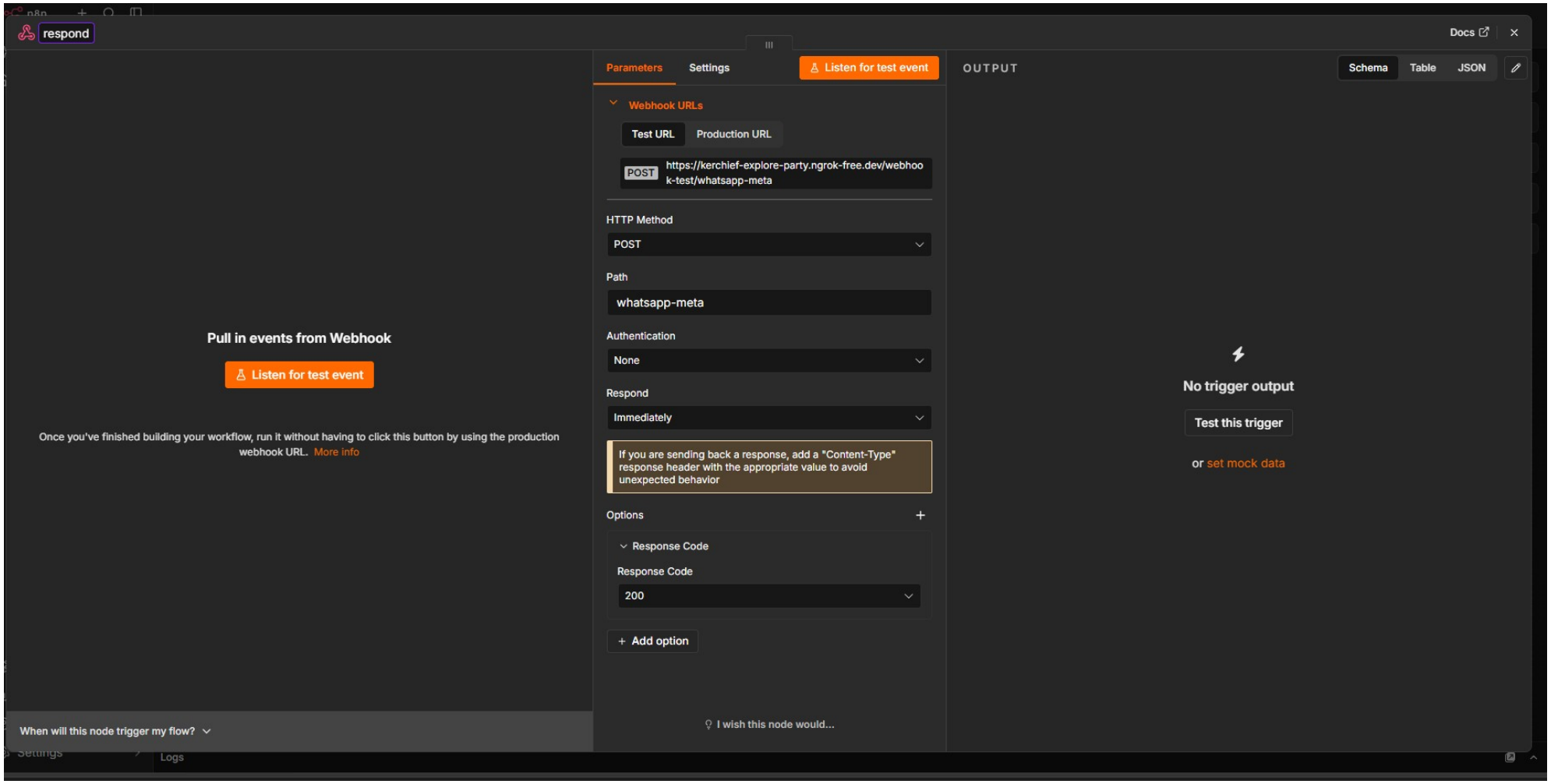
A2. Cloudflare Quick Tunnel created successfully; connectivity pre-checks passed and a trycloudflare.com URL was generated.

What this proves: Cloudflare solved public reachability without requiring an account for the Quick Tunnel test.



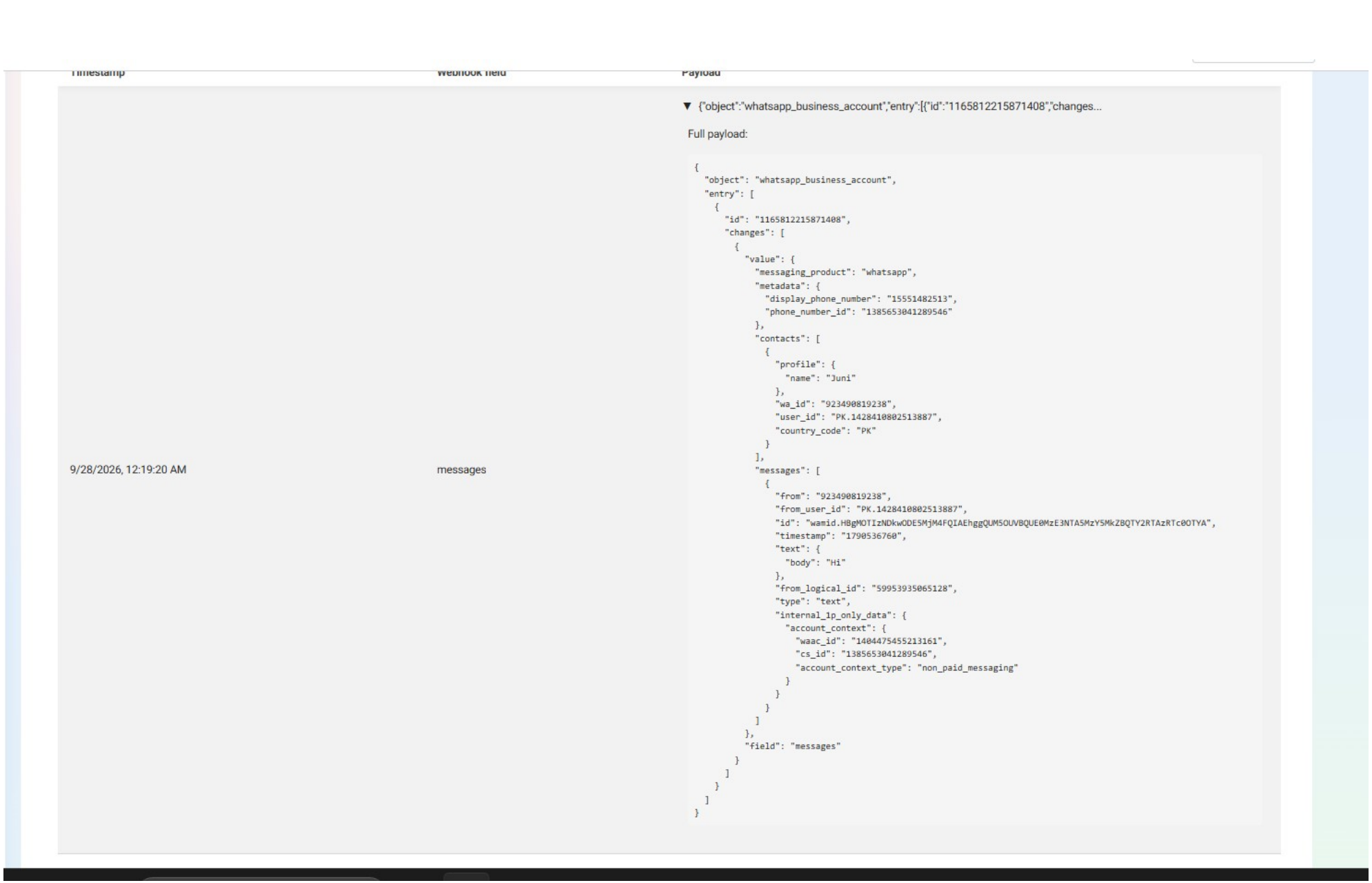
A3. n8n verification architecture: GET Webhook + Respond to Webhook, plus a separate POST Webhook for incoming events.

What this proves: GET and POST webhook responsibilities are separated while sharing the same logical callback path.



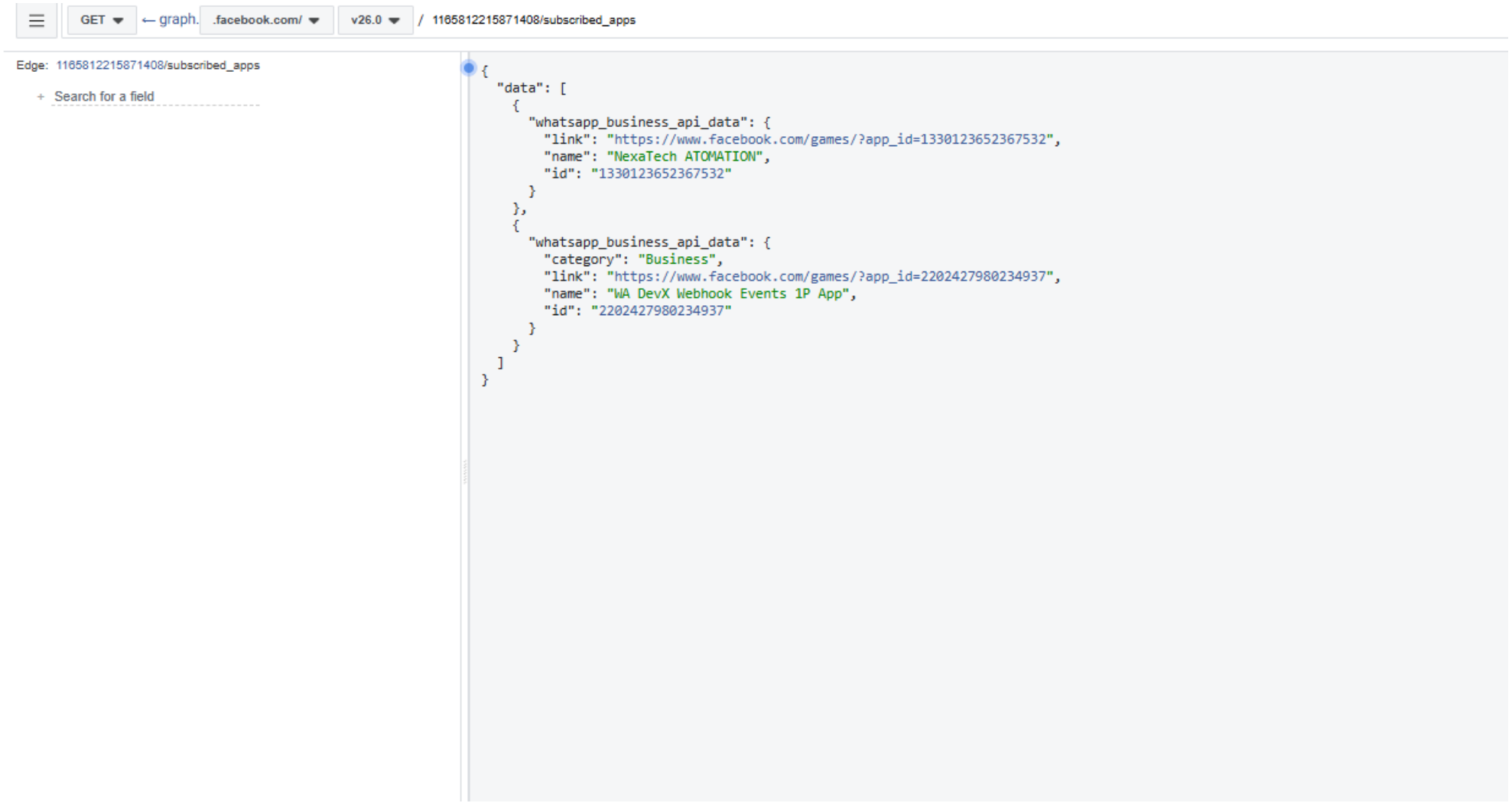
A4. POST Webhook configuration: POST, path whatsapp-meta, Respond Immediately, HTTP 200.

What this proves: The production POST endpoint is ready to acknowledge Meta quickly with HTTP 200.



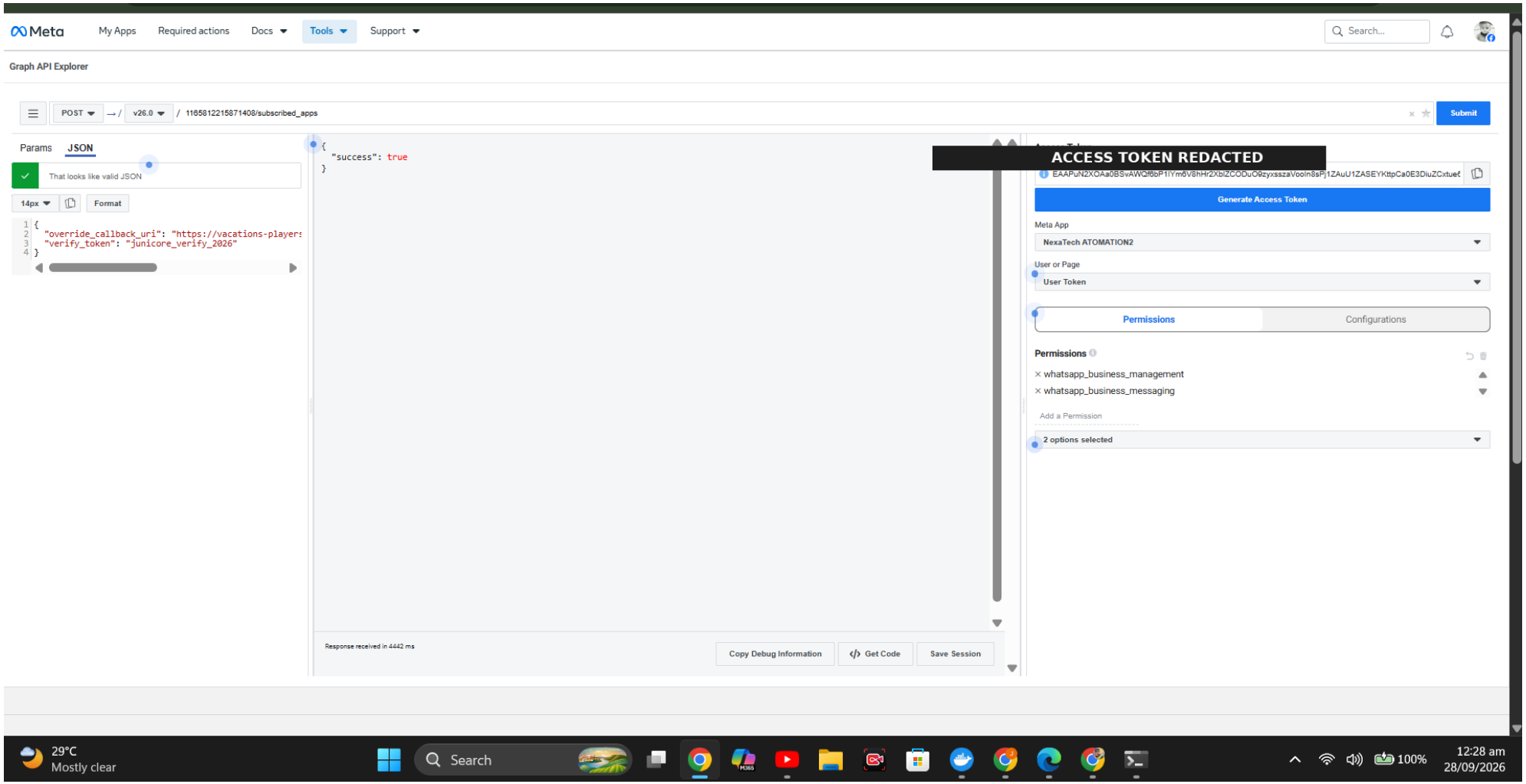
A5. Meta Check test webhooks shows a real incoming phone message with field=messages and message text in the payload.

What this proves: WhatsApp -> Meta is healthy; the real phone message is present in Meta's incoming webhook payload.



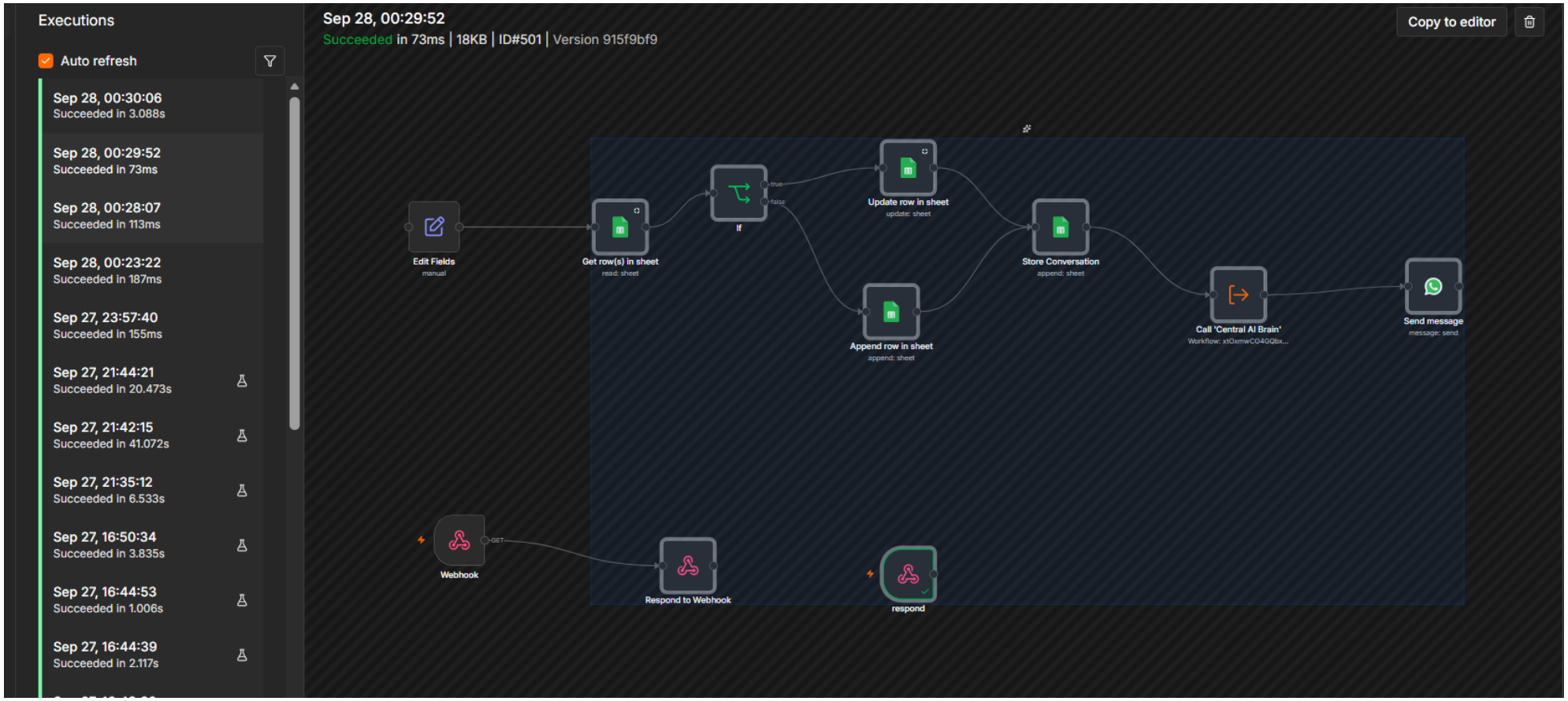
A6. Graph API Explorer GET <WABA_ID>/subscribed_apps confirms WhatsApp apps are subscribed to the WABA.

What this proves: The WABA knows which apps are subscribed; this is different from app-level callback verification.



A7. Graph API Explorer POST <WABA_ID>/subscribed_apps with override_callback_uri returns success=true. Access token redacted.

What this proves: The per-WABA callback was explicitly pointed at the current Cloudflare n8n production endpoint.



A8. Final proof: new n8n executions appear automatically after the WABA callback is overridden to the Cloudflare production webhook.

What this proves: The end-to-end incoming path now triggers n8n automatically, without curl or manual Execute Workflow.

19. One-page rebuild checklist

- ☐ Docker Desktop running
- ☐ n8n opens at http://localhost:5678
- ☐ cloudflared installed
- ☐ Run: cloudflared tunnel --url http://localhost:5678
- ☐ Copy current https://*.trycloudflare.com URL
- ☐ GET Webhook: GET + whatsapp-meta + Respond to Webhook
- ☐ Respond body: {{ $json.query["hub.challenge"] }} and HTTP 200
- ☐ Publish workflow
- ☐ Meta Callback URL = <Cloudflare URL>/webhook/whatsapp-meta
- ☐ Meta Verify Token = your chosen token
- ☐ Verify and Save succeeds
- ☐ POST Webhook: POST + whatsapp-meta + Respond Immediately + 200
- ☐ Publish workflow again
- ☐ Graph API GET <WABA_ID>/subscribed_apps confirms app subscription
- ☐ Graph API POST <WABA_ID>/subscribed_apps with override_callback_uri + verify_token returns success=true
- ☐ Send real WhatsApp message to Meta test number
- ☐ Meta Check test webhooks shows field=messages
- ☐ n8n Executions shows a new automatic POST execution
- ☐ Connect POST Webhook -> IF -> Edit Fields -> CRM -> AI -> Reply

The most important lesson
If Meta sees the incoming message and curl reaches the n8n POST webhook, but n8n still does not receive Meta events, check the WABA subscription callback. In this setup, explicitly setting override_callback_uri to the Cloudflare production webhook was the step that made real automatic triggering work.

BIG BRAINS INTERNSHIP

Multi-channel AI Agent

PHASE 7

Cross-channel Conversation Context

Project	Multi-channel AI Agent
Fictional Business	NexaTech Solutions
Core Platform	n8n + Docker + Google Sheets + AI / APIs
Channels	Website Chat, Email, WhatsApp
Document Type	Complete detailed archive + implementation notes + screenshots
Status	IMPLEMENTED; PROVIDER AVAILABILITY ISSUE ISOLATED

Retrieve prior CRM conversation history and pass previous_context into the AI so customers can move across Website, Email and WhatsApp.

1. Phase 7 Objective - Cross-channel Conversation Context

Phase 7 makes the AI continue a customer conversation even when the customer changes channel. The same customer profile and Conversations sheet are used to retrieve prior messages and pass them to the Central AI Brain as previous_context.

Phase 7 website workflow extension

```
Website Webhook -> Edit Fields -> Customer lookup -> IF -> Update/Append
-> Get Previous Conversations -> Build Previous Context -> Store Conversation
-> Central AI Brain -> Respond to Webhook
```

Previous conversation retrieval

Get Previous Conversations reads the Conversations sheet using Customer ID. Always Output Data is enabled so first-time customers with no history do not stop the workflow.

Build Previous Context code

```
const rows = $input.all().map(item => item.json).filter(row => row && row["Message"]);
const history = rows.slice(-10);
let previousContext = "No previous context available";
if (history.length > 0) {
previousContext = history.map((row, index) => {
const channel = row["Channel"] || "unknown";
const message = row["Message"] || "";
const response = row["Response"] || "";
return `${index + 1}. [${channel}] Customer: ${message}` + (response ? `\nAI: ${response}` : "");
}).join("\n\n");
}
return [{json:{previous_context: previousContext, previous_conversation_count: history.length}}];
```

AI context handoff

```
previous_context = {{ $("Build Previous Context").first().json.previous_context }}
```

Basic LLM Chain and final-response prompts were updated to include previous conversation context and explicitly instruct the AI not to ask the customer to repeat information already provided.

Live Website Chat

• NexaTech business website created with a modern AI chat UI.

• Website deployed publicly on Netlify.

• Live site used a public n8n website-chat webhook.

• Respond to Webhook returned the AI response back to the browser.

Provider availability issue

During the latest end-to-end test, Gemini returned 503 high-demand/service-unavailable responses. This was isolated as a provider-side availability issue, not a CRM/context-logic failure. Alternative/fallback providers were explored without rebuilding the Phase 7 workflow.

WhatsApp continuity and tunnel recovery

• Meta callback verification was reactivated on the fixed ngrok domain.

• A 404 callback error was traced to ngrok connectivity, not the n8n GET/POST configuration.

• TCP 443 to connect.ngrok-agent.com failed on one Wi-Fi network and succeeded after switching to a working hotspot.

• After ngrok restart, the public URL returned HTTP 200, GET challenge returned 12345, and Meta callback POST returned success:true.

Phase 7 final target

```
Website conversation -> same Customer ID -> Email follow-up -> previous_context
-> WhatsApp follow-up -> same centralized profile/history -> AI continues without repetition
```

The original Day 20 submission evidence and the full ngrok troubleshooting/recovery guide are appended. Together they document the Phase 7 context architecture, live website channel, AI handoff and the real WhatsApp connectivity recovery.

Day 20 - Project Phase 7

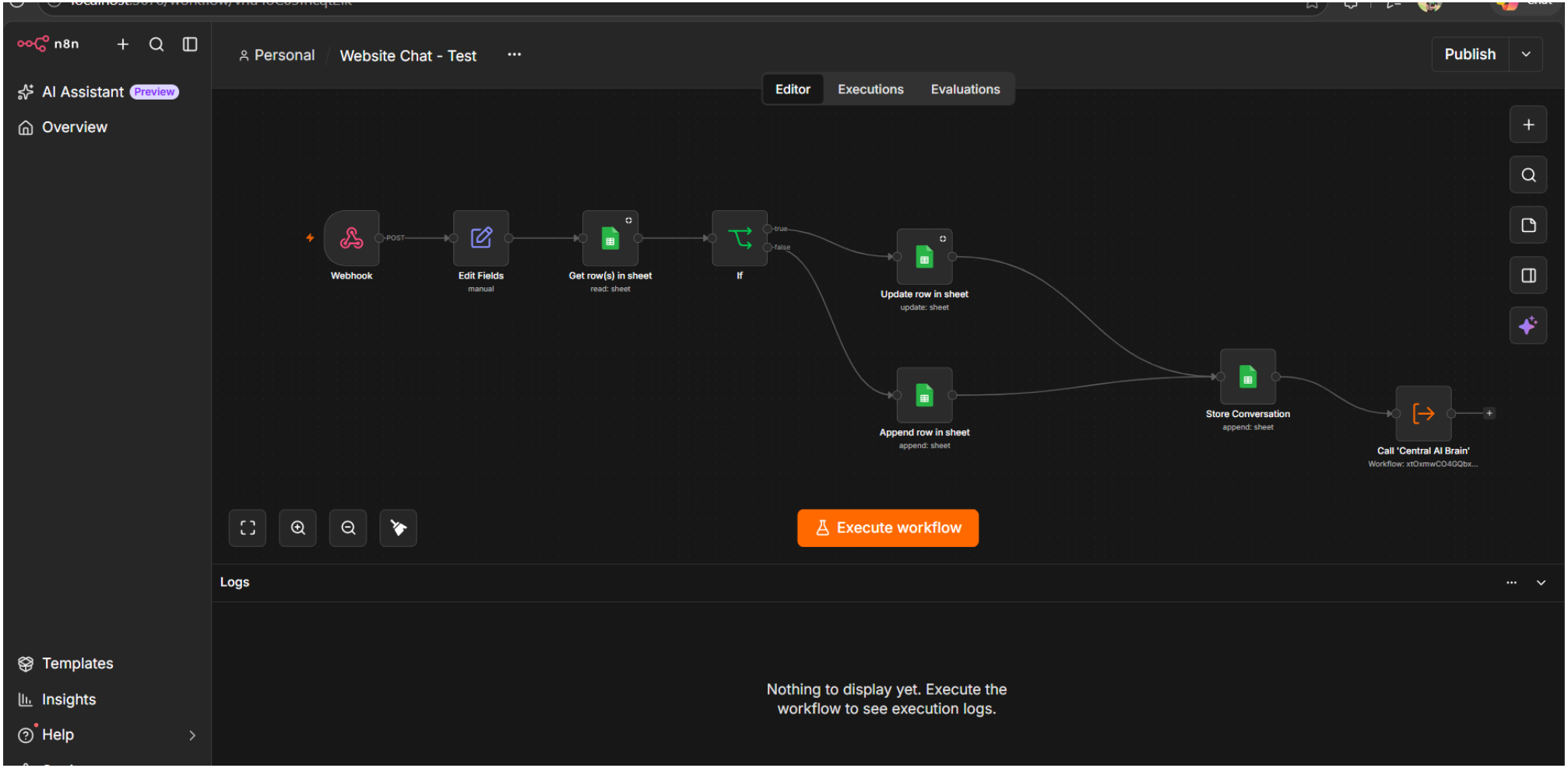
Multi-channel AI Agent: Cross-channel Conversation Context

Project: Multi-channel AI CRM / Agent using n8n
Live website: https://nevtech-main-website.netlify.app/

Requirement	Implementation
Central customer identity	Customer lookup/update/append workflow keeps one CRM profile using customer identifiers.
Conversation storage	Messages are stored in the centralized Conversations sheet.
Previous context retrieval	Get Previous Conversations filters conversation history by Customer ID.
Context preparation	Build Previous Context converts prior records into one previous_context value.
AI context handoff	previous_context is passed into the Central AI Brain workflow.
AI prompt behavior	LLM prompts were updated to use previous context and avoid unnecessary repeated questions.
Multi-channel design	Website, Email and WhatsApp workflows share the same CRM and Central AI Brain architecture.

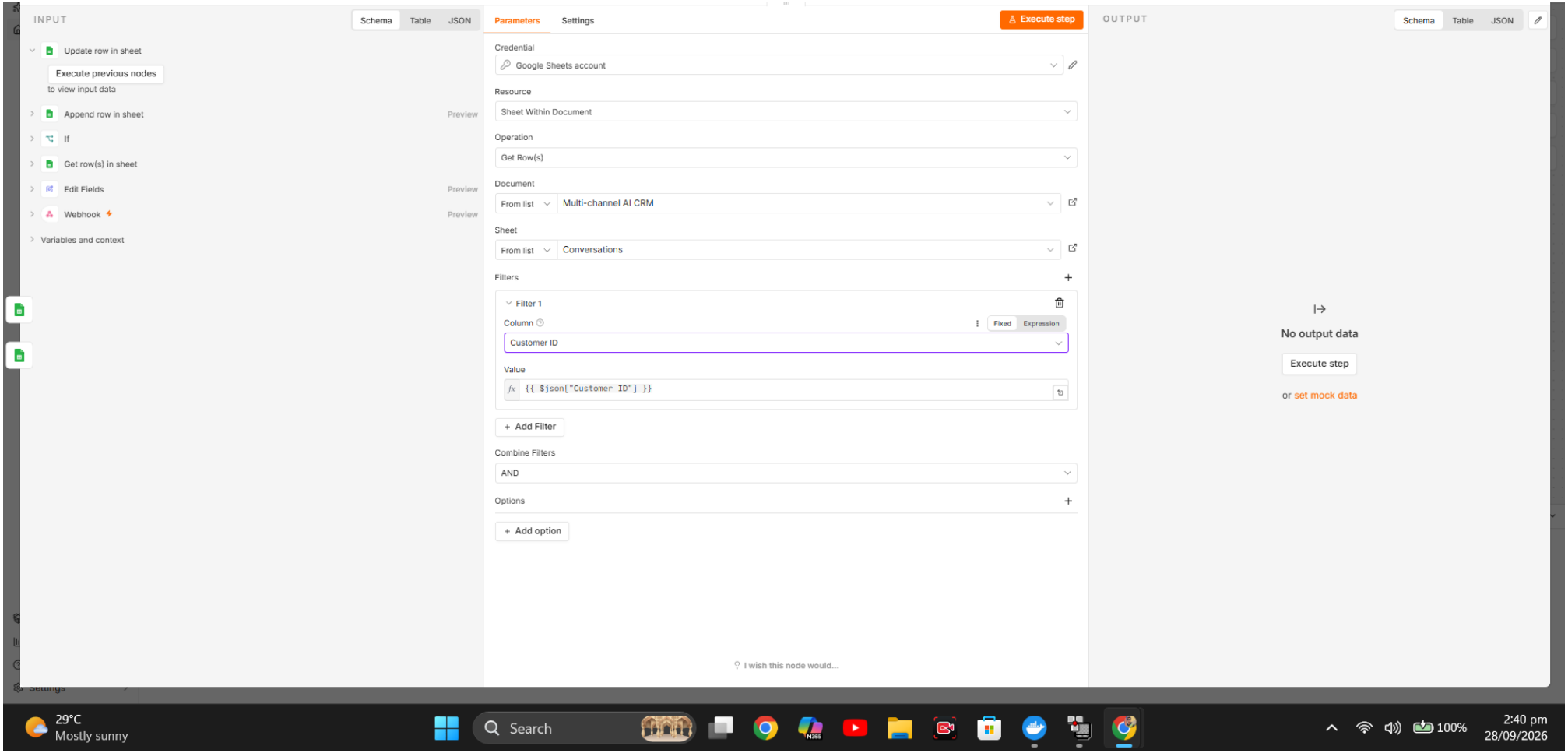
Implementation note: During the latest end-to-end test, the Gemini provider returned a temporary 503 high-demand response. The Phase 7 workflow structure and cross-channel context logic were implemented; the provider-side availability issue was isolated during testing.

Website Chat workflow extended for Phase 7



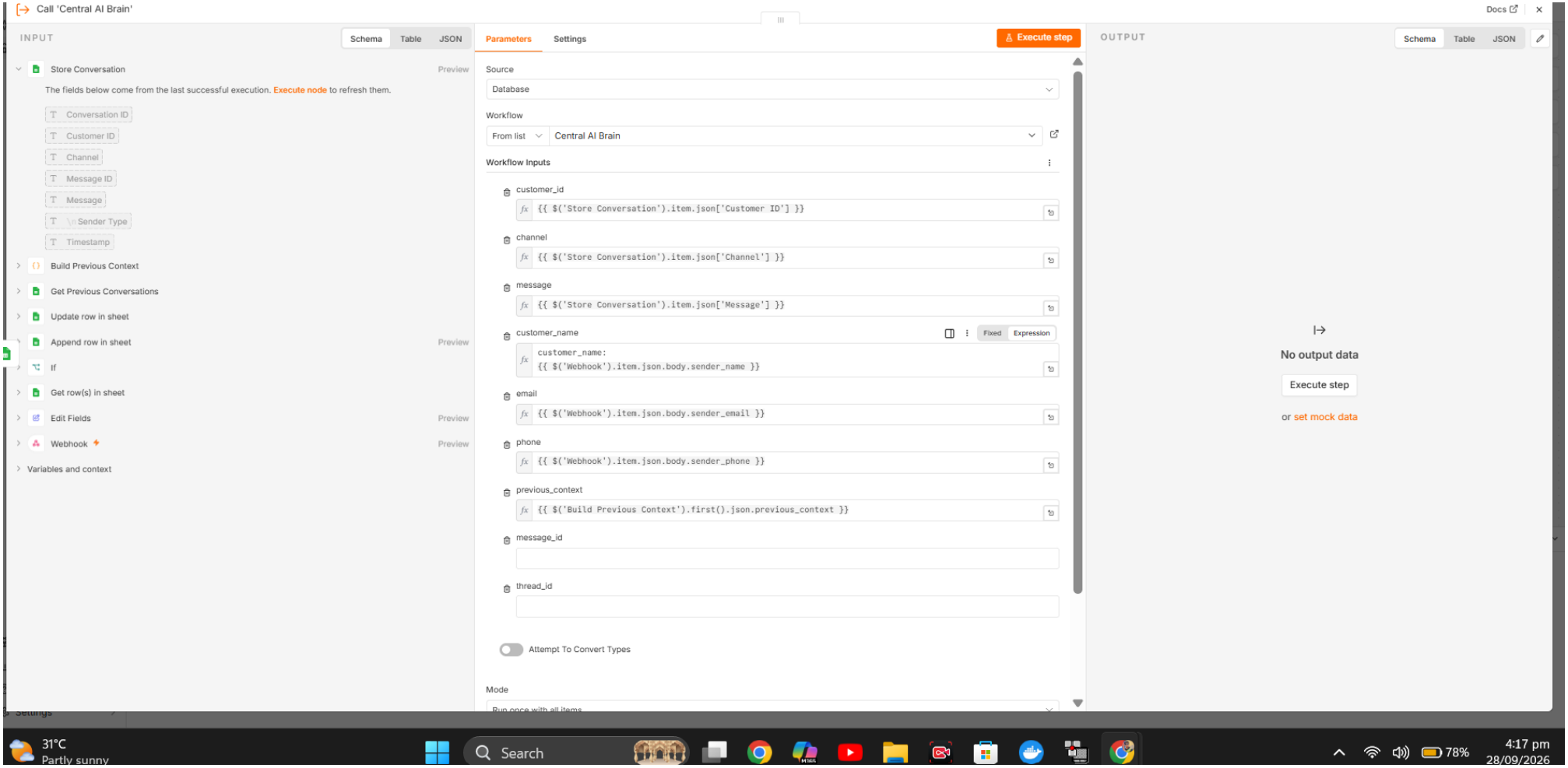
The existing website channel now performs customer lookup, update/append, conversation storage and calls the shared Central AI Brain.

Previous conversation retrieval



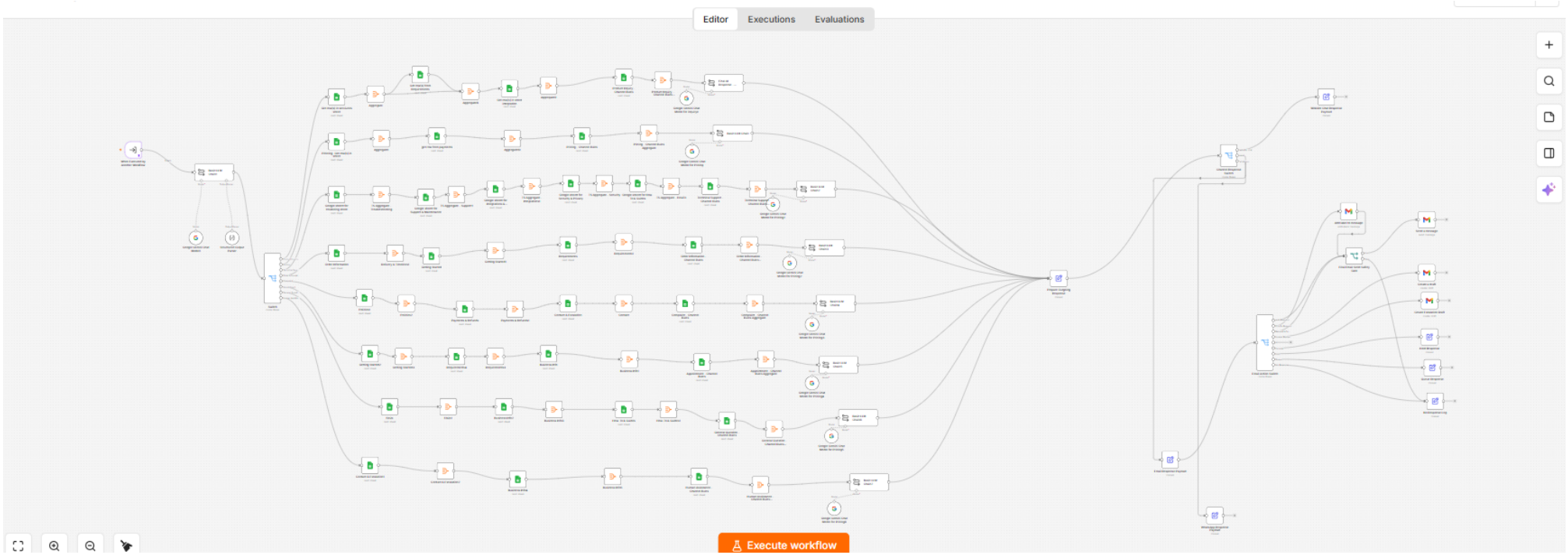
The Conversations sheet is queried by Customer ID so history is retrieved from the same centralized CRM.

Passing context to the Central AI Brain



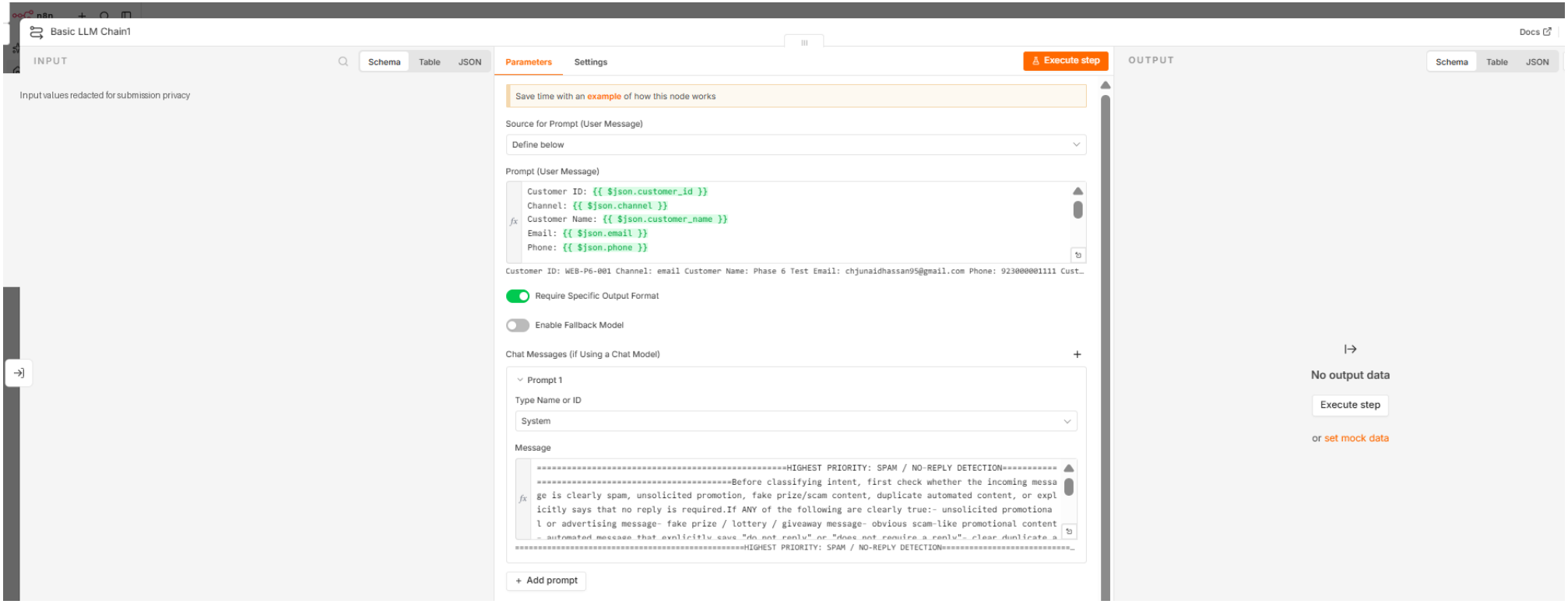
The website workflow passes previous_context into the reusable Central AI Brain sub-workflow.

Central AI Brain architecture



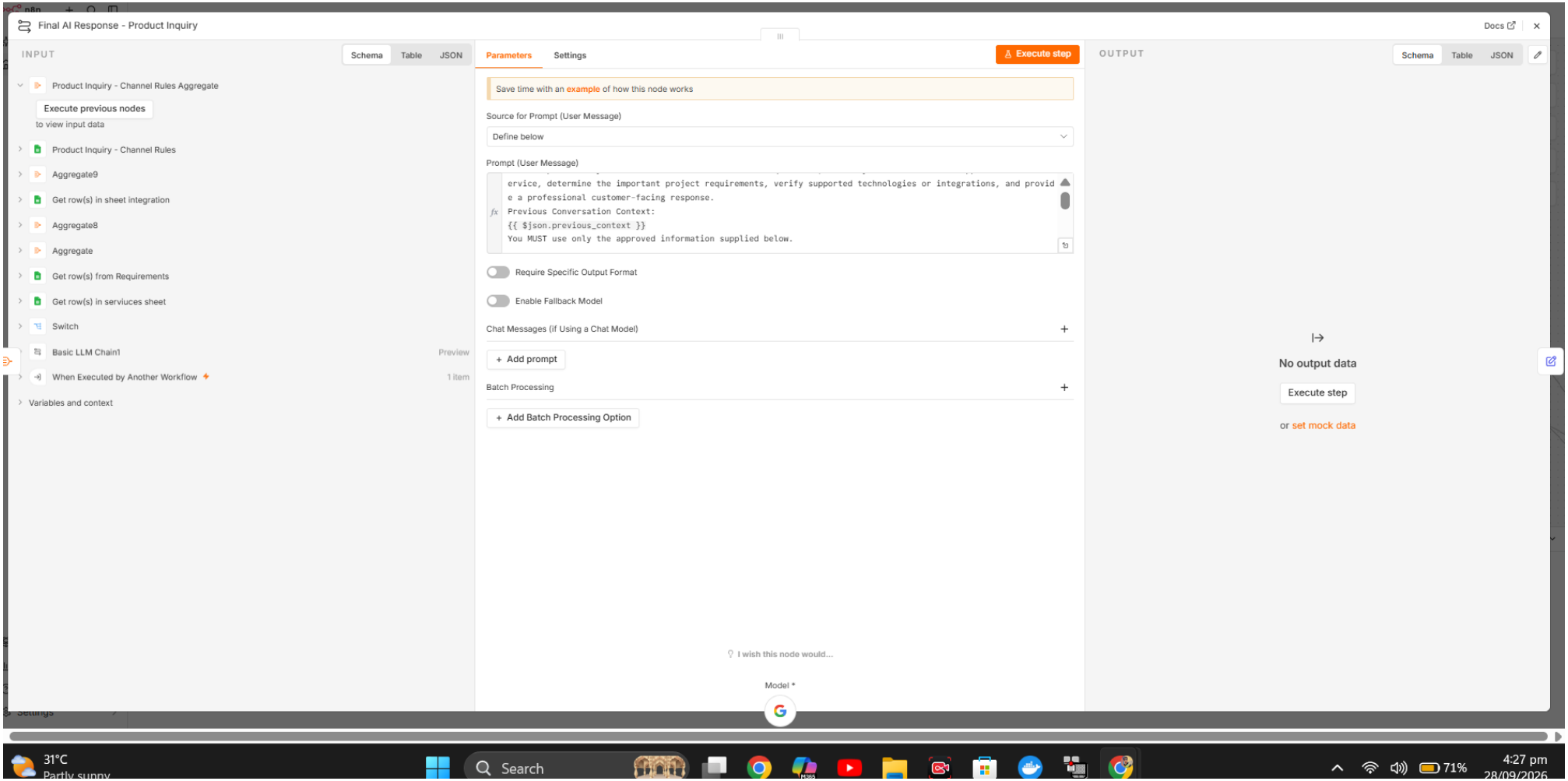
The shared AI brain contains intent routing and channel-specific processing used by multiple communication channels.

LLM prompt receives customer context



The AI prompt receives customer data plus previous_context, enabling continuity when the customer changes channels.

Final response prompt uses previous context



Final response generation was updated to reference previous conversation context before generating the customer-facing answer.

Submission Description

In Phase 7, I extended my existing Multi-channel AI Agent to support cross-channel conversation context. The system identifies customers through the centralized CRM, retrieves their previous conversations using Customer ID, builds a clean previous-context summary, and sends that context with the new message to the Central AI Brain. I also updated the AI prompts so the assistant can continue a conversation without unnecessarily asking the customer to repeat information already provided. The same centralized Customers and Conversations data is designed to be shared by the website, email, and WhatsApp channels. A live business website was also prepared and connected to the website-chat workflow for realistic testing.

Evidence included: Website Chat workflow, previous-conversation lookup, context handoff, Central AI Brain architecture, and AI prompt integration.

WhatsApp + n8n + ngrok

Full Troubleshooting & Recovery Guide

Docker • Fixed ngrok Domain • Meta Graph API • GET Verification • POST Messages

Purpose

This guide documents the exact issue we solved: WhatsApp messages were visible in Meta, but n8n was not receiving them because the ngrok tunnel could not connect to ngrok's agent service over TCP 443 on the current Wi-Fi network. The fix was to test the network, switch to a working connection, restart ngrok, verify the public URL, verify the GET challenge, and then re-activate the Meta callback.

Main root cause

connect.ngrok-agent.com:443 was blocked/timing out on the original network. Docker and n8n were running correctly, but the public ngrok tunnel was unreachable. Switching to a mobile hotspot changed **TcpTestSucceeded** from False to True.

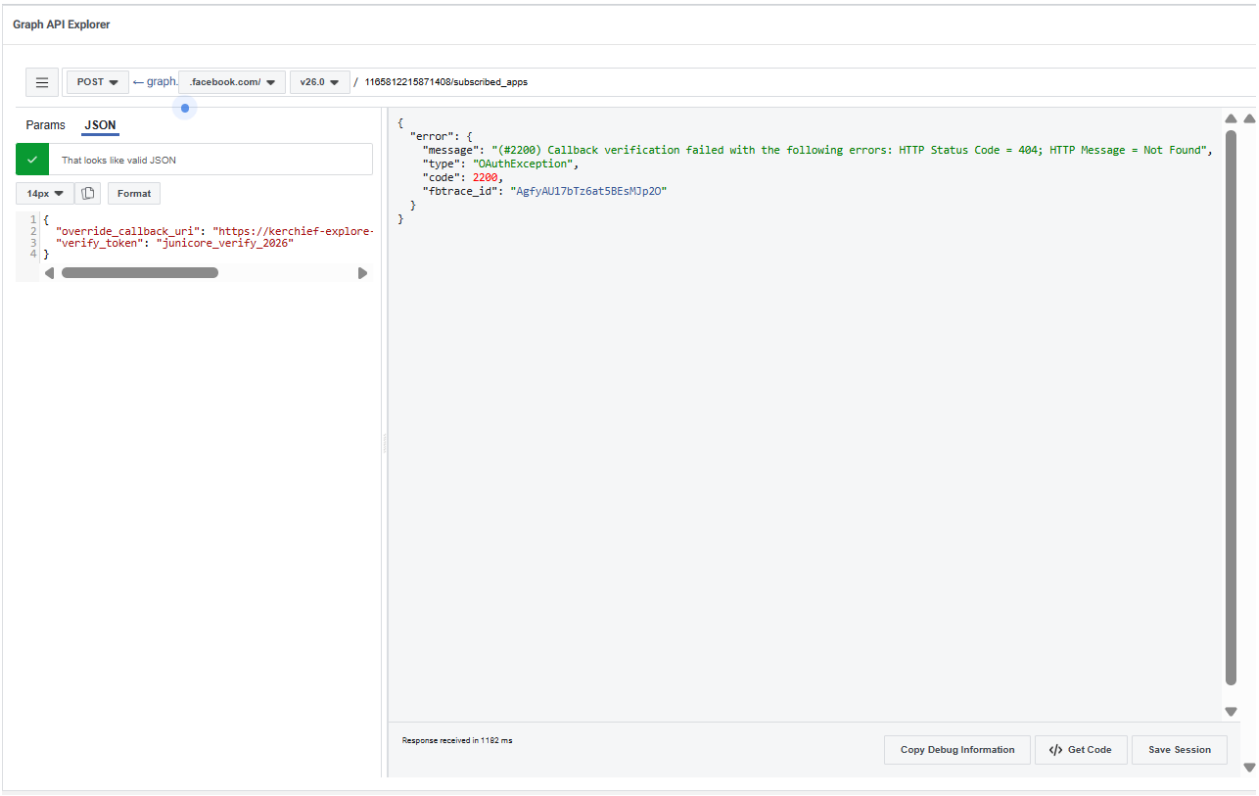
1. Quick Diagnosis Flow

Check	Question	If NO / Failed
1	Are n8n and ngrok containers Up?	Start/restart Docker Compose
2	Can connect.ngrok-agent.com:443 be reached?	Change network / hotspot
3	Does public ngrok URL return HTTP?	Restart ngrok, re-test network
4	Does GET challenge return 12345?	Fix GET webhook / publish workflow
5	Does Graph API callback return success:true?	Re-run callback POST
6	Does real WhatsApp create n8n execution?	Check POST webhook and Meta subscription

Golden rule

Do not reset/delete n8n data or rebuild the project first. Diagnose from the outside in: network → ngrok → n8n production webhook → Meta callback → real WhatsApp message.

2. Error We Saw: Meta Callback 404



Screenshot: Meta Graph API returned (#2200) callback verification failed, HTTP 404 Not Found.

This 404 means Meta tried to verify the callback URL but could not get the expected response from the public n8n GET webhook. A 404 can be caused by: the workflow not being published, the wrong URL (/webhook-test instead of /webhook), or the public tunnel being unreachable.

Step 1 - Open the project folder

When to do this: At the beginning of troubleshooting, before running Docker Compose commands.

Why: Docker Compose reads docker-compose.yml and .env from the project directory.

```
cd /d "C:\Users\Junaid Hassan\OneDrive\Documents\multichannel-ai-agent"
```

Expected: CMD prompt changes to the multichannel-ai-agent folder.

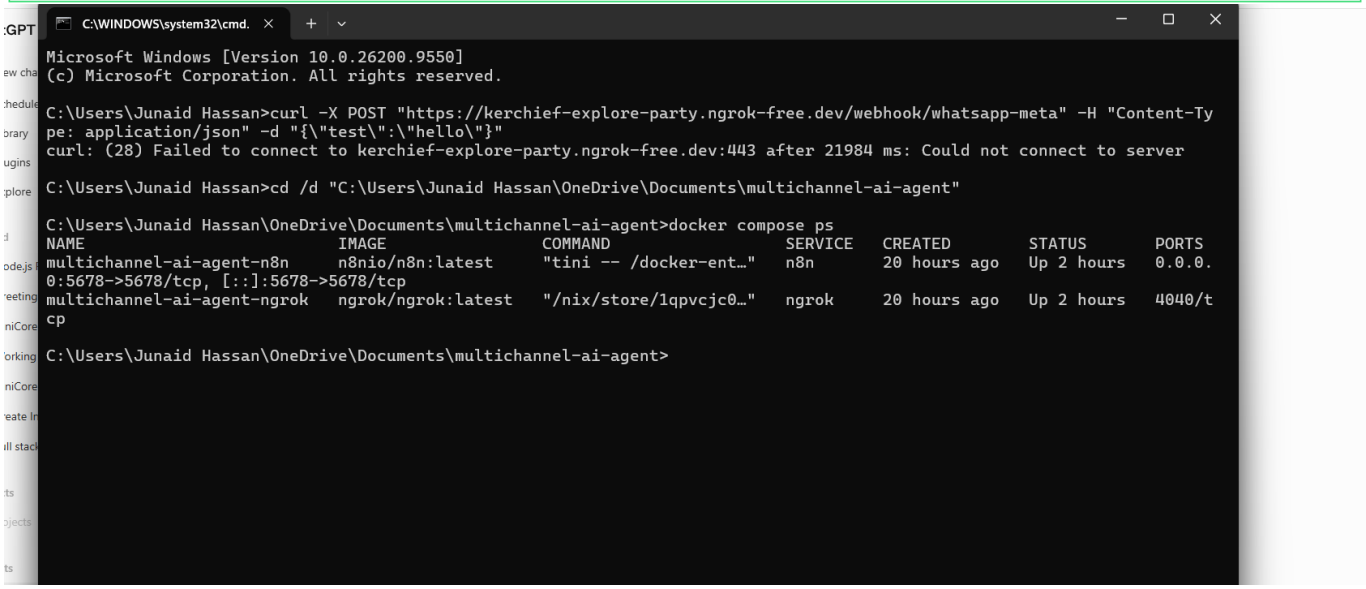
Step 2 - Check container status

When to do this: When WhatsApp stops reaching n8n or the public URL is not working.

Why: This confirms whether the local n8n service and ngrok agent are actually running.

```
docker compose ps
```

Expected: Both multichannel-ai-agent-n8n and multichannel-ai-agent-ngrok should show Up.



Screenshot: docker compose ps showed both n8n and ngrok containers Up.

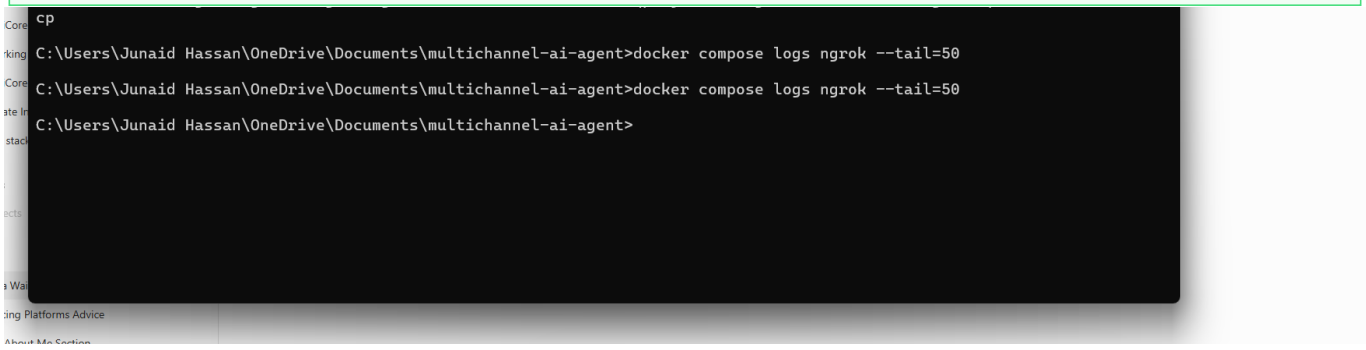
Step 3 - Inspect ngrok logs

When to do this: If the container says Up but the public URL still fails.

Why: A running container can still have a tunnel/auth/network problem. Logs may reveal it.

```
docker compose logs ngrok --tail=50
```

Expected: Normally you should see ngrok startup/tunnel information. Blank logs are a clue that the container state alone is not enough.



Screenshot: ngrok logs returned no useful tunnel output, so we continued with network tests.

Step 4 - Confirm the ngrok command

When to do this: If you want to make sure the container is using the fixed domain and correct upstream.

Why: This confirms the container is trying to expose n8n:5678 through the expected ngrok domain.

```
docker inspect multichannel-ai-agent-ngrok --format="{{.Config.Cmd}}"
```

Expected: Expected command: [http http://n8n:5678 --url=https://kerchief-explore-party.ngrok-free.dev]

3. The Critical Network Test

Step 5 - Test ngrok agent TCP 443

When to do this: When curl to the ngrok domain times out, or ngrok appears Up but Meta cannot reach n8n.

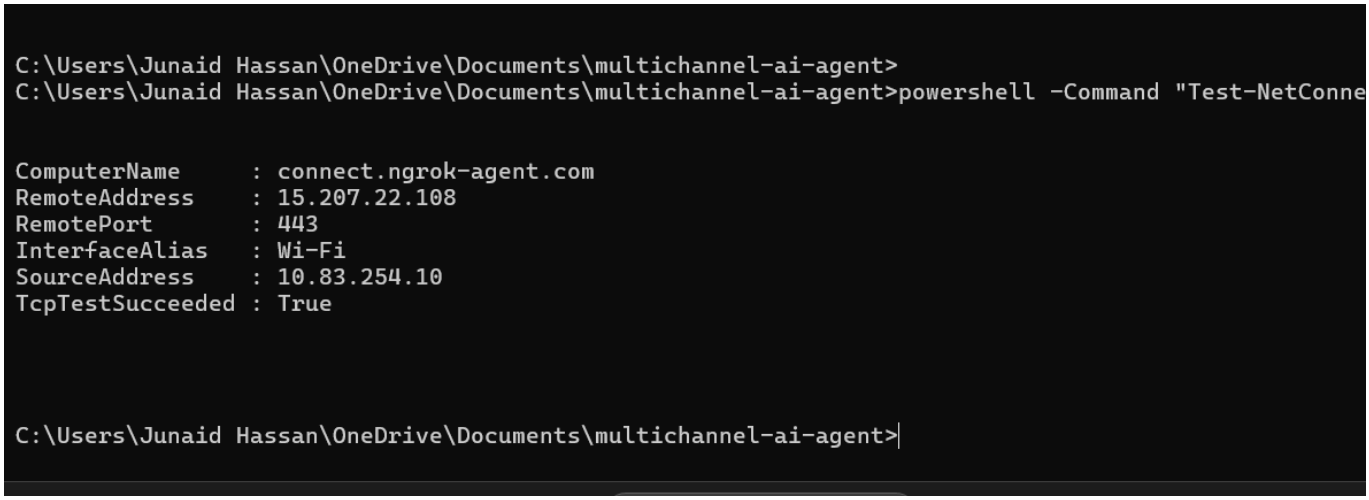
Why: ngrok must establish an outbound TLS connection to connect.ngrok-agent.com on port 443. If this is blocked, the tunnel cannot work even though Docker is Up.

```
powershell -Command "Test-NetConnection connect.ngrok-agent.com -Port 443"
```

Expected: TcpTestSucceeded : True

If you get False or a warning like TCP connect ...:443 failed

The current Wi-Fi/network is blocking or failing the ngrok agent connection. Switch to another network or mobile hotspot, then run the same test again.



Screenshot: after switching network, TcpTestSucceeded became True. This was the turning point.

What happened in our case

Original Wi-Fi: TCP 443 to connect.ngrok-agent.com failed. Mobile hotspot: TCP test passed. Therefore the root cause was network connectivity, not the n8n workflow or Docker data.

4. Restart and Verify the Public Tunnel

Step 6 - Restart only ngrok

When to do this: After TCP 443 becomes True.

Why: The ngrok agent needs to reconnect using the working network. Restarting only ngrok avoids touching n8n data.

```
docker compose restart ngrok
```

Expected: Container restarts without deleting workflows or credentials.

Step 7 - Test the fixed ngrok domain

When to do this: Immediately after the ngrok restart.

Why: This confirms the public hostname is reachable over HTTPS.

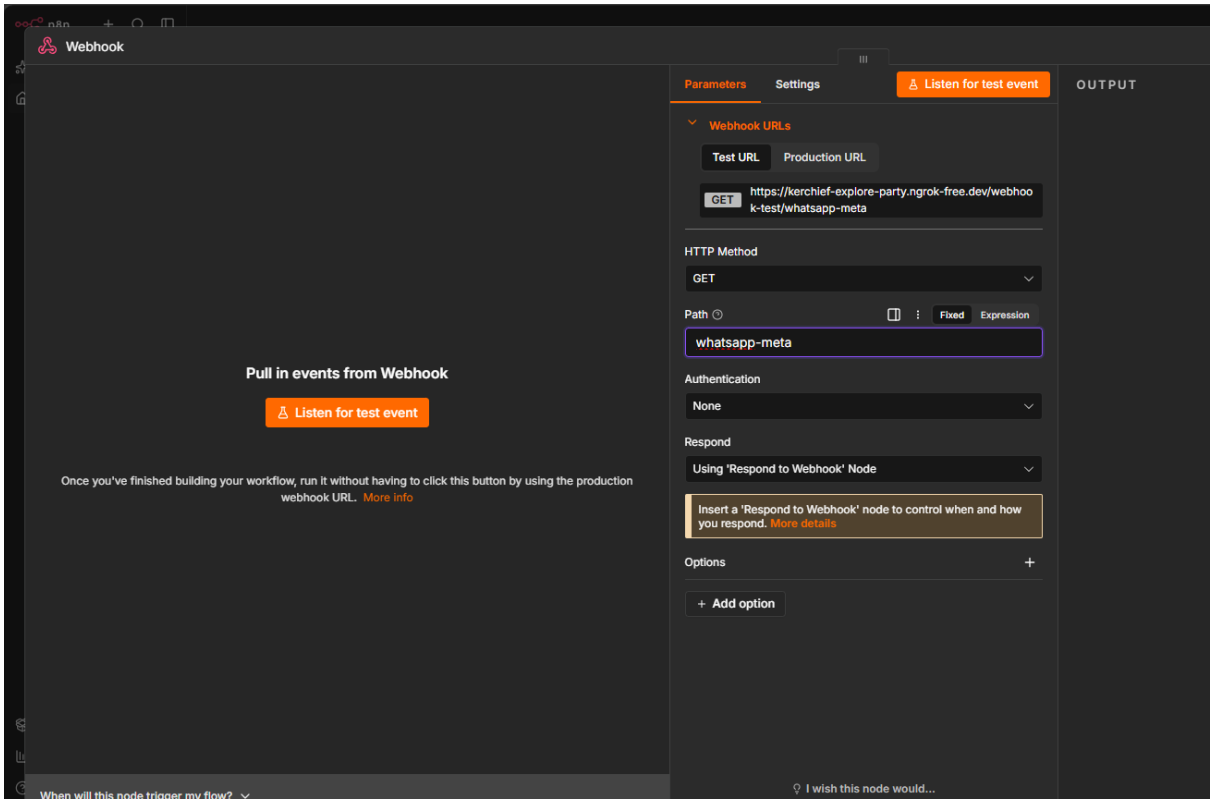
```
curl -I https://kerchief-explore-party.ngrok-free.dev
```

Expected: Any valid HTTP response proves reachability. In our case: HTTP/1.1 200 OK.

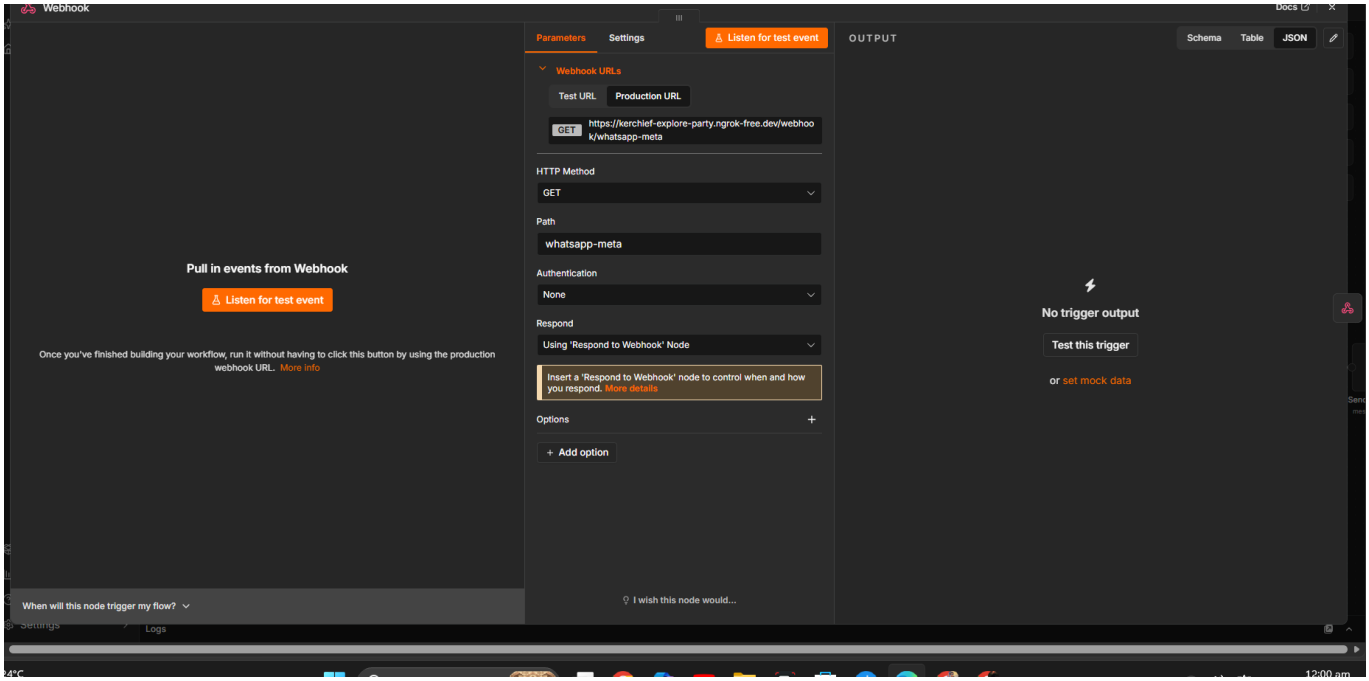
If this still times out, do not continue to Meta. Re-check Step 5 first.

5. n8n GET Webhook for Meta Verification

Meta verifies the webhook with a GET request. This is separate from the POST webhook that receives real WhatsApp message events.



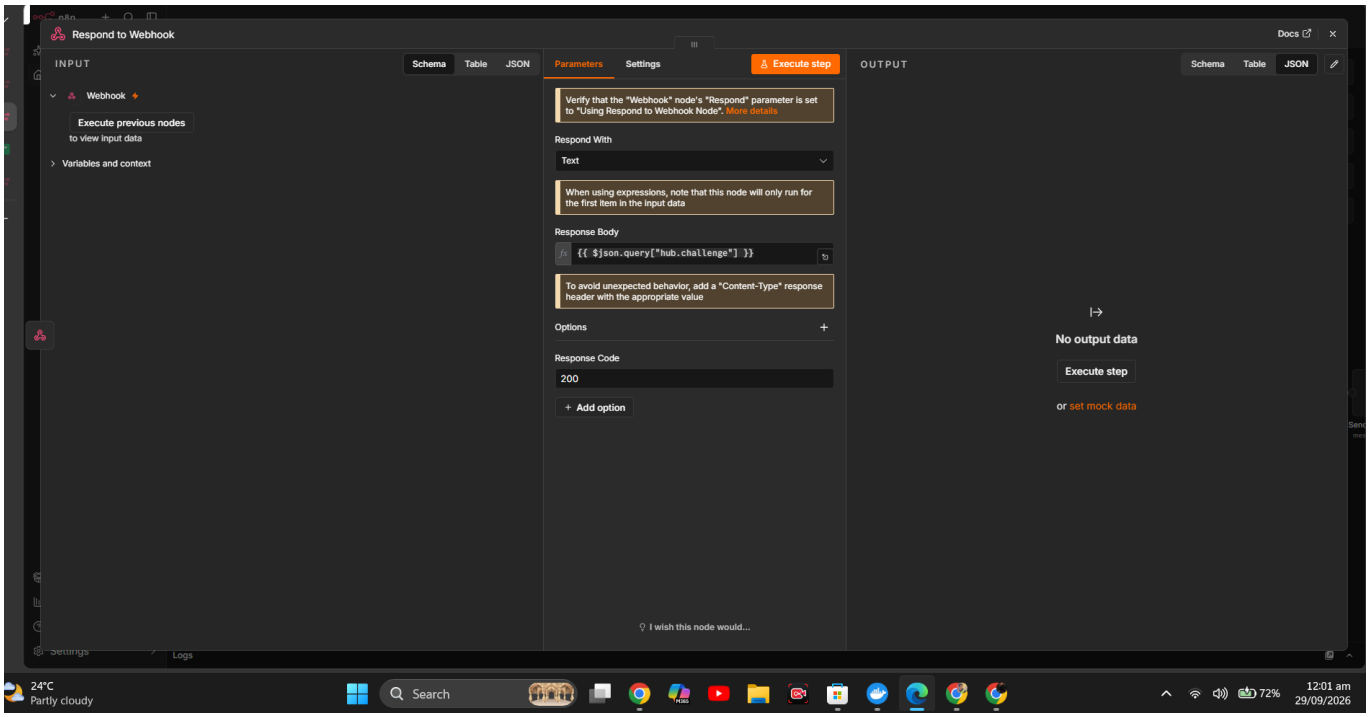
Screenshot: GET Webhook node with method GET, path whatsapp-meta, and Respond using Respond to Webhook Node.



Screenshot: Production URL selected. Important: Meta must use /webhook/whatsapp-meta, not /webhook-test/whatsapp-meta.

GET Webhook settings

```
HTTP Method: GET Path: whatsapp-meta Authentication: None Respond: Using "Respond to Webhook" Node
```



Screenshot: Respond to Webhook node configured to return the hub.challenge as Text with status 200.

```
Respond With: Text Response Body: {{ $json.query["hub.challenge"] }} Response Code: 200
```

Why this is required

During callback verification, Meta sends hub.challenge in the query string. n8n must return that exact challenge value with HTTP 200.

6. Direct GET Challenge Test

Step 8 - Verify the GET webhook yourself

When to do this: Before re-running Meta callback registration.

Why: This separates n8n/ngrok problems from Meta problems. If this command returns the challenge, your GET callback path is working.

```
curl "https://kerchief-explore-party.ngrok-free.dev/webhook/whatsapp-meta?hub.mode=subscribe&hub.verify_token=junicore_verify_2026&hub.challenge=12345"
```

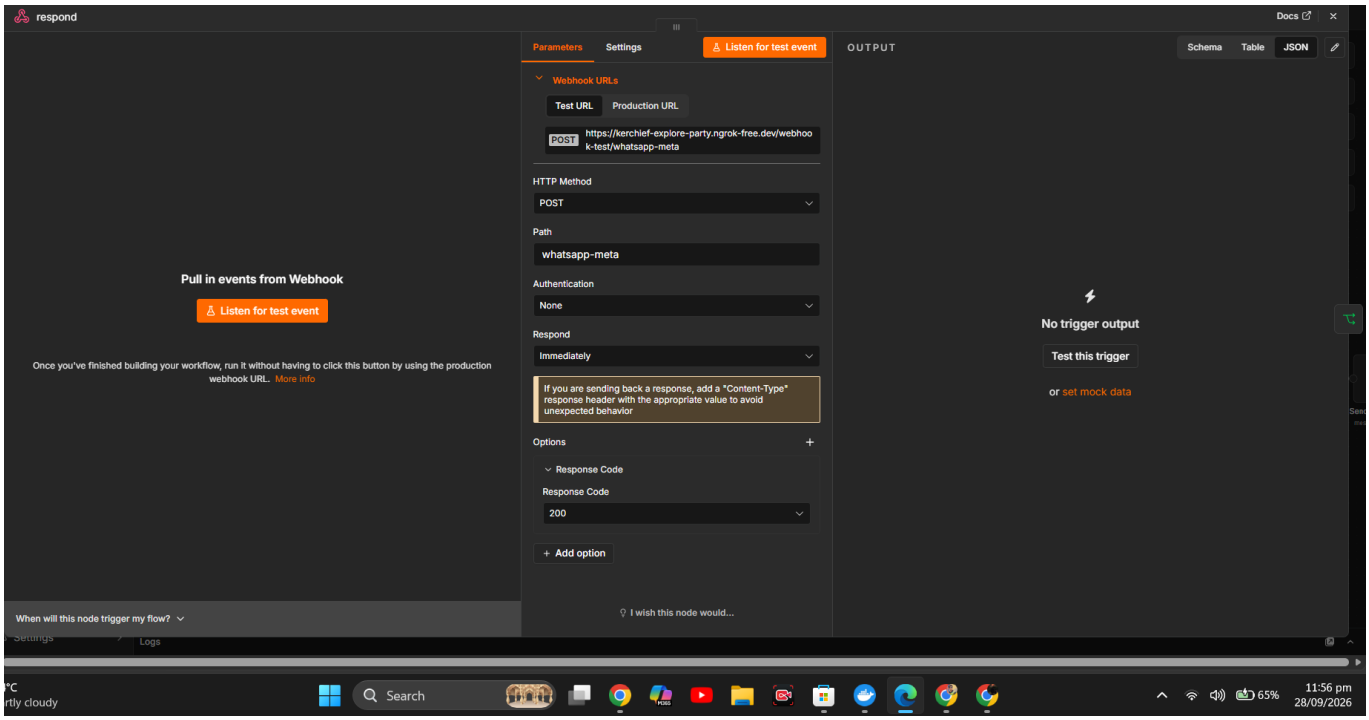
Expected: The terminal should print exactly: 12345

If you see 404

Check that the workflow is Published/Active and that the URL contains /webhook/ (production), not /webhook-test/. Also re-check that ngrok is reachable.

7. n8n POST Webhook for Incoming WhatsApp Events

Real WhatsApp messages arrive as POST requests. Keep a separate POST Webhook node using the same path.



Screenshot: POST webhook node for whatsapp-meta, Respond Immediately, response code 200.

```
HTTP Method: POST Path: whatsapp-meta Authentication: None Respond: Immediately Response Code: 200
```

Important

GET and POST can use the same path because the HTTP methods are different. GET handles Meta verification; POST handles message/status events.

8. Re-activate the Meta Callback

Step 9 - Send the callback registration request

When to do this: Only after the ngrok domain is reachable and the GET challenge returns 12345.

Why: This tells the WhatsApp Business Account to send webhook events to your fixed n8n production URL.

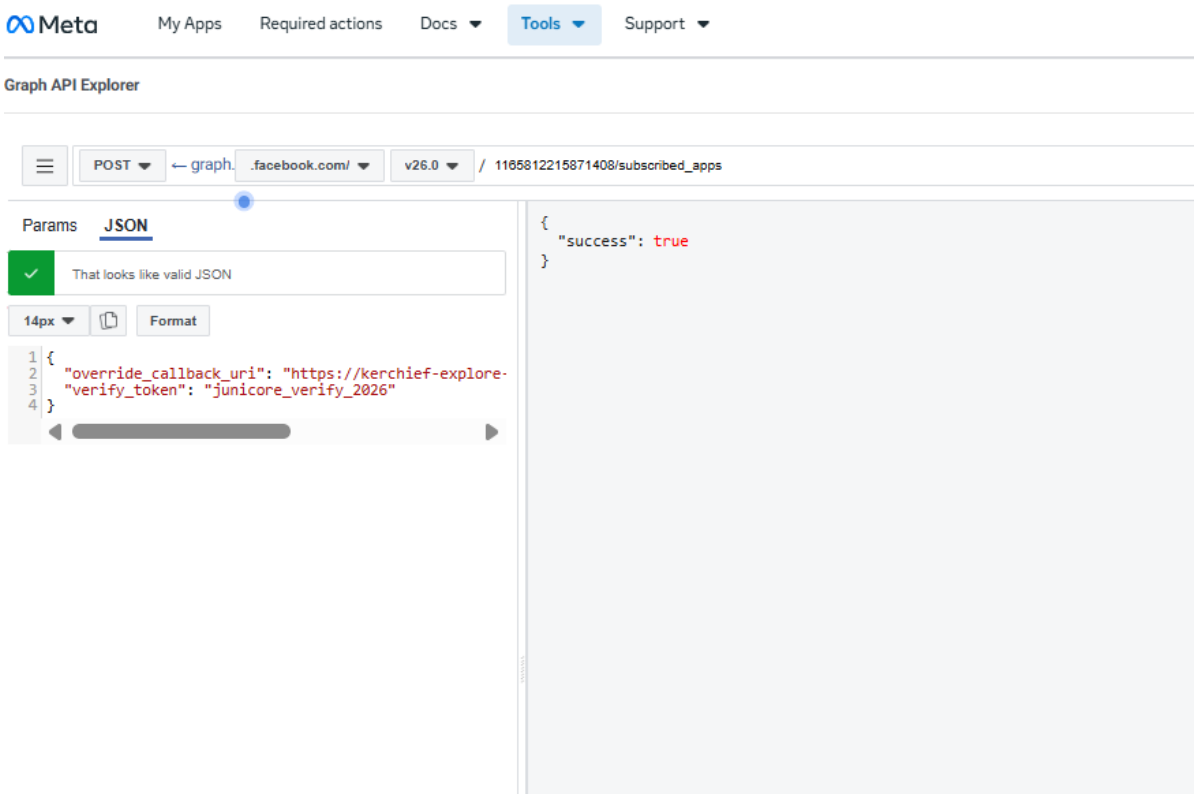
Expected: Graph API should return {"success": true}.

Graph API Explorer

```
Method: POST Path: 1165812215871408/subscribed_apps
```

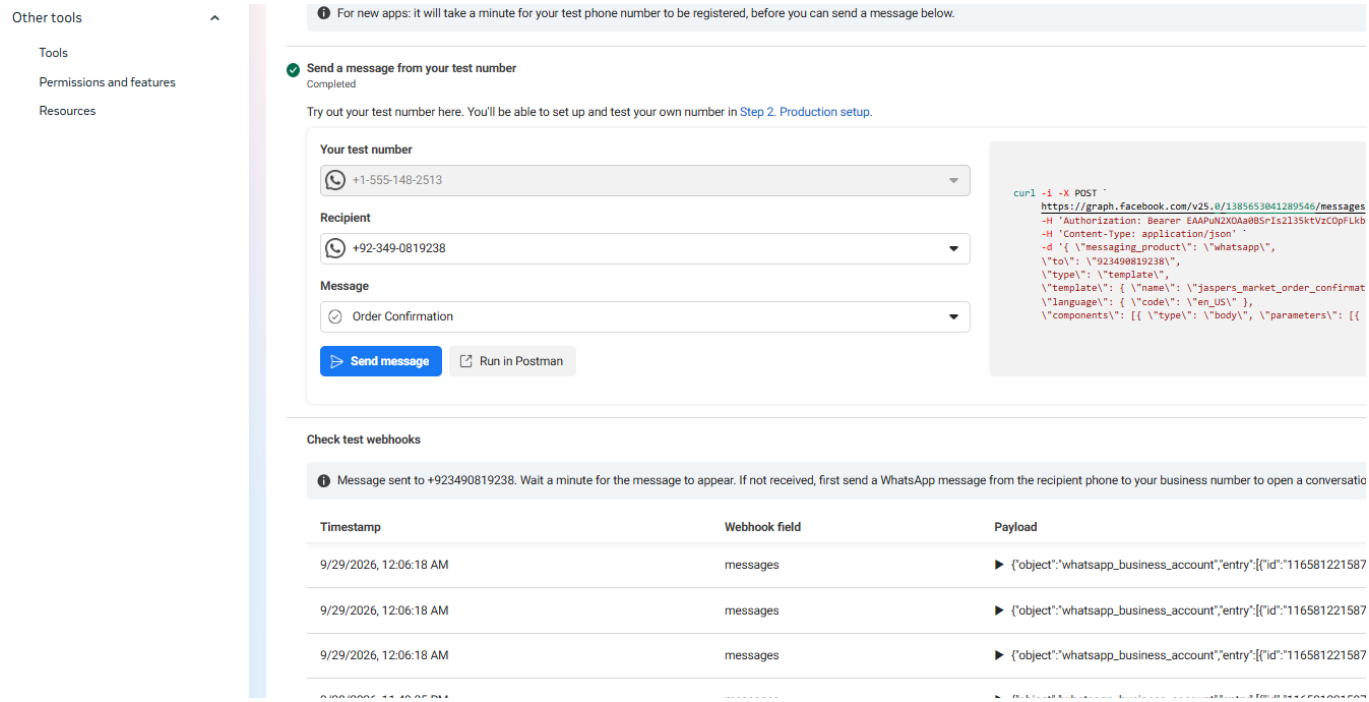
JSON body

```
{ "override_callback_uri": "https://kerchief-explore-party.ngrok-free.dev/webhook/whatsapp-meta",
"verify_token": "junicore_verify_2026" }
```



Screenshot: Meta Graph API returned success: true after the network/tunnel fix.

9. Meta Can Show Messages Even When n8n Is Not Receiving Them



Screenshot: Meta test page showed messages/webhook events even while our n8n delivery path still needed fixing.

Seeing events under Meta's test webhook area proves WhatsApp → Meta is working. It does not automatically prove Meta → your ngrok callback → n8n is working. Always verify the public callback and n8n executions separately.

10. Direct POST Test to n8n

Step 10 - Test the POST webhook directly

When to do this: If Meta shows messages but n8n still has no execution.

Why: This tests ngrok → n8n POST delivery without involving Meta.

```
curl -X POST "https://kerchief-explore-party.ngrok-free.dev/webhook/whatsapp-meta" -H "Content-Type: application/json" -d "{\"test\":\"hello\"}"
```

Expected: A new n8n execution should appear for the POST webhook.

How to interpret the result

Execution appears: ngrok + n8n POST are working; investigate Meta subscription/callback.
No execution / curl timeout: fix ngrok/network first.

11. Common Errors and Exact Meaning

Error / Symptom	Meaning	Fix
(#2200) Callback verification failed, 404	Meta cannot get the expected GET callback response	Use Production URL, publish workflow, test challenge manually
curl (28) Could not connect to ngrok-free.dev:443	Public ngrok endpoint is unreachable	Test connect.ngrok-agent.com:443; change network; restart ngro
TcpTestSucceeded : False	Current network cannot reach ngrok agent over 443	Use another Wi-Fi / mobile hotspot
Containers Up, but tunnel fails	Container status is not proof of public connectivity	Run TCP test + curl public domain
Meta test events visible, n8n empty	WhatsApp→Meta works, callback delivery does not	Direct POST test; verify Meta callback
GET URL works but POST does not	Verification route is okay, message route/config may be wrong	Check POST webhook published and Respond Immediately

12. Full Command Cheat Sheet

Open project folder

```
cd /d "C:\Users\Junaid Hassan\OneDrive\Documents\multichannel-ai-agent"
```

Check services

```
docker compose ps
```

Check all service states

```
docker compose ps -a
```

View ngrok logs

```
docker compose logs ngrok --tail=50
```

Inspect ngrok command

```
docker inspect multichannel-ai-agent-ngrok --format="{{.Config.Cmd}}"
```

Test ngrok agent TCP

```
powershell -Command "Test-NetConnection connect.ngrok-agent.com -Port 443"
```

Restart only ngrok

```
docker compose restart ngrok
```

Restart full Compose stack if needed

```
docker compose restart
```

Test public domain headers

```
curl -I https://kerchief-explore-party.ngrok-free.dev
```

Test GET challenge

```
curl "https://kerchief-explore-party.ngrok-free.dev/webhook/whatsapp-meta?hub.mode=subscribe&hub.verify_token=junicore_verify_2026&hub.challenge=12345"
```

Test POST webhook

```
curl -X POST "https://kerchief-explore-party.ngrok-free.dev/webhook/whatsapp-meta" -H "Content-Type: application/json" -d "{\"test\":\"hello\"}"
```

13. Recovery Order - Use This Every Time

1. docker compose ps → confirm n8n and ngrok are Up.
2. Test-NetConnection connect.ngrok-agent.com -Port 443.
3. If False → switch network/hotspot; do not change n8n yet.
4. docker compose restart ngrok.
5. curl -I fixed ngrok domain → require an HTTP response.
6. Test GET hub.challenge → require 12345.
7. Confirm GET workflow is Published and uses Production URL.
8. Confirm POST workflow is Published and responds 200.
9. Graph API POST subscribed_apps → require success:true.
10. Send real WhatsApp message and check n8n Executions.

Do NOT do these first

Do not delete containers, remove the n8n data folder, regenerate workflows, or change Meta URLs randomly before the network/tunnel tests. Those actions can create new problems while the real issue is only connectivity.

Security

Never paste Meta access tokens, ngrok authtokens, API keys, or passwords into screenshots or public chats. If a token is exposed, rotate/revoke it. The verify token shown in this guide is a webhook verification string, not a Meta access token.

PHASE 8 SUBMISSION REPORT

Multi-channel AI Agent - AI Routing, Human Handoff and Escalation

Objective

Build an intelligent routing system that decides whether AI should answer, route the request, or stop automation and escalate the conversation to a human representative.

Project	Multi-channel AI Agent
Phase	8 - AI Routing, Human Handoff and Escalation
Platform	n8n + Google Sheets CRM + Gmail + Website Chat + WhatsApp
AI Control	Central AI Brain workflow
Result	Human escalation branch implemented and tested across supported channels

Submission Description

In Phase 8, I extended the existing multi-channel AI agent with intelligent human escalation. The Central AI Brain now identifies complaints, sensitive requests, repeated failed answers, complex questions, and explicit requests for a human. When escalation is required, the automation generates a customer summary, sends a notification email, updates the CRM status to "Escalated to Human", stops the normal automated response path, and returns a controlled handoff confirmation through the active customer channel.

The implementation remains part of the same project and reuses the existing website, email, WhatsApp, customer profile, conversation history, and centralized AI routing workflows.

1. Phase 8 Requirements Coverage

Requirement	Implementation
1-3. Detect human-assistance situations	Central AI Brain classification and structured field: human_escalation_required
Complaints / sensitive / failed / complex / human request	Explicit escalation rules added to AI routing prompt
4. Escalation branch	IF node checks the boolean escalation decision
5-6. Conversation summary	Generate Escalation Summary node includes identity, channel, history, issue, and AI analysis
7. Test notification	Gmail node sends a Human Escalation Required email
8. CRM update	Customers sheet is updated to Customer Status = Escalated to Human
9. Stop inappropriate auto replies	Escalated requests bypass normal response and use Human Handoff Response
10. Channel tests	Website Chat, Email, and WhatsApp are routed through the same Central AI Brain

Escalation decision logic

Normal case: AI response -> IF false -> Channel Response Switch -> website/email/WhatsApp response.

Escalation case: AI response -> IF true -> Generate Escalation Summary -> Email notification + CRM update -> Human Handoff Response -> Channel Response Switch.

The boolean field human_escalation_required is the key control signal. This lets one centralized AI decision govern all supported channels consistently.

2. Human Escalation Workflow

The central workflow now contains a dedicated true branch for human escalation while the false branch continues to the existing channel response switch.

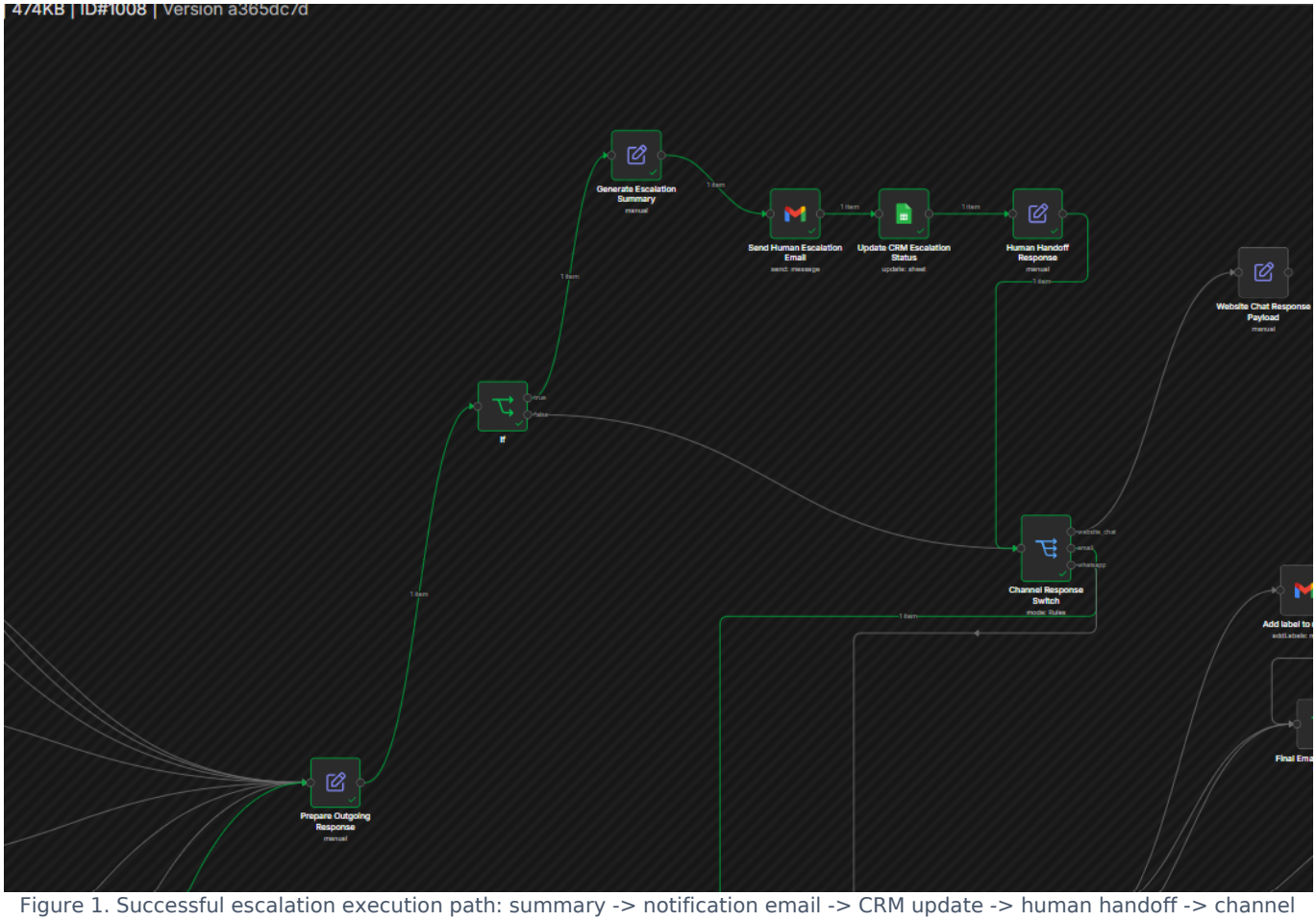


Figure 1. Successful escalation execution path: summary -> notification email -> CRM update -> human handoff -> channel switch.

Human Handoff Response

When a case is escalated, the normal AI-generated answer is not used as the customer-facing response. Instead, the workflow returns a controlled message confirming that the request has been escalated to a human support representative.

3. CRM Escalation Status Update

The CRM update matches the customer using Customer ID and changes only the Customer Status field. Existing customer data such as name, email, phone, company, and interaction history is preserved.

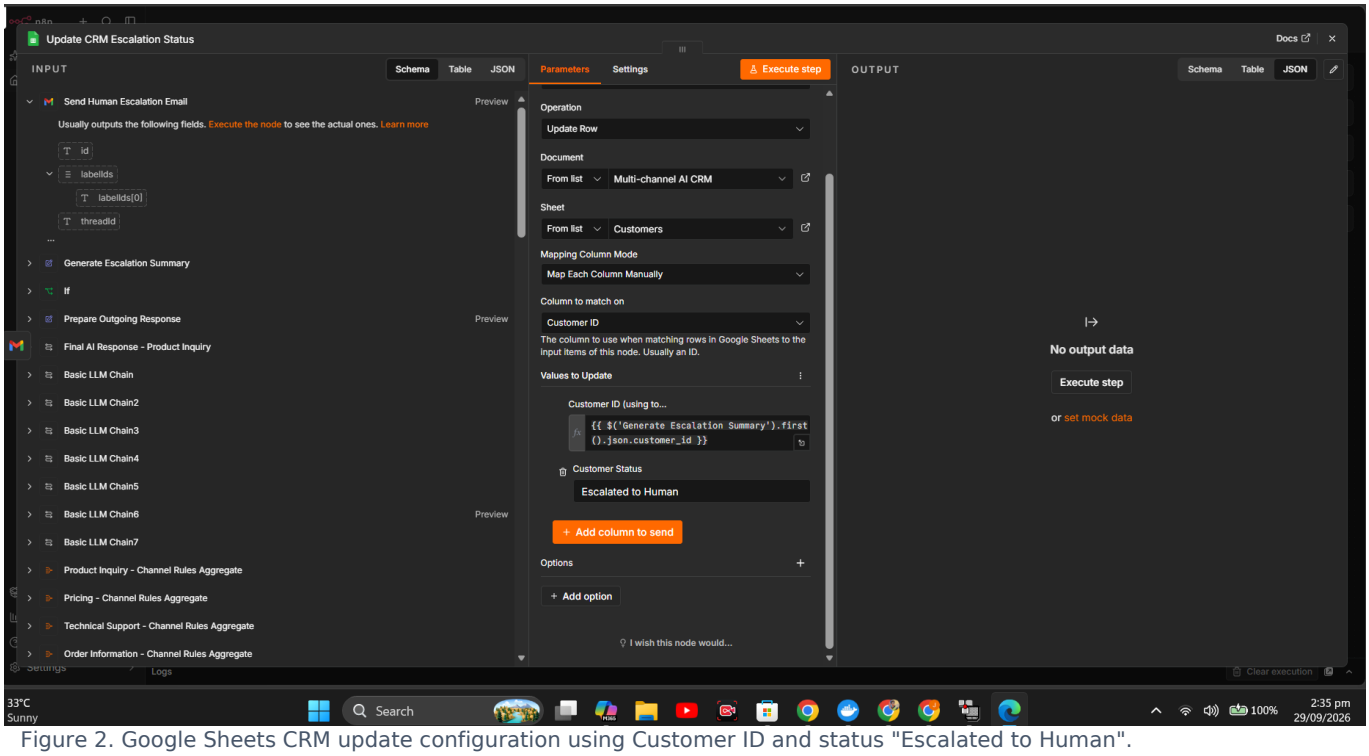


Figure 2. Google Sheets CRM update configuration using Customer ID and status "Escalated to Human".

CRM value used:

Customer Status = Escalated to Human

This provides a persistent record that the automation has handed the case to a person and should no longer treat the conversation as a normal automated support case.

4. Email Routing and Human Notification

The escalation flow sends an internal notification email containing the customer identity, channel, current issue, previous conversation context, AI analysis, and escalation status. The customer receives a separate handoff confirmation through the correct communication channel.

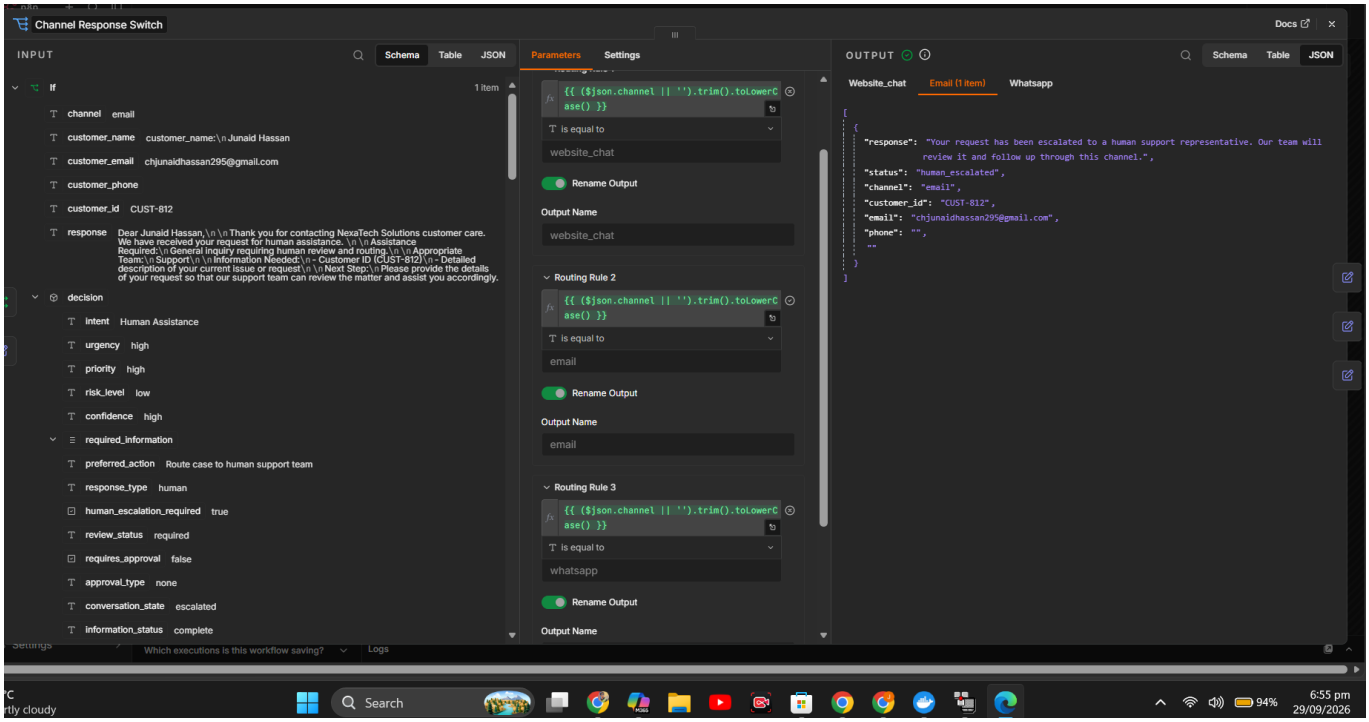


Figure 3. Channel Response Switch routes the handoff payload to the active channel.

During email-channel testing, two emails can be expected when the same inbox is used for both roles: one internal escalation notification and one customer handoff confirmation. This is intentional behavior, not a duplicate execution.

5. Website Chat Test Evidence

The website workflow continues to reuse the existing customer lookup, append/update CRM logic, previous-conversation retrieval, conversation storage, and Central AI Brain sub-workflow.

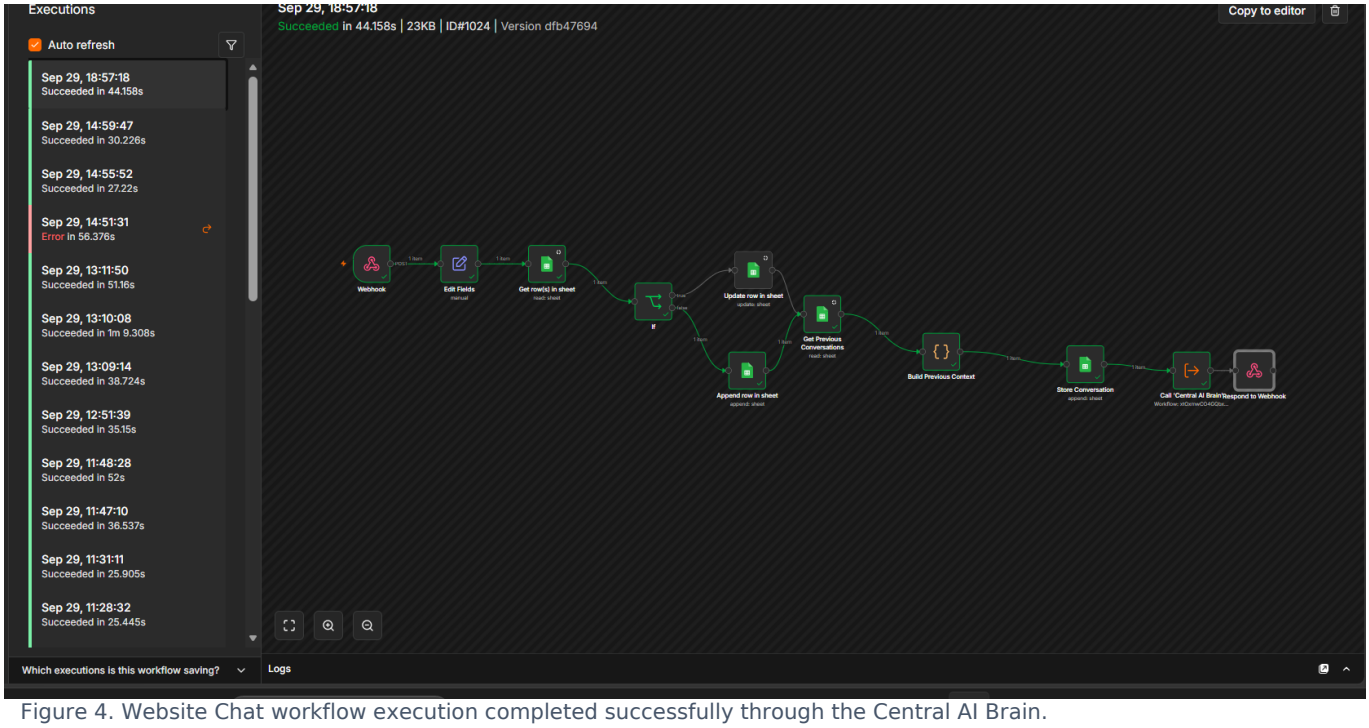


Figure 4. Website Chat workflow execution completed successfully through the Central AI Brain.

Test message used for escalation:

I want to speak with a human support representative.

Expected result: the request is classified for human assistance, the escalation branch runs, the CRM is updated, an internal email is generated, and the customer receives the handoff confirmation instead of a normal AI answer.

6. WhatsApp Integration and Final Test

WhatsApp uses the same centralized customer and AI-routing architecture. Incoming messages are normalized, matched against the CRM, stored in the conversation log, sent to the Central AI Brain, limited to one outgoing item, and then delivered through the WhatsApp send-message node.

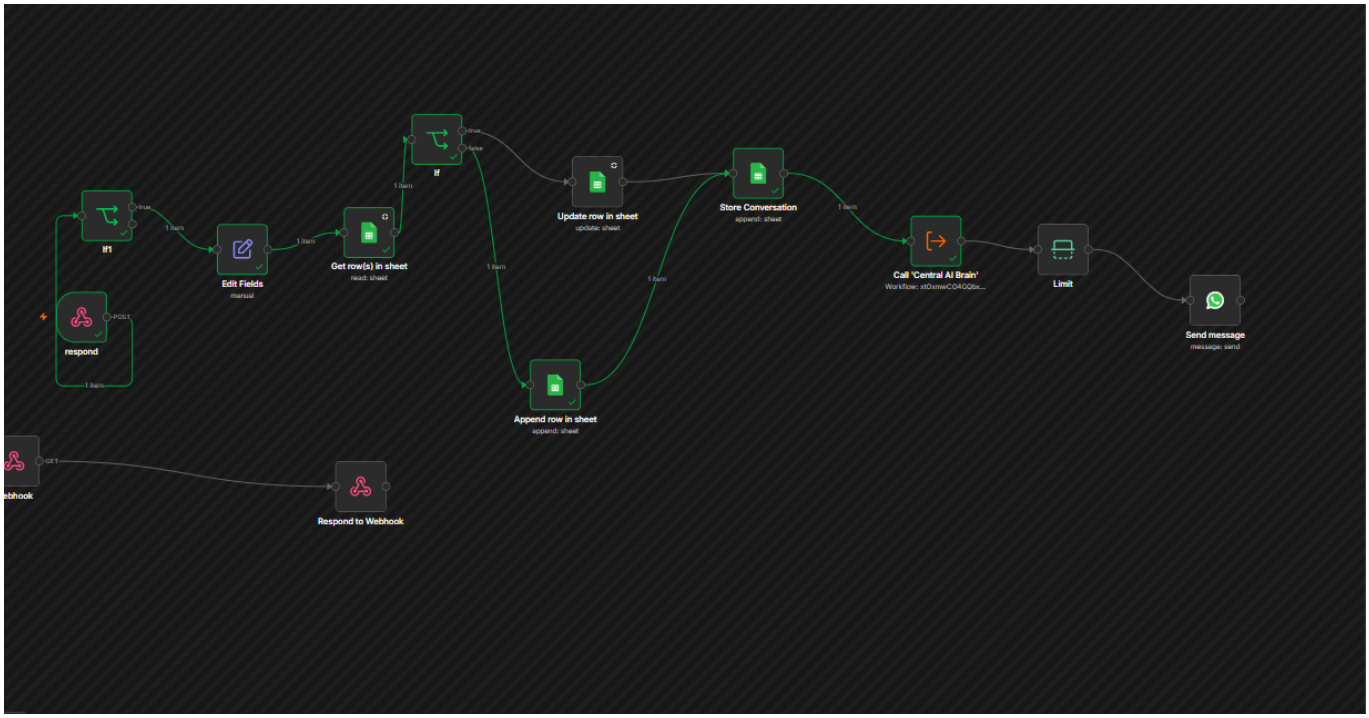


Figure 5. WhatsApp workflow architecture connected to the Central AI Brain and final Send message node.

For Phase 8 testing, the same explicit human-assistance message is used. The centralized escalation decision ensures that website, email, and WhatsApp follow one consistent human-handoff policy.

7. Final Result and Submission Checklist

Item	Status
Human escalation detection	Implemented
Complaint / sensitive / failed / complex / explicit-human rules	Implemented
IF escalation branch	Implemented
Conversation escalation summary	Implemented
Internal email notification	Implemented
CRM escalation status	Implemented
Normal automated reply blocked after escalation	Implemented
Controlled human handoff response	Implemented
Website Chat test	Completed
Email test	Completed
WhatsApp integration/test	Completed

Ready-to-paste submission description

Phase 8 completed: I implemented AI routing, human handoff, and escalation in my existing multi-channel AI Agent. The Central AI Brain detects complaints, sensitive requests, repeated failed answers, complex requests, and explicit requests for a human. Escalated conversations are summarized with customer identity, channel, conversation history, issue, and AI analysis. The system sends an internal escalation email, updates the Google Sheets CRM status to "Escalated to Human", prevents the normal automated reply path, and returns a controlled human-handoff confirmation. The same logic is integrated with Website Chat, Email, and WhatsApp and was tested using real workflow executions.

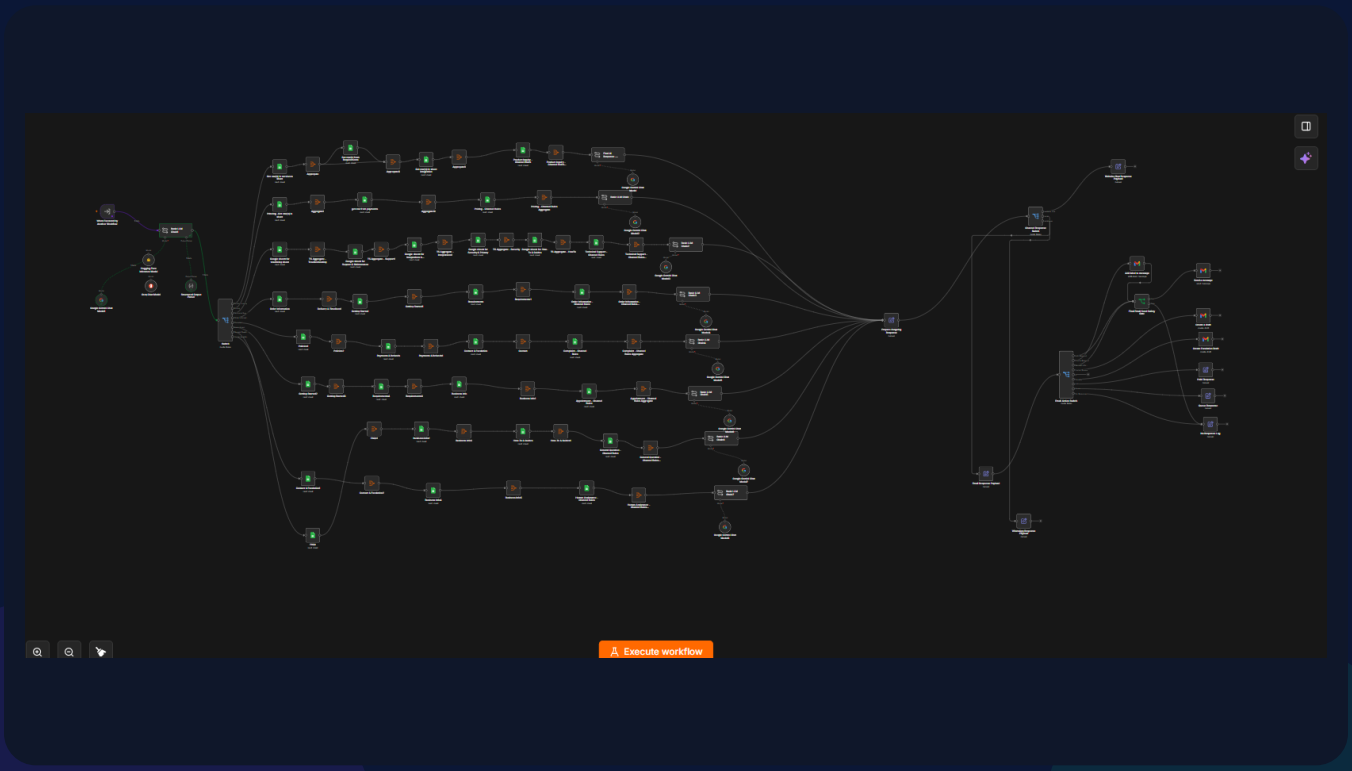
Phase 8 Status: COMPLETE

FINAL INTERNSHIP PROJECT

Multi-channel AI Agent

Centralized AI automation across Website, Email & WhatsApp

n8n | AI Routing | Google Sheets CRM | Human Handoff



Muhammad Junaid Hassan

AI Automation • Full-Stack Learning • Portfolio Build

PROJECT OVERVIEW

One AI brain. Three channels. One customer context.

A portfolio-grade prototype for a fictional business, designed to keep customer data and conversation context centralized.

3

Channels

Website Chat • Email • WhatsApp

1

Unified CRM

Customer profile + conversation history

8

Intent Routes

Product, pricing, support, complaint & more

Human

Handoff

AI stops and escalates when needed

THE PROBLEM

Customers can arrive through different channels, but fragmented automation can lose identity, history and context.

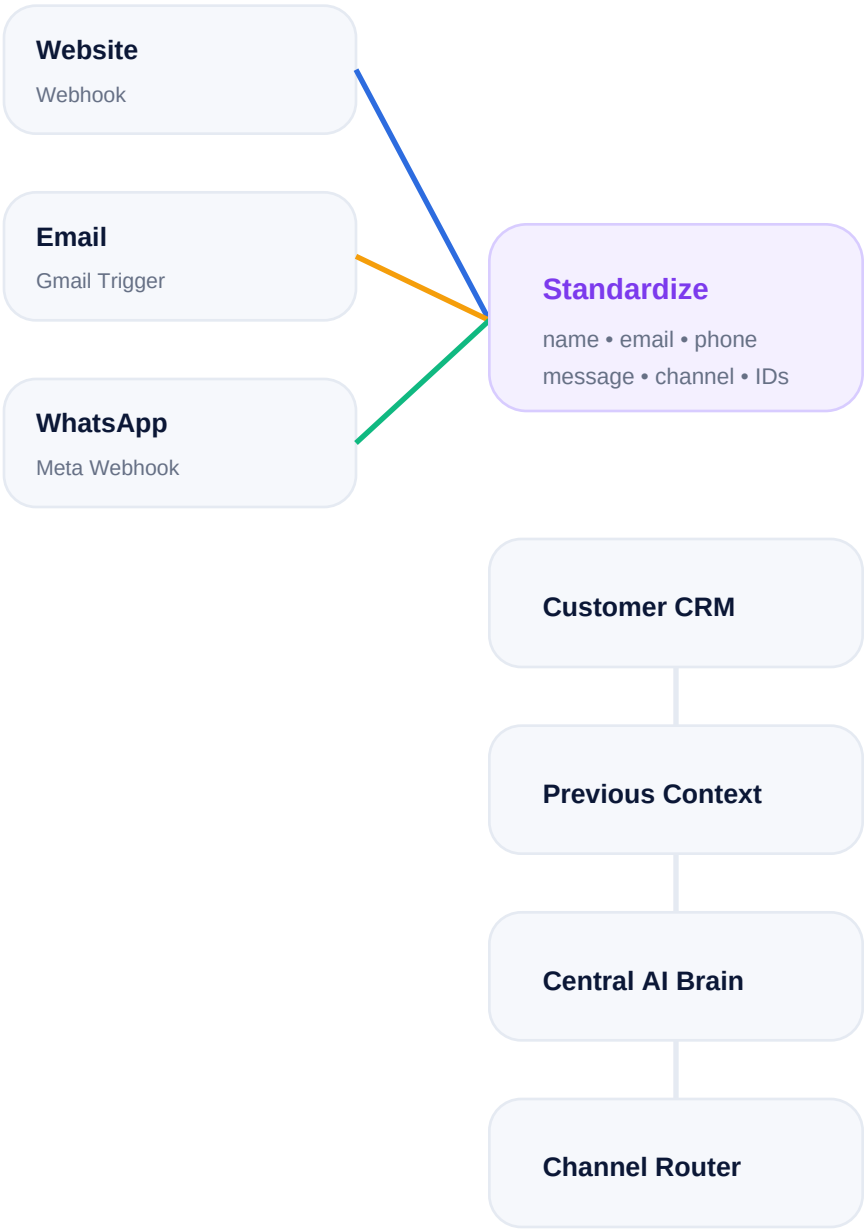
THE SOLUTION

Standardize every message, identify the customer, retrieve context, call the same Central AI Brain, route the reply, and update one simulated CRM.

ARCHITECTURE

End-to-end multi-channel pipeline

Each channel has its own trigger, but the core reasoning, customer context and routing logic are centralized.



Identity first

Find existing customer or create a new simulated CRM profile.

Context aware

Retrieve previous conversations before the AI answers.

One brain

Intent detection, action selection and knowledge retrieval stay centralized.

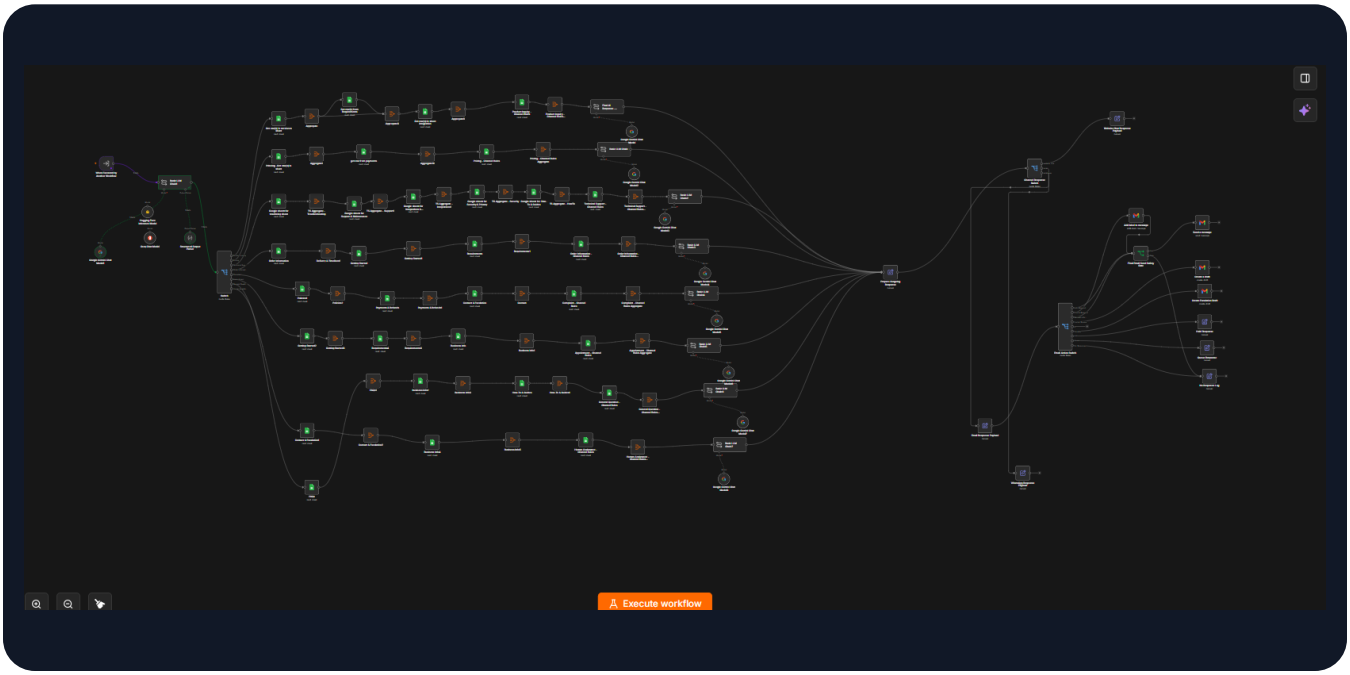
Correct channel

The final response is routed back to Website, Email or WhatsApp.

CENTRAL AI BRAIN

Structured intent + action routing

The core workflow turns raw customer messages into a structured decision before generating the final response.



Product Inquiry | Pricing | Technical Support | Order Information | Complaint | Appointment | General Question | Human Assistance

Structured decision object

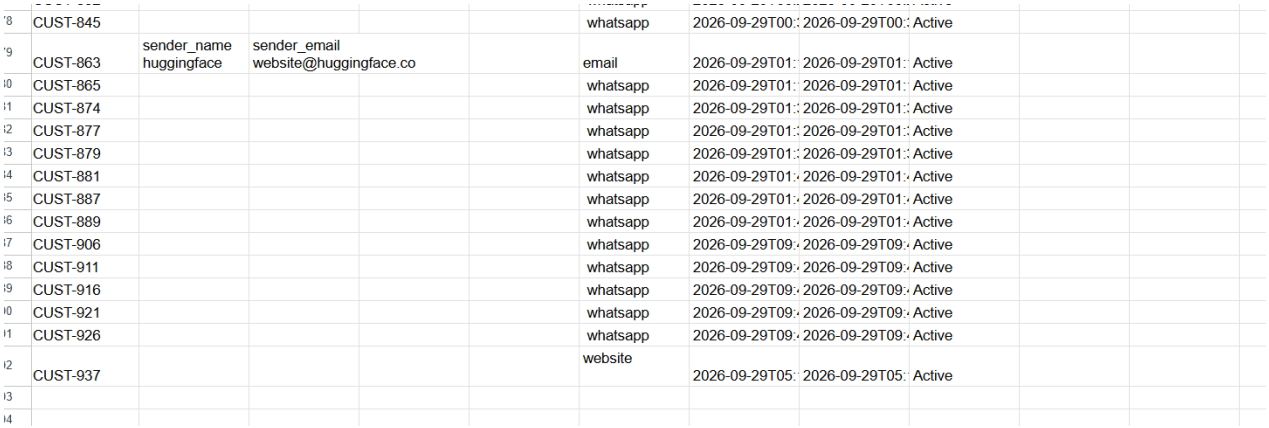
Intent • urgency • priority • risk • confidence • required information • recommended action • response type • human escalation • review state • conversation state

This makes the routing explainable and keeps channel-specific logic separate from the AI reasoning.

UNIFIED CUSTOMER CONTEXT

Simulated CRM + conversation memory

Google Sheets acts as a lightweight CRM and conversation database for the internship prototype.



8	CUST-845			whatsapp	2026-09-29T00::	2026-09-29T00::	Active
9	CUST-863	sender_name huggingface	sender_email website@huggingface.co	email	2026-09-29T01::	2026-09-29T01::	Active
10	CUST-865			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
11	CUST-874			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
12	CUST-877			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
13	CUST-879			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
14	CUST-881			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
15	CUST-887			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
16	CUST-889			whatsapp	2026-09-29T01::	2026-09-29T01::	Active
17	CUST-906			whatsapp	2026-09-29T09::	2026-09-29T09::	Active
18	CUST-911			whatsapp	2026-09-29T09::	2026-09-29T09::	Active
19	CUST-916			whatsapp	2026-09-29T09::	2026-09-29T09::	Active
20	CUST-921			whatsapp	2026-09-29T09::	2026-09-29T09::	Active
21	CUST-926			whatsapp	2026-09-29T09::	2026-09-29T09::	Active
22	CUST-937			website	2026-09-29T05::	2026-09-29T05::	Active
23							
24							

Identify — Match customer using email / phone / customer ID.

Create / Update — Append new profile or update the existing row.

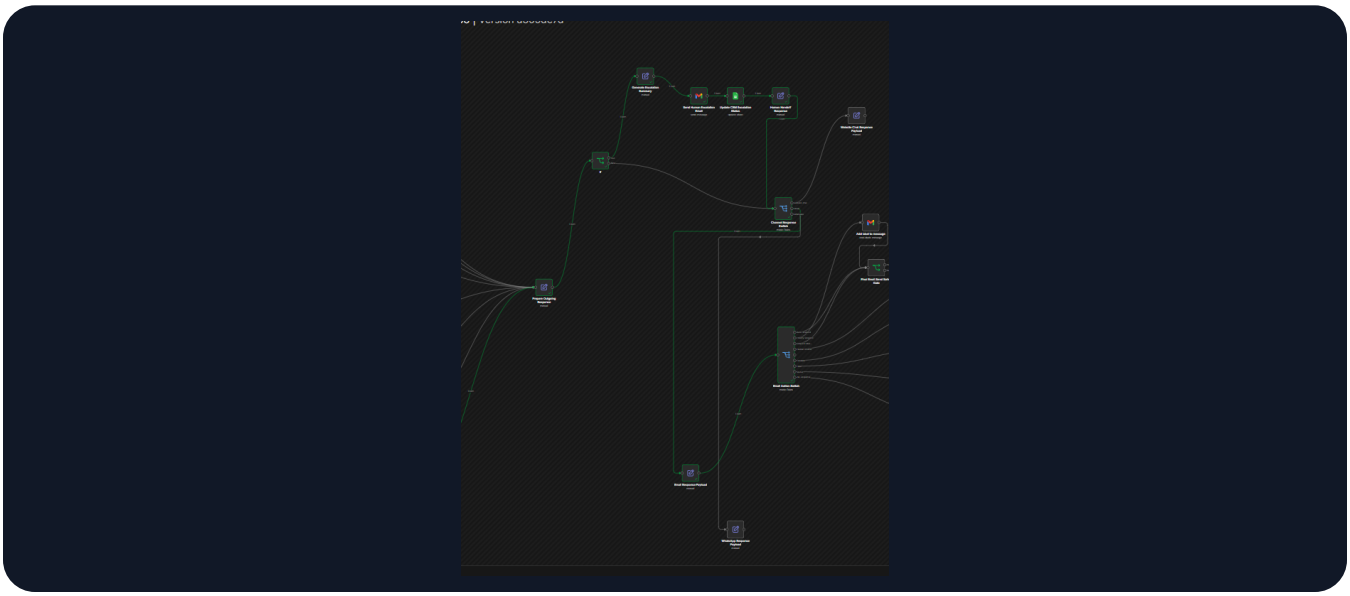
History — Retrieve previous conversations for the same customer.

Cross-channel — Use one profile when the customer switches channels.

HUMAN HANDOFF

Know when automation should stop

Complaints, sensitive requests, repeated failures, complex questions and explicit human requests can trigger escalation.



Escalation branch

Generate conversation summary → Send internal escalation email → Update CRM status → Return a safe handoff message to the customer.

CRM status: “Escalated to Human” • Normal automated answer is blocked for the escalated path.

Example decision

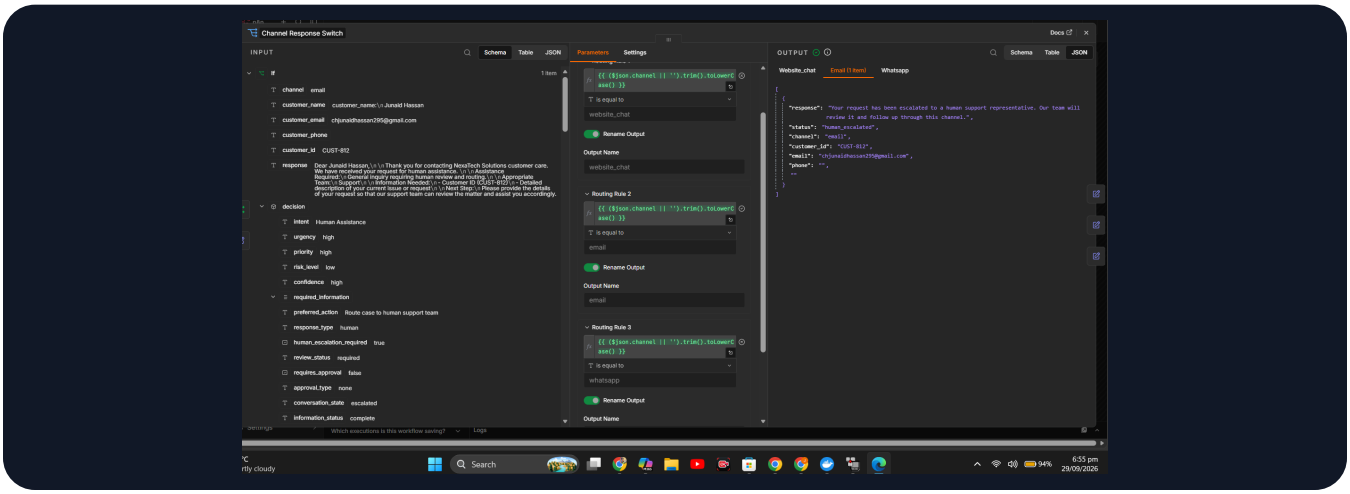
human_escalation_required = true

reply_mode = human_handoff • should_reply = false • escalation_target = Human Support Team

CHANNEL-AWARE RESPONSE ROUTING

Same AI, channel-appropriate delivery

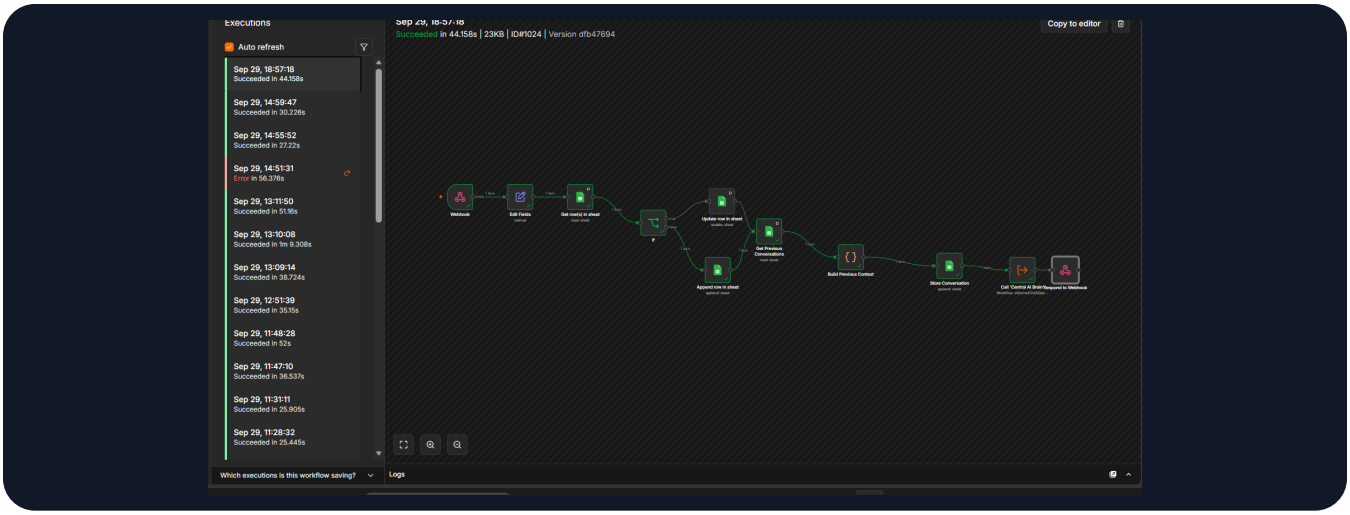
The router selects exactly one output based on the standardized channel field.



website_chat

email

whatsapp



TECHNOLOGY & FINAL QA

Built as a practical automation portfolio project

The final development phase focuses on reliability, edge cases, documentation and demonstration.

Core stack

- n8n workflow automation
- AI / LLM nodes
- Google Sheets CRM & knowledge base
- Gmail integration
- Meta WhatsApp webhook
- Website webhook + chat UI
- Docker-based local n8n
- Public tunnel for webhook testing

Phase 10 validation plan

- 30 fictional customer scenarios
- 10 Website + 10 Email + 10 WhatsApp
- New + returning customers
- Cross-channel identity continuity
- Knowledge-base missing-answer cases
- Human escalation from every channel
- Duplicate + incomplete customer data
- Error handling + workflow cleanup

What this project taught me

System design

Breaking a large automation into reusable, connected workflows.

Context engineering

Passing identity, history and structured state into the AI.

Reliability

Handling duplicates, missing data, routing errors and external service failures.

Human-in-the-loop

Designing escalation so AI knows when not to continue automatically.

PORTFOLIO PROJECT

Multi-channel AI Agent

A centralized n8n automation prototype that connects three communication channels to one AI brain, one customer context and one escalation system.

- **3 channels** — Website • Email • WhatsApp
- **Unified profile** — Simulated CRM + cross-channel history
- **Central AI** — Intent, knowledge retrieval and response generation
- **Human handoff** — Safe escalation with summary, email and CRM update

Built during an AI Automation internship as a hands-on final project.

Next step: complete the 30-scenario validation matrix, record the final three-channel demonstration, and publish the project as a documented portfolio case study.

#n8n #AIAutomation #WorkflowAutomation #WhatsAppAPI #AIProjects #PortfolioProject

Muhammad Junaid Hassan

Multi-channel AI Agent • Final Internship Project